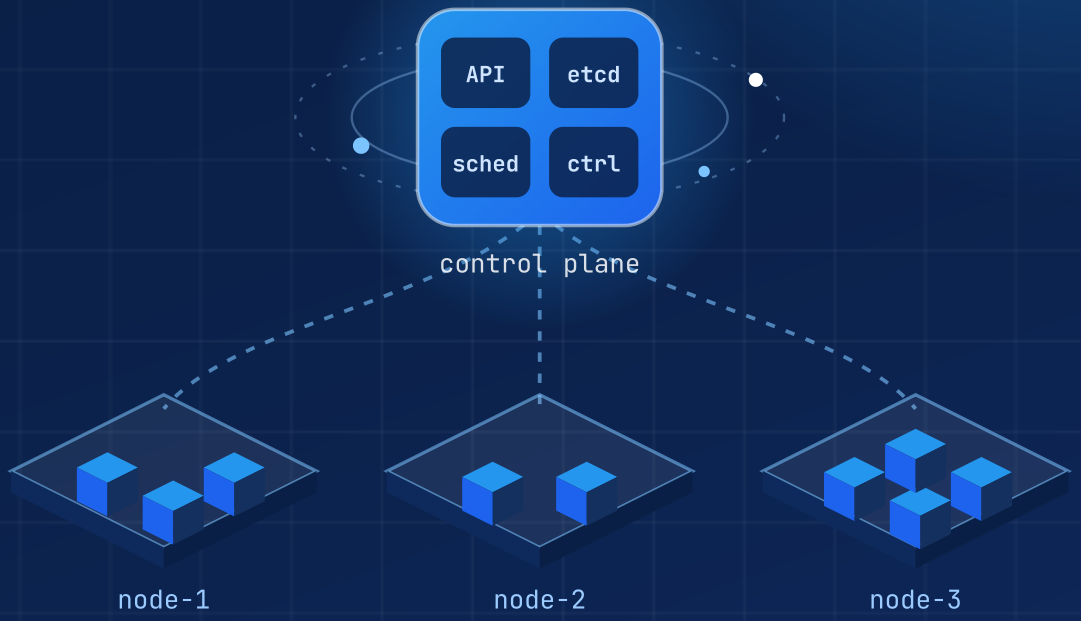
دليل مذاكرة عملي · بالعامية المصري

Kubernetes

من أول Pod لحد ما تدير Cluster كامل

الـ Cluster والـ Pods، الـ Probes والـ Limits، الـ Deployments والـ Namespaces،
الـ Services والـ Ingress والـ TLS، الـ Volumes والـ ConfigMaps والـ Secrets،
والـ RBAC والـ HPA و Helm.



`kubectl get pods -A` `kubectl apply -f deployment.yaml` `kind: Ingress` `minikube start` `kubectl rollout undo deploy/web` `helm install`

إعداد

MOAZ ABDEL-FATTAH

نسخة مُعاد تصميمها

+ فصول إضافية و kubectl Cheat Sheet

المحتويات

الكتاب ماشي على 6 مراحل بالترتيب اللي هتحتاجه وانت بتتعلم: نفهم المشكلة والـ Structure الأول، وبعدين الـ Pod من جوه، وبعدها إدارة الـ Workloads والشبكات والبيانات، وفي الآخر التشغيل الاحترافي. الفصول والأجزاء اللي عليها علامة إضافة اتضافت علشان الكتاب يبقى كامل.

إزاي تذاكر من الكتاب ده؟ دليل الألوان

- **معلومة مهمة**: حاجة لازم تفضل فاكرها
- **خلي بالك**: غلطة شائعة أو فخ
- **مشكلة شائعة**: العَرَض والسبب والحل
- **Best Practice**: الطريقة الصح في الشغل الحقيقي
- **مثال عملي**: جرّبه بإيدك على جهازك
- **ملخص + أسئلة + صح ولا غلط**: آخر كل فصل

01 المقدمة: ليه Kubernetes؟ 6
- 1.1 المشكلة: لما الـ Containers تكتر 6
- 1.2 مفاهيم جديدة: Cluster و Node و Pod 7
- 1.3 المشاكل القديمة اللي Kubernetes حلها 7
- 1.4 مميزات Kubernetes أضافها من الأصل 8

02 الـ Structure بتاع Kubernetes 10
- 2.1 شكل الـ Cluster من فوق 10
- 2.2 الإدارة العليا: الـ Control Plane 11
- 2.3 العمال والموقع: الـ Worker Nodes 11
- 2.4 آلية عمل Kubernetes: Desired State Reconciliation 12

03 الشغل نفسه: الـ Pods 14
- 3.1 يعني إيه Pod؟ 14
- 3.2 الـ Containers اللي جوه نفس الـ Pod مابينهمش أسرار 14
- 3.3 الـ Pod مؤقت (Ephemeral) 15
- 3.4 نوعين Pods على حسب تصميمك 15

04 تطبيق عملي: minikube وأول Pod 17
- 4.1 (1) خلينا نسطّب Kubernetes 17
- 4.2 (2) نشغّل الـ Cluster اللي نزّلناه 20
- 4.3 (3) نجرب نعمل كام Pod 20

05 الـ Probes: هل الـ Container بخير؟ 26
- 5.1 ليه محتاجين Probes؟ 26
- 5.2 الـ Mechanisms: هيعرف الحالة إزاي؟ 27
- 5.3 الـ Parameters: إمتى وكام مرة؟ 27
- 5.4 الأنواع التلاتة لـ Probes 28
- 5.5 الـ 3 Probes مع بعض 30

06 الـ Requests والـ Limits: موارد الـ Pod 32
- 6.1 المشكلة: موارد محدودة متشاركة 32
- 6.2 (1) الـ Limits: السقف 32
- 6.3 وحدات الموارد 33
- 6.4 (2) الـ Requests: الأرضية 33
- 6.5 الـ QoS Classes: مين يتطرد الأول؟ إضافة 34

07 الـ Labels والـ Selectors 37
- 7.1 (1) الـ Label 37
- 7.2 الفرق بين الـ name والـ Label 38
- 7.3 (2) الـ Selectors 38

08 الـ ReplicaSets: عدد ثابت من النسخ 42
8.1 يعني إيه ReplicaSet؟ 42
8.2 إيه هو Syntax الـ ReplicaSet؟ 43

09 الـ Namespaces: تقسيم الـ Cluster 46
9.1 يعني إيه Namespace؟ 46
9.2 الـ Namespaces اللي بتتعمل تلقائي 46
9.3 إيه اللي بيتحط في Namespace وإيه لأ؟ 47
9.4 أوامر الـ Namespaces 47
9.5 إزاي الـ Pods بتكلم بعضها؟ 49
9.6 الـ ResourceQuota: تحجيم الموارد للفرق 50
9.7 الـ LimitRange: القيم الافتراضية للـ Containers 51

10 الـ Deployment: إدارة الـ Pods باحتراف 53
10.1 يعني إيه Deployment؟ 53
10.2 السبب الحقيقي: الـ Update 53
10.3 الـ Structure: مين بيدير مين؟ 54
10.4 شكل الـ YAML بتاعه 54
10.5 الـ Rolling Update: تحديث من غير Downtime إضافة 55
10.6 الـ Scaling بأمر واحد إضافة 57

11 الـ DaemonSet: Pod على كل Node 59
11.1 يعني إيه DaemonSet؟ 59
11.2 شكل الـ YAML بتاعه 59
11.3 تحديد Nodes معينة 61
11.4 مشاكل شائعة 64

12 الـ Jobs والـ CronJobs: شغل يخلص ويقفل إضافة 66
12.1 يعني إيه Job؟ 66
12.2 الـ CronJob: Job على مواعيد 67

13 الـ Services: عنوان ثابت للـ Pods 71
13.1 المشكلة: الـ IP بيتغير 71
13.2 إزاي الـ Service بتشتغل فعلاً؟ 72
13.3 الأنواع الأربعة للـ Services 73
13.4 ملاحظات على أنواع الـ Services 74
13.5 إزاي الـ Pods بتلاقي بعضها؟ (DNS) 74
13.6 أوامر kubectl السريعة للـ Services 75
13.7 مشاكل وأخطاء شائعة 76

14 الـ Ingress: باب واحد لكل الـ Traffic 78
14.1 المشكلة: Load Balancer لكل حاجة 78
14.2 الـ Ingress عنده جزئين 79
14.3 تركيب الـ Ingress Controller 79
14.4 الـ Routing: الطلب ده يروح فين؟ 81
14.5 (1) على حسب الـ Path (Path-based) 82
14.6 (2) على حسب الـ Domain (Host-based) 83
14.7 مقارنة الـ Controllers وأوامر سريعة 85
14.8 مشاكل شائعة 86
14.9 الـ Gateway API: البديل الحديث للـ Ingress إضافة 87

15 الـ TLS و cert-manager: HTTPS ببلاش 89
15.1 ليه محتاجين TLS؟ 89
15.2 الـ TLS بيحقق 3 أهداف رئيسية 89
15.3 الفكرة التأسيسية: التشفير غير المتماثل 90
15.4 التطور الحديث: نظام الـ HTTPS والـ TLS 90
15.5 يعني إيه TLS Termination في الـ Ingress؟ 91
15.6 إزاي نعمل الموضوع ده بـ cert-manager 91
15.7 إيه اللي هيحصل في الخلفية؟ 93

94 15.8 مشاكل شائعة

16 الـ Volumes والتخزين: البيانات اللي متضيعش إضافة 96
96 16.1 المشكلة: الـ Container ذاكرته قصيرة
96 16.2 أنواع الـ Volumes الأساسية
98 16.3 التخزين الدائم: PV و PVC
100 16.4 لمحة عن الـ StatefulSet إضافة

17 الـ ConfigMaps: الإعدادات برّه الكود 103
103 17.1 الفكرة: متغيرات في مكان واحد
104 17.2 إنشاء ConfigMap من الـ Terminal
105 17.3 ConfigMap بالـ YAML إضافة
105 17.4 إزاي الـ Pod ياخد القيم؟ إضافة
106 17.5 لو عدّلت الـ ConfigMap، إيه اللي يحصل؟ إضافة

18 الـ Secrets: البيانات الحساسة إضافة 108
108 18.1 يعني إيه Secret؟
109 18.2 إنشاء Secret من الـ Terminal
110 18.3 Secret بالـ YAML
110 18.4 استخدام الـ Secret في الـ Pod
111 18.5 تأمين الـ Secrets بجد

19 الـ RBAC: مين يقدر يعمل إيه؟ 113
113 19.1 إيه هي الـ RBAC؟ مين يقدر يعمل إيه جوه الـ Namespace؟
113 19.2 الـ Role بالتفصيل إضافة
114 19.3 الـ RoleBinding إضافة
115 19.4 ClusterRole و ClusterRoleBinding إضافة
115 19.5 الـ ServiceAccount: هوية الـ Pod إضافة
116 19.6 اتأكد من الصلاحيات: kubectl auth can-i إضافة

20 الـ HPA: الـ Scaling الأوتوماتيك إضافة 118
118 20.1 يعني إيه HPA؟
118 20.2 محتاج إيه عشان يشتغل؟
119 20.3 المعادلة
119 20.4 إنشاء HPA بالأمر
119 20.5 إنشاء HPA بالـ YAML
120 20.6 تابعه وجرّبه

21 الـ Troubleshooting: إزاي تحل أي مشكلة إضافة 122
122 21.1 صندوق العدة: أوامر التشخيص
123 21.2 قاموس الحالات (STATUS)
124 21.3 لو المشكلة في الـ Service
125 21.4 الـ kubectl debug
125 21.5 خريطة التشخيص

22 الـ Helm: مدير الحزم لـ Kubernetes 127
127 22.1 المشكلة اللي Helm بيحلها
127 22.2 المصطلحات الأساسية إضافة
128 22.3 دورة حياة الـ Release إضافة
129 22.4 جوه الـ Chart إضافة

أ Cheat Sheet: أوامر kubectl و Helm 131
133 أ.1 مرجع الـ Flags
134 أ.2 مرجع العلامات (Symbols)

ب قاموس المصطلحات 135

ج إجابات أسئلة المراجعة و صح ولا غلط 138

خريطة الكتاب

قبل ما تبدأ، بص على الطريق كله مرة واحدة. كل محطة بتبني على اللي قبلها، فلو حسيت إن حاجة مش واضحة ارجع للمحطة اللي قبلها. الأرقام اللي جوه الدوايز هي أرقام الفصول.



الطريق من أول فكرة الـ Orchestration لحد التشغيل الاحترافي.

1 · الأساسيات

ليه Kubernetes، والـ Structure بتاع الـ Cluster، والـ Pod، وأول تجربة عملية على minikube.

2 · الـ Pod من جوه

إزاي Kubernetes يعرف إن الـ Container سليم (Probes)، وإزاي نحدد موارده (Requests و Limits).

3 · إدارة الـ Workloads

الـ Labels، والـ ReplicaSets، والـ Namespaces، والـ Deployment، والـ DaemonSet، والـ Jobs.

4 · الشبكات

الـ Services وأنواعها، والـ Ingress والـ Routing، والـ TLS بشهادات مجانية من Let's Encrypt.

5 · البيانات والإعدادات

الـ Volumes والـ PV والـ PVC، والـ ConfigMaps، والـ Secrets.

6 · التشغيل الاحترافي

الصلاحيات (RBAC)، والـ Autoscaling (HPA)، والـ Troubleshooting بـ kubectl، و Helm.

Best Practice

أحسن طريقة تذاكر بيها: اقرا الفصل، جرّب الأوامر بنفسك على minikube، وبعدين جاوب على **أسئلة المراجعة** و **صح ولا غلط** في آخره من غير ما تبص، وقارن إجابتك بالملحق.

01

المرحلة 1 · الأساسيات

الفصل الأول

المقدمة: ليه Kubernetes؟

Container واحد سهل تديره بإيدك. بس لما يبقوا 100 أو 1000 بتظهر مشاكل محتاجة حد ينظم الحركة كلها. الحد ده اسمه Orchestrator، و Kubernetes هو الأشهر فيهم.

بعد الفصل ده هتكون عارف

- المشاكل اللي بتظهر لما الـ Containers تكتر
- يعني إيه Orchestrator ودور Kubernetes
- يعني إيه Cluster و Node و Pod
- المشاكل القديمة اللي Kubernetes حلها
- مميزات Kubernetes زي الـ Declarative Model

1.1 المشكلة: لما الـ Containers تكتر

تخيل لو عندك Container واحد، الموضوع سهل. لكن لو بقوا 100؟ أو 1000؟ هنا بتظهر مشاكل:

مين يقومه؟

لو Container وقع، مين يقومه؟

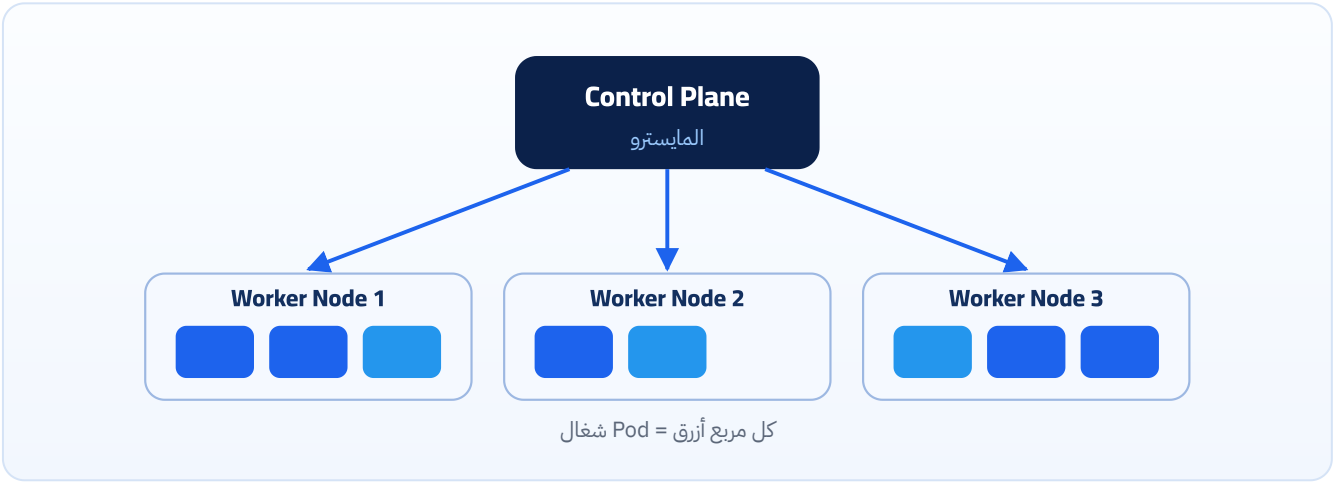
السيرفر باظ؟

لو السيرفر (Host) نفسه باظ، الـ Containers اللي عليه هتروح فين؟

الأحمال

إزاي نوزع الأحمال (Traffic) ما بينهم؟

هنا بييجي دور المنسق (**Orchestrator**)، Kubernetes هو **"المايسترو"** اللي بينظم الحركة دي كلها، بيتأكد إن الـ Containers شغالة، ولو حاجة وقعت بيشغل غيرها فوراً.



شكل 1.1 زي المايسترو في الأوركسترا: الـ Control Plane بيقود، والـ Worker Nodes (العازفين) بتنفّذ، والجمهور هو الـ Users والـ Traffic.

1.2 مفاهيم جديدة: Cluster و Node و Pod

عشان Kubernetes يقدر يحل المشاكل دي، قدم لنا شوية مفاهيم جديدة.

زمان كنت بترفع الـ Containers على جهاز واحد أو أكتر وتحتاس في إدارتهم وتوصيلهم ببعض. Kubernetes حل ده بإنشاء حاجة اسمها **Cluster**، وهي ببساطة إنك بتوصل مجموعة الأجهزة والسيرفرات دي ببعض عشان تحولهم لـ **"كمبيوتر واحد عملاق"**. ده بيخليك تاخد أقصى استفادة من الـ Hardware المتاح في كل الأجهزة في نفس الوقت، وتقدر توزع الـ Containers والأحمال (Traffic) عليهم بذكاء.

ومع النظام الجديد ده، Kubernetes غيّر شوية مسميات عشان تناسب شغله:

Node

بدل ما بنقول "جهاز" أو "مكنة"، بقى اسمها **Node** طالما دخلت واشتغلت جوه الـ Cluster.

Pod

وبدل ما بنتعامل مع كل Container لوحده بشكل مباشر، Kubernetes عمل كبسولة أو غلاف اسمه **Pod**.

معلومة مهمة

الـ Pod هو أصغر وحدة Kubernetes بيتعامل معاها، وبنحط جواه الـ Container الأساسي أو معاه Container تاني مساعد ليه، والـ Pod ده بيوفرلهم بيئة مشتركة (زي شبكة واحدة ومساحة تخزين واحدة) يشتغلوا فيها مع بعض.

1.3 المشاكل القديمة اللي Kubernetes حلها

يعني إيه Cloud و AWS و EC2؟

الـ **Cloud** = إنك تأجّر سيرفرات من شركة كبيرة بالساعة بدل ما تشتريها وتحطها عندك. أشهرهم **AWS** (بتاعة Amazon) و Google Cloud و Azure. و **EC2** اسم خدمة السيرفرات الافتراضية في AWS. وعكسها **On-premises**: السيرفرات عندك في الشركة. Kubernetes بيشتغل على الاتنين، وفي الكتاب ده هنجرّب على جهازك بـ minikube.

المشكلة قبل Kubernetes	الحل
تشغيل Container وقع؟ **حد لازم يصحى يعمله restart.**	الـ kubelet والـ Controller بيعملوا restart أوتوماتيك.
زحمة على السيرفر؟ **حد يشتري سيرفر جديد ويعمل setup.**	الـ Scheduler بيحط الـ Pods على Nodes فيها مكان. بل كمان لو مفيش مكان وانت رابطه بسيرفرات على AWS، بيكلم الموقع ويزود سيرفرات (EC2) زيادة (عن اللي انت واخدهم منهم بالفعل) عشان يستحمل الضغط ده، ولما الضغط ده يروح يقفلهم ويلغيهم، وساعتها هتدفع على قد استهلاكك للـ EC2 الجديدة دي خلال المدة اللي استخدمتهم فيها بس (كأنك أجّرت سيرفرات زيادة للمدة اللي كان فيها الضغط بس، وبعد كده لغيت عقد الإيجار ده).
سيرفر بكامله وقع؟ **كل الـ Containers اللي عليه راحت.**	بيشيل الـ Pods ويعيد جدولتها على Nodes تانية.

عايز توزع الـ Traffic على كذا Container؟ هتتعب في تظبيط nginx بإيدك.	Kubernetes عنده Services بتتعامل مع ده أوتوماتيك.
الـ Containers تتكلم مع بعض؟ الـ IP بيتغير كل ما تعيد تشغيل.	الـ Service Discovery بيدي كل خدمة اسم ثابت.

جدول 1.1 قبل Kubernetes وبعده

> **دقة: مين اللي بيزود السيرفرات؟**
>
> زيادة عدد السيرفرات نفسها (الـ Nodes) مش بتحصل لوحدها في أي Cluster. ده شغل مكوّن إضافي اسمه **Cluster Autoscaler** (أو Karpenter على AWS) لازم تركّبه وتربطه بحساب الـ Cloud. وزيادة عدد الأجهزة كده اسمها في الغالب **Scale Out** (أفقي)، أما **Scale Up** فمعناها الأدق إنك تكبّر موارد نفس الجهاز. الفكرة اللي في الجدول صح، بس محتاجة الإضافة دي.

1.4 مميزات Kubernetes أضافها من الأصل

الميزة	يعني إيه؟
الـ Declarative Model	الفرق مش بس أوتوماتيكي. إنت بقيت بتوصف الهدف مش الخطوات. بتكتب `replicas: 5` وهو يفهم إزاي يوصل لده ويحافظ عليه.
الـ Portability الحقيقية	الـ YAML بتاعك بيشتغل على أي Cloud أو On-premises من غير ما تغير حاجة. الـ Infrastructure بقت abstractable.
الـ Storage Orchestration	ربط الـ Container بـ Storage Volumes بيتم ديناميكياً، سواء كان local disk أو AWS EBS أو NFS، من غير ما الـ Container يعرف الفرق.
الـ Batch Execution المنظمة	تقدر تشغل Job يخلص ويقفل، أو CronJob على schedule، بنفس الـ Infrastructure بدون أي setup إضافي.

جدول 1.2 مميزات مش بس "أوتوماتيك"

> **معلومة مهمة**
>
> فيه مصطلحات وكلمات ممكن متكونش واضحة أوي دلوقتي، هتتوضح بعدين، فمتقلقش. ولو نسيت معنى أي كلمة، ارجع لـ **قاموس المصطلحات** في آخر الكتاب.

ملخص الفصل

- لما الـ Containers تكتر، محتاجين حد يقوّم اللي يقع، وينقل الشغل لو سيرفر باظ، ويوزع الـ Traffic.
- Kubernetes هو الـ Orchestrator (المايسترو) اللي بيعمل ده.
- الـ Cluster: مجموعة أجهزة متوصلة كأنها كمبيوتر واحد. كل جهاز فيه اسمه Node.
- الـ Pod: أصغر وحدة Kubernetes بيتعامل معاها، وجواها Container أو أكتر.
- أهم ميزة: الـ Declarative Model، إنت بتوصف الهدف وهو يحافظ عليه.

؟ أسئلة مراجعة

1. اذكر 3 مشاكل بتظهر لما يبقى عندك مئات الـ Containers.
2. يعني إيه Cluster؟ وإيه الفرق بين الـ Node والـ Pod؟
3. إيه معنى الـ Declarative Model؟ اديني مثال.
4. مين اللي بيزود عدد السيرفرات في الـ Cloud لما الضغط يزيد؟

صح ولا غلط؟

1. Kubernetes بيتعامل مع كل Container لوحده بشكل مباشر. صح / غلط
2. الـ YAML بتاع Kubernetes ممكن يشتغل على أكتر من Cloud من غير تغيير. صح / غلط
3. أي Cluster بيزود عدد الـ Nodes لوحده من غير أي إعداد. صح / غلط

02

المرحلة 1 · الأساسيات

الفصل التاني

الـ Structure بتاع Kubernetes

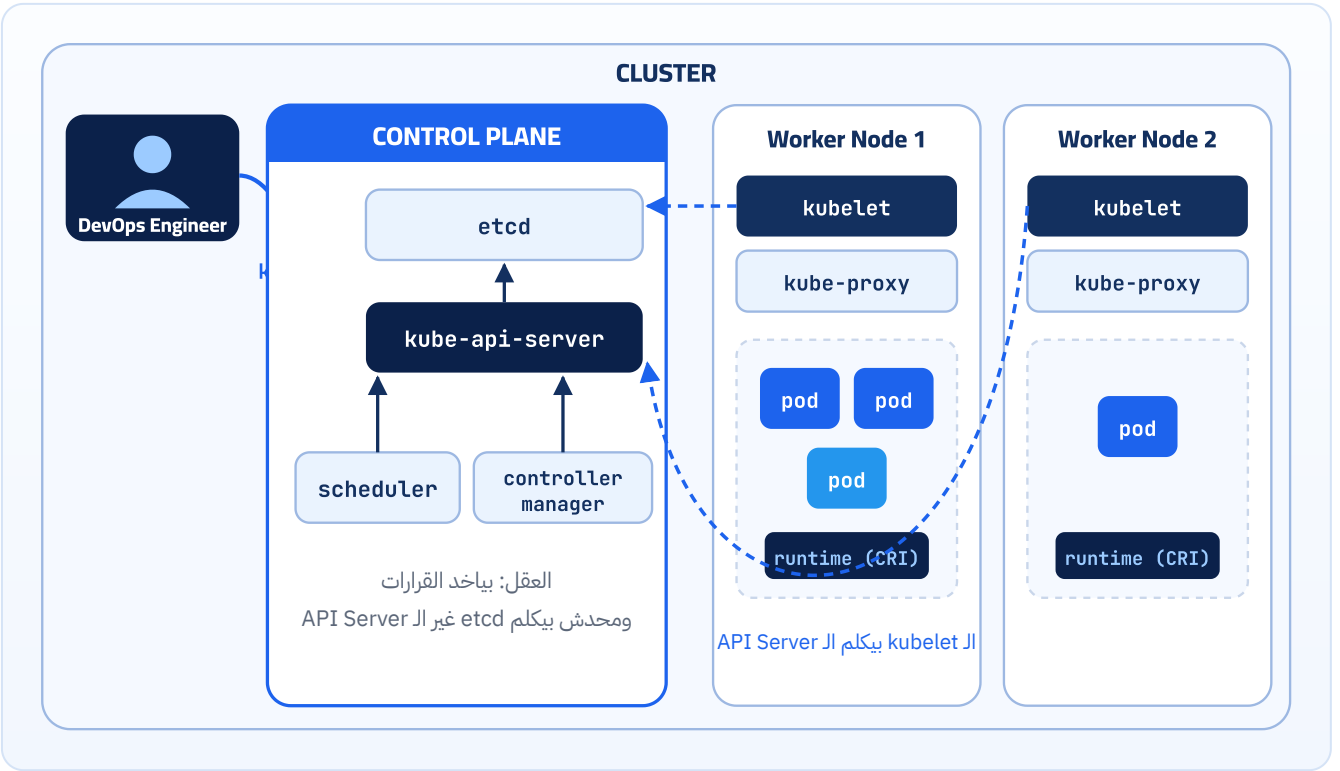
لما بنشغّل Kubernetes على مشروع، هو بيبني Cluster واحد أو مجموعة Clusters يقوم عليهم المشروع، فلازم نفهم تكوين الـ Cluster. تخيل إنه عبارة عن "شركة مقاولات": فيها إدارة عليا بتاخد القرارات، وعمال في الموقع بينفّذوا.

بعد الفصل ده هتكون عارف

- مكونات الـ Control Plane ودور كل واحد
- مكونات الـ Worker Node
- يعني إيه Container Runtime
- آلية عمل Kubernetes: Desired State Reconciliation

2.1 شكل الـ Cluster من فوق

لما بنيجي نشغّل Kubernetes على مشروع، فهو بيبني Cluster واحد أو مجموعة Clusters يقوم عليهم المشروع، فلازم نفهم تكوين الـ Cluster. تخيل إنه عبارة عن **"شركة مقاولات"**: فيها **إدارة عليا** (الـ Control Plane) و**عمال وموقع** (الـ Worker Nodes). وإنت كـ DevOps Engineer بتكلم الإدارة عن طريق أداة اسمها `kubectl`.



شكل 2.1 الـ Structure بتاع الـ Cluster: Control Plane بيدير، و Worker Nodes عليها الـ Pods، وإنت بتكلم الـ API Server بـ kubectl.

2.2 الإدارة العليا: الـ Control Plane

ده **"عقل"** السيستم. هو اللي بياخد القرارات، بيراقب الوضع، وبيوزع الشغل. بيتكون من 4 أجزاء:

مكتب الاستقبال (API Server)

ده الواجهة بتاعة السيستم. أي حد عايز حاجة من Kubernetes (سواء إنت كمبرمج بتكتب أوامر، أو حتة تانية في السيستم بتكلمه) لازم يعدي على الـ API Server الأول. هو اللي بيستقبل الطلبات ويتأكد منها ويوزعها.

خزنة الأسرار والذاكرة (etcd)

دي Database سريعة جداً وخفيفة. Kubernetes بيسجل فيها كل تفصيلة عن السيستم: (إيه اللي شغال دلوقتي؟ إيه اللي المفروض يشتغل؟ مين وقع؟). لو الـ etcd دي باظت، السيستم كله بيفقد الذاكرة.

مدير الموارد (Scheduler)

تخيل إنك طلبت تشغّل أبلكيشن جديد. الـ Scheduler ده هو اللي بيبص على الـ Nodes، ويشوف مين فيهم فاضي، راماته مستريحة، ومناسب للشغلانة دي، ويبعتله الشغل. بيستغل الموارد صح عشان ميحمّلش على Node ويسيب التانية.

المشرف / المراقب (Controller Manager)

ده راجل بيطابق "الواقع" باللي "المفروض يحصل". يعني لو إنت قايل للسيستم "أنا عايز 5 نسخ من الأبلكيشن شغالين طول الوقت"، وفجأة سيرفر وقع ونسختين ماتوا.. المشرف ده بيلاحظ فوراً إن العدد بقى 3، ويقوم مدّي أمر فوري: "شغّلوا نسختين كمان بسرعة في أي حتة تانية عشان نرجّعهم 5!".

Best Practice

علشان الـ etcd هي ذاكرة الـ Cluster كله، في الـ Production بيتعمل لها **Backup** بشكل دوري، وبيتشغّل منها أكتر من نسخة (غالباً 3) علشان لو واحدة وقعت السيستم ميفقدش الذاكرة.

2.3 العمال والموقع: الـ Worker Nodes

دي السيرفرات الفعلية (سواء كانت Physical أو Virtual) اللي بتشيل الأبلكيشن بتاعك وترن الكود. كل مكنة (Node) عليها 3 حاجات أساسية عشان تعرف تشتغل مع المايسترو:

رئيس العمال (Kubelet)

ده عميل (Agent) صغير كده متسطب على كل مكنة. وظيفته إنه يسمع كلام الـ API Server (المايسترو). لما المايسترو يقوله "شغّل كذا عندك"، هو بينفذ، وبيبعت تقارير مستمرة للإدارة يقولهم "أنا تمام والشغل اللي عندي شغال زي الفل".

عسكري المرور (Kube-proxy)

ده المسئول عن الشبكات (Networking) جوه المكنة. هو اللي بيظبط قواعد التوجيه وبيفتح السكك عشان الأبلكيشنز تعرف تكلم بعضها، ويتأكد إن الزوار اللي جايين من بره بيروحوا للمكان الصح من غير ما يتوهوا.

الموتور (Container Runtime)

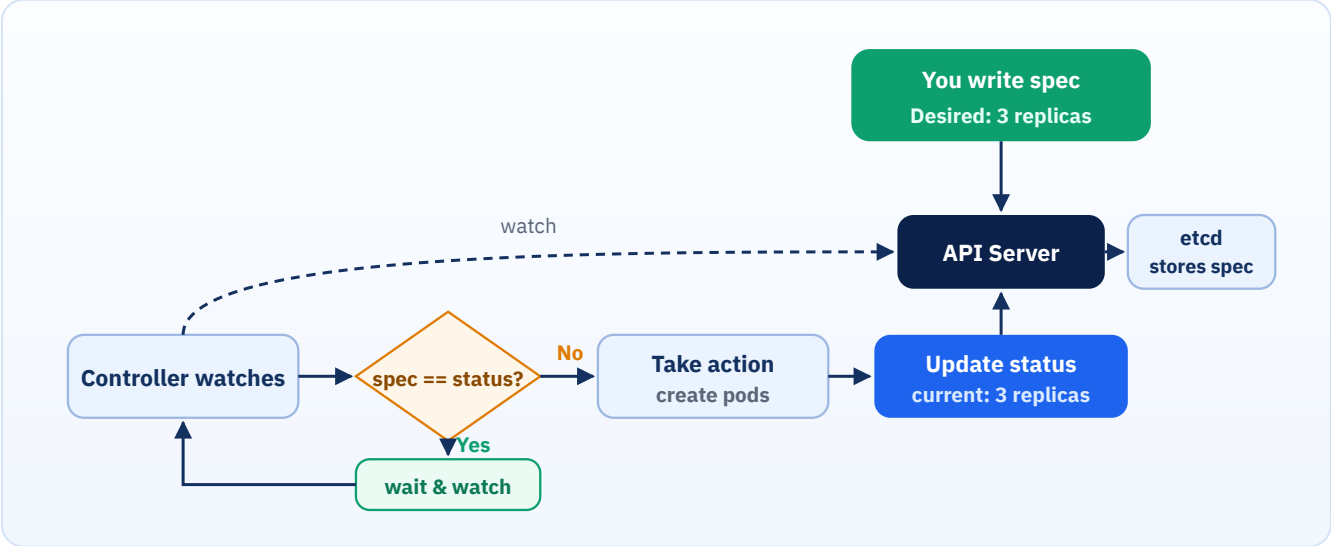
Kubernetes مبيقدرش يـ run الـ Containers بنفسه، ولكن بيوكّل المهمة دي لبرنامج تاني ألا وهو الـ Container Runtime، بيكون بيعرف يقرا ويشغّل الـ Containers ويتعامل معاهم. زي Docker أو containerd.

دقة: Docker و kube-proxy

- الـ IP بتاع كل Pod مش kube-proxy اللي بيدّيه، ده شغل الـ **CNI plugin** (زي Calico أو Flannel أو Cilium). kube-proxy بيظبط قواعد الشبكة (iptables أو IPVS) اللي بتوصّل الـ Traffic الجاي على الـ Services للـ Pods الصح.
- من Kubernetes 1.24 الدعم المباشر لـ Docker Engine كـ Runtime اتشال (اللي كان اسمه dockershim). الـ Runtimes المستخدمة النهارده هي **containerd** و **CRI-O**. الـ Images اللي بتبنيها بـ Docker لسه شغالة عادي جداً لأنها بنفس الـ Standard (OCI).

2.4 آلية عمل Kubernetes: Desired State Reconciliation

الـ Kubernetes بيشتغل على مبدأ **Desired State Reconciliation**: إنت بتقوله "أنا عايز 5 نسخ من الأبلكيشن" وهو يفضل يحارب عشان يحقق ده ويحافظ عليه أوتوماتيك.



شكل 2.2 الـ Reconciliation Loop: الـ Controller بيقارن الـ spec (المطلوب) بالـ status (الواقع) طول الوقت، ولو مختلفين بيتحرك.

مثال من الحياة

زي التكييف بالظبط: إنت بتحدد درجة الحرارة اللي عايزها (الـ Desired State = 22°). التكييف بيقيس الحرارة الحالية طول الوقت، ولو لقاها 26° يشتغل لحد ما توصل 22، ولو وصلت يهدى ويفضل يراقب. إنت مقولتلوش "اشتغل 10 دقايق"، إنت وصفتله النتيجة بس.

ملخص الفصل

- الـ Cluster = Control Plane (الإدارة) + Worker Nodes (العمال).
- الـ Control Plane: API Server (الاستقبال)، etcd (الذاكرة)، Scheduler (توزيع الشغل)، Controller Manager (المطابقة بين المطلوب والواقع).
- كل Worker Node عليها: kubelet و kube-proxy و Container Runtime.
- Kubernetes شغال بالـ Desired State Reconciliation: بيقارن المطلوب بالواقع طول الوقت ويصلّح الفرق.

؟ أسئلة مراجعة

1. إيه الجزء اللي بيكلمه `kubectl` لما تكتب أمر؟
2. لو الـ etcd باظت من غير Backup، إيه اللي هيحصل؟
3. مين اللي بيختار الـ Node اللي الـ Pod هيشتغل عليها؟ ومين اللي بيشغّله فعلاً؟
4. اشرح الـ Reconciliation Loop في 3 خطوات.

صح ولا غلط؟

1. الـ kubelet موجود على كل Worker Node. صح / غلط
2. Kubernetes بيشغّل الـ Containers بنفسه من غير Runtime. صح / غلط
3. الـ Scheduler هو اللي بيعمل restart للـ Pods اللي وقعت. صح / غلط

المرحلة 1 · الأساسيات

الفصل التالت

الشغل نفسه: الـ Pods

الـ Pods مهمة جداً في Kubernetes، لأنه مبيتعاملش مع الـ Container لوحده مباشرة، وإنما بيحط الـ Container أو أكتر من Container ليهم علاقة ببعض جوه كبسولة اسمها الـ Pod.

بعد الفصل ده هتكون عارف

- يعني إيه Pod وليه هو أصغر وحدة
- الـ Containers اللي جوه نفس الـ Pod بيشاركوا إيه
- ليه الـ Pod مؤقت (Ephemeral)
- الفرق بين Single-Container و Multi-Container Pod

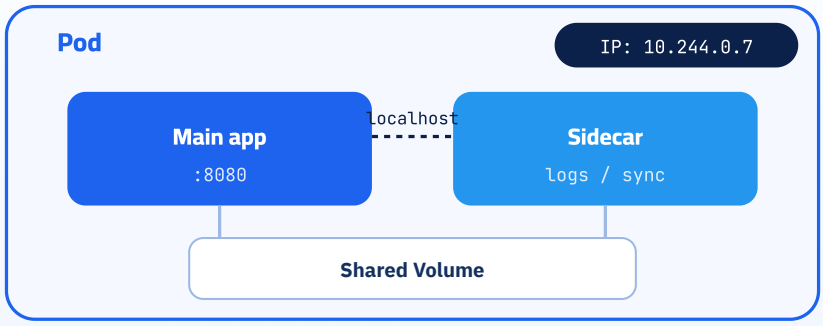
3.1 يعني إيه Pod؟

الـ Pods مهمة شوية في Kubernetes، لأنه مابيتعاملش مع الـ Container لوحده مباشرة، وإنما بيحط الـ Container أو أكتر من Container ليهم علاقة ببعض جوه كبسولة اسمها الـ Pod.

- الـ Pod هو أصغر وأبسط وحدة Kubernetes بيتعامل معاها، وهو بيمثل **"نسخة واحدة شغالة"** (Single instance) من تطبيقك جوه الـ Cluster. لو عايز يعمل Scale ويزود النسخ، هو بيكبّر الـ **Pods** مش الـ Containers الفردية.

3.2 الـ Containers اللي جوه نفس الـ Pod مابينهمش أسرار

لو الـ Pod جواه أكتر من Container، كلهم بيشاركوا بعض في كل حاجة:



شكل 3.1 الـ Containers جوه نفس الـ Pod بياخدوا نفس الـ IP، ويكلموا بعض بـ localhost، ويشاركوا نفس الـ Volumes.

بيشاركوا في	يعني إيه؟
الشبكة (Networking)	بياخدوا نفس الـ IP address ونفس مساحة المنافذ (Port space)، وبيقدروا يكلموا بعض ببساطة عن طريق `localhost`.
المساحة التخزينية (Storage volumes)	بيشاركوا نفس وحدات التخزين، يعني لو Container كتب ملف، الـ Container التاني اللي معاه في نفس الـ Pod يقدر يشوفه ويقرأه.
دورة الحياة (Lifecycle)	بيموتوا ويعيشوا مع بعض.

جدول 3.1 الـ Containers جوه الـ Pod بيشاركوا في إيه؟

> **خلي بالك**
> بما إنهم بيشاركوا نفس مساحة المنافذ، مينفعش اتنين Containers في نفس الـ Pod يسمعوا على نفس الـ Port. لو الاتنين عايزين 80 هيحصل تعارض.

3.3 الـ Pod مؤقت (Ephemeral)

الـ Pod بيكون **Ephemeral**، يعني مش عايش للأبد. لو حصلت مشكلة والـ Pod مات، مش بيعمل Restart لنفسه تلقائياً، إلا لو كان مِتدار عن طريق حاجة أكبر منه (**Controller**) زي الـ Deployment، وهو ده اللي بيضمن إن لو Pod مات، يقوم واحد جديد مكانه.

> **توضيح مهم**
> فيه فرق بين حاجتين: لو **Container** جوه الـ Pod وقع، الـ kubelet بيعمله restart جوه نفس الـ Pod (حسب الـ `restartPolicy` اللي الافتراضي بتاعها `Always`). لكن لو **الـ Pod نفسه** اتمسح أو الـ Node اللي عليها وقعت، الـ Pod ده مش هيرجع تاني لوحده، وهنا بييجي دور الـ Controller (زي الـ Deployment) إنه يعمل Pod جديد بدل منه.

3.4 نوعين Pods على حسب تصميمك

Single-Container Pod

وده الشكل الأشهر والأكتر استخداماً، إن كل Pod يكون جواه Container واحد بس، شايل وظيفة واحدة محددة (زي ويب سيرفر، أو API، أو Worker). وده تبعاً لمبدأ **"حاوية واحدة لكل عملية"**.

Multi-Container Pod

هنا بقى بنحط حاويتين أو أكتر متصلين ببعض جامد (Tightly coupled) عشان يخدموا على بعض. مثال مشهور: يكون عندك Container أساسي شغال عليه الموقع وبيقدّم الخدمة (Main app)، ومعاه Container مساعد صغير جنبه (**Sidecar**) وظيفته يلم الـ Logs أو يعمل Sync للملفات بس.

- بالشكل ده، الـ Kubernetes بيضمن إن الحاجات اللي لازم تشتغل مع بعض وتخدم على بعض تفضل دايماً في مكان واحد (جوه الـ Pod).
- بالإضافة لكل ده، فـ Kubernetes عنده Services كتيرة بيقدمها بتساعده في تشغيل الـ Structure ده بأفضل شكل.

ملخص الفصل

- الـ Pod أصغر وحدة في Kubernetes، وبيمثل نسخة واحدة من تطبيقك.
- الـ Scaling بيكون بزيادة عدد الـ Pods، مش الـ Containers جوه الـ Pod.
- الـ Containers في نفس الـ Pod بيشاركوا الـ IP والـ Ports والـ Volumes ودورة الحياة.
- الـ Pod مؤقت، واللي بيضمن إن فيه بديل هو Controller زي الـ Deployment.
- أغلب الوقت Container = Pod واحد، والـ Sidecar للحالات المترابطة جامد.

أسئلة مراجعة

1. لو عايز 3 نسخ من الـ API بتاعك، هتزود Containers جوه الـ Pod ولا Pods؟ وليه؟
2. Container بيكلم التاني في نفس الـ Pod إزاي؟
3. اديني مثال على Sidecar Container.
4. إيه الفرق بين إن Container يقع، وإن Pod يتمسح؟

صح ولا غلط؟

1. كل Container جوه نفس الـ Pod بياخد IP مختلف. صح / غلط
2. Pod اتعمل لوحده (من غير Controller) واتمسح، Kubernetes هيعمل واحد بداله. صح / غلط
3. الشكل الأشهر هو Pod فيه Container واحد. صح / غلط

04

المرحلة 1 · الأساسيات

الفصل الرابع

تطبيق عملي: minikube وأول Pod

خلينا نشوف تطبيق عملي بسيط على كل اللي فات ده. طبعاً إحنا هنجرب على VM بسيطة، فهتكون هي الـ Node بتاعتنا وهتكون شايلة الـ Cluster كله، يعني الـ Control node والـ Worker node هيبقوا على نفس الـ VM.

بعد الفصل ده هتكون عارف

- تسطيب Docker و kubectl و minikube على Ubuntu
- تشغيل Cluster بـ minikube start
- عمل Pod من ملف YAML
- عمل Pod من Image جاهزة بـ kubectl run
- قراية خانات READY و STATUS

4.1 (1) خلينا نسطّب Kubernetes

هنستخدم نسخة مصغرة من Kubernetes عشان نطبق عليها، ألا وهي **minikube**، بتكون عبارة عن fake cluster جواه Node واحدة شايلة الـ Cluster كله.

> ⚠ **على الماك: 3 أوامر بس**
>
> الخطوات اللي تحت مكتوبة لسيرفر Ubuntu (بـ `apt`، مدير الحزم بتاع Ubuntu زي brew عندنا). على الماك مش محتاجها، وده المقابل:

macOS

```
brew install --cask docker      # Docker Desktop (افتحه مرة وسيبه شغال)
brew install kubectl minikube   # الأداتين
minikube start --driver=docker  # شغّل الـ Cluster جوه Docker
```

وبعد كده كل أوامر `kubectl` و `minikube` في الكتاب زي ما هي بالظبط. والـ `moaz@Linux:~$` اللي في الأمثلة مجرد شكل الـ Prompt على جهاز الكاتب.

1 Install Dependencies & Update System

Step 1

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl wget apt-transport-https
```

تشريح الأمر

```
1    2    3    4          5
sudo apt install -y curl wget apt-transport-https
```

1	`sudo`	نفّذ الأمر بصلاحيات الـ root (الأدمن).
2	`apt`	مدير الحزم (Package Manager) في Ubuntu/Debian.
3	`install`	نزّل وسطّب الحزم اللي بعده.
4	`-y`	جاوب "yes" أوتوماتيك على أي سؤال تأكيد.
5	`curl wget apt-transport-https`	أسامي الحزم: أدوات تنزيل من النت، ودعم الـ repositories اللي على https.

وفي السطر الأول: `&&` معناها "نفّذ الأمر اللي بعدي **بس لو** اللي قبلي نجح"، و `apt update` بيحدّث قايمة الحزم، و `apt upgrade` بيرقّي الحزم المتسطبة.

2 Install Docker (هيبقى الـ Driver بتاع minikube)

Step 2

```
sudo apt install -y docker.io
sudo systemctl enable docker
sudo systemctl start docker
# Allow your user to run docker without sudo
sudo usermod -aG docker $USER
newgrp docker
```

تشريح الأمر

```
      1       2    3      4
sudo usermod -aG docker $USER
```

1	`usermod`	أمر بيعدّل بيانات User موجود.
2	`-aG`	اختصار `-a -G`: ضيف (append) الـ User على Group، من غير ما تشيله من الـ Groups اللي هو فيها.
3	`docker`	اسم الـ Group اللي أعضاؤه يقدروا يكلموا Docker من غير sudo.
4	`$USER`	متغير (Variable) فيه اسم الـ User الحالي. علامة `$` معناها "هات قيمة المتغير ده".

و `systemctl enable` بيخلي Docker يشتغل مع كل boot، و `start` بيشغّله دلوقتي. و `newgrp docker` بيفعّل الـ Group الجديد في الـ Terminal الحالي من غير logout. وأي سطر بيبدأ بـ `#` ده تعليق (Comment) مش بيتنفذ.

3 Install kubectl (الـ CLI بتاع Kubernetes)

Step 3

```
curl -LO "https://dl.k8s.io/release/$(curl -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
# To Verify =>  kubectl version --client
```

تشريح الأمر

```
curl -LO "https://dl.k8s.io/release/$(curl -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
```

1. `curl` أداة بتنزّل حاجة من URL.
2. `-LO` : `-L` : لو الرابط بيحوّل (redirect) لرابط تاني امشي وراه. `-O` : احفظ الملف بنفس اسمه اللي في الرابط.
3. `$(` بداية **Command Substitution**: نفّذ الأمر اللي جوه الأقواس الأول، وحط الناتج بتاعه مكانه في النص.
4. `curl -s https://dl.k8s.io/release/stable.txt` الأمر الداخلي: بيجيب رقم آخر نسخة مستقرة (زي `v1.31.0`). و `-s` يعني silent من غير progress bar.
5. `/bin/linux/amd64/kubectl` باقي المسار: نسخة Linux لمعالجات amd64 (لو جهازك ARM غيّرها لـ `arm64`).

تشريح الأمر

```
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
```

1. `install` بينسخ الملف ويظبط صلاحياته في خطوة واحدة.
2. `-o root` صاحب الملف (owner) هيبقى root.
3. `-g root` الـ Group بتاع الملف root.
4. `-m 0755` الصلاحيات: صاحبه يقرا ويكتب وينفّذ، والباقي يقروا وينفّذوا بس.
5. `kubectl /usr/local/bin/kubectl` من فين لفين: الملف اللي نزّلناه، للمكان اللي الـ Terminal بيدوّر فيه على الأوامر.

Install Minikube 4

Step 4

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
# To Verify => minikube version
```

4.2 (2) نشغّل الـ Cluster اللي نزّلناه

```
Terminal
minikube start --driver=docker
```

لما نـ run الأمر ده هيبني Cluster لأول مرة ويستخدم Docker كـ Driver يشغّل بيه الـ Cluster. لكن بعد كده لما نـ run الأمر ده تاني، فهو هيعيد تشغيل نفس الـ Cluster اللي قومناه في الأول، وحتى لو غيرت الـ Driver مش هيقوم Cluster جديد مختلف عن القديم، بل هيطلعلك رسالة خطأ، لأن دي مش الطريقة اللي بيسمح بيها minikube إنك تعمل أكتر من Cluster، وإنما بالطريقة دي:

```
Terminal
minikube start --profile cluster-1-name --driver=docker
minikube start --profile cluster-2-name --driver=docker
```

تشريح الأمر

```
1        2       3                    4
minikube start --profile cluster-1-name --driver=docker
```

1. `minikube` الأداة اللي بتعمل Cluster محلي للتجربة.
2. `start` قوّم الـ Cluster (أو اعمله لو مش موجود).
3. `--profile cluster-1-name` اسم الـ Cluster ده. كل Cluster = profile منفصل. لو مكتبتهوش الاسم الافتراضي `minikube`.
4. `--driver=docker` شغّل الـ Node جوه Docker Container. علامة `=` بتربط الـ flag بقيمته، وينفع تكتبها بمسافة بدلها.

> **خلي بالك**
>
> ولكن ده مش مفضّل ولا محتاجينه حالياً في مرحلة التعلم، كمان إن الـ VM ليها حدود، فكتر الـ Clusters ممكن يعمل مشكلة.

`minikube status`	تتأكد إن الـ Cluster شغال.
`kubectl get nodes`	هتلاقي Node واحدة اسمها `minikube` حالتها `Ready`.
`minikube stop`	توقف الـ Cluster من غير ما تمسحه.
`minikube delete`	تمسح الـ Cluster خالص وتبدأ من الأول.

4.3 (3) نجرب نعمل كام Pod

عندنا كذا طريقة:

الطريقة الأولى: ملف YAML

نعمل ملف ونكتب فيه الـ Configuration بتاعة الـ Pod اللي عايزين نعمله (`pod.yaml`):

> **أول ملف YAML**
>
> كل ملفات Kubernetes مكتوبة بـ YAML. لو القواعد مش واضحة (المسافات، و `key: value`، والشرطة `-`)، ارجع لصفحة YAML في كتاب APIs و JSON (فصل 21). وأشهر Error في الأول سببه مسافة زيادة أو ناقصة.

pod.yaml

1 نسخة الـ API ونوع الـ Object اللي بنعمله.	`apiVersion: v1` `kind: Pod`
2 اسم الـ Pod اللي هنتعامل بيه معاه.	`metadata:` `  name: single-container-pod`

3 المواصفات: Container اسمه nginx، من الـ Image دي، ويسمع على 80.	`spec:`
	`  containers:`
	`  - name: nginx`
	`  image: nginx:latest`
	`  ports:`
	`  - containerPort: 80`

> **خلي بالك من المسافات**
>
> الـ YAML حساس جداً للـ Indentation. استخدم **Spaces** مش Tabs، وخلي كل مستوى بمسافتين. أي مسافة غلط هتديك Error أو هتخلي الحقل يتقري في مكان غلط.

نشغّل الملف ونستنى لحد ما الـ Pod تقوم (بنعرف ده من اللي مكتوب في خانة **STATUS**)، ونقدر نعرف عدد الـ Containers اللي جواه من **READY**: العدد اللي شغال فعلياً / العدد الكلي.

Terminal · توضيح للناتج

```
moaz@Linux:~$ kubectl apply -f pod.yaml
pod/single-container-pod created
moaz@Linux:~$ kubectl get pods
NAME                 READY   STATUS              RESTARTS   AGE
single-container-pod   0/1     ContainerCreating   0          10s
moaz@Linux:~$ kubectl get pods
NAME                 READY   STATUS              RESTARTS   AGE
single-container-pod   0/1     ContainerCreating   0          19s
moaz@Linux:~$ kubectl get pods
NAME                 READY   STATUS    RESTARTS   AGE
single-container-pod   1/1     Running   0          26s
```

تشريح الأمر

```
kubectl apply -f pod.yaml
```
(1) (2) (3) (4)

1. `kubectl` — الـ CLI اللي بيكلم الـ API Server.
2. `apply` — طبّق الحالة اللي في الملف: لو الـ Object مش موجود اعمله، ولو موجود حدّثه بالفرق.
3. `-f` — اختصار `--filename`: اقرا من الملف (أو الفولدر أو الـ URL) اللي بعدي.
4. `pod.yaml` — اسم الملف اللي فيه الـ Configuration.

تشريح الأمر

```
kubectl get pods
```
(1) (2)

1. `get` — اعرض قايمة بالـ Objects.
2. `pods` — نوع الـ Object. ينفع تكتبه `pod` أو `po` اختصار.

العمود	معناه
`NAME`	اسم الـ Pod.
`READY`	عدد الـ Containers الجاهزة / العدد الكلي جوه الـ Pod.
`STATUS`	المرحلة: `Pending` (مستني Node)، `ContainerCreating` (بينزّل الـ Image ويجهّز)، `Running`، `Completed`، `CrashLoopBackOff` (بيقع ويقوم ويقع).
`RESTARTS`	الـ Container عمل restart كام مرة. لو بيزيد يبقى فيه مشكلة.
`AGE`	من إمتى الـ Pod اتعمل (s ثواني، m دقايق، h ساعات، d أيام).

جدول 4.1 أعمدة `kubectl get pods`

الطريقة التانية: Image جاهزة

نشغّل الـ Pod اعتماداً على Image جاهزة (نسخة جاهزة للتجربة زي busybox في Docker):

Terminal · توضيح للناتج

```
moaz@Linux:~$ kubectl run my-pod --image=nginx
pod/my-pod created
moaz@Linux:~$ kubectl get pods
NAME                  READY   STATUS              RESTARTS   AGE
my-pod                0/1     ContainerCreating   0          4s
single-container-pod  1/1     Running             0          3m14s
moaz@Linux:~$ kubectl get pods
NAME                  READY   STATUS    RESTARTS   AGE
my-pod                1/1     Running   0          31s
single-container-pod  1/1     Running   0          3m41s
```

تشريح الأمر

```
kubectl run my-pod --image=nginx
        1   2      3
```

1. `run` اعمل Pod واحد على طول من الـ Terminal.
2. `my-pod` اسم الـ Pod.
3. `--image=nginx` الـ Image اللي الـ Container هيتعمل منها. لو مكتبتش tag هيستخدم `latest`.

معلومة مهمة

- ولكن الطريقتين دول مش حاجة عملية ولا هما اللي بنستخدمهم في الشغل العملي، لأنهم بياخدوا وقت ومش طرق احترافية، وإنما للتجربة والتدريب فقط.
- الطريقة العملية إننا بننقل المحتوى اللي جوه الملف (`pod.yaml`) لملف تاني أكبر منه بيكون فيه حاجات أكتر عن الـ Pods ألا وهو (`deployment.yaml`).
- خلينا نتكلم أكتر عن إزاي نعمل Configurations أكتر لـ Pods في ملف (`pod.yaml`)، وبعدها نبقى نشوف إزاي المحتوى ده هيتنقل لملف (`deployment.yaml`).

أوامر هتحتاجها مع أي Pod إضافة

`kubectl get pods -o wide`	الـ Pods ومعاها الـ IP والـ Node اللي شغالة عليها.
`kubectl describe pod my-pod`	كل التفاصيل، وأهم جزء فيها الـ **Events** في الآخر.
`kubectl logs my-pod`	الـ logs بتاعة الـ Container.
`kubectl exec -it my-pod -- sh`	تدخل جوه الـ Container وتفتح shell.
`kubectl delete pod my-pod`	تمسح الـ Pod (أو `kubectl delete -f pod.yaml`).

تشريح الأمر

```
kubectl exec -it my-pod -- sh
```

1. `exec` نفّذ أمر جوه Container شغال.
2. `-it` (interactive): خلي الـ input مفتوح عشان تكتب. `-t` (tty): اعمل terminal حقيقي. الاتنين مع بعض = shell تفاعلي.
3. `my-pod` اسم الـ Pod. لو جواه أكتر من Container ضيف `-c name` .
4. `--` فاصل: أي حاجة بعده مش flags لـ kubectl، دي الأمر اللي هيتنفذ جوه الـ Container.
5. `sh` الأمر نفسه: افتح shell. اكتب `exit` عشان تخرج.

تشريح الأمر

```
kubectl get pods -o wide
```

1. `-o` اختصار `--output` : شكل الناتج.
2. `wide` أعمدة زيادة (IP والـ Node). قيم تانية: `yaml` و `json` و `name` و `jsonpath=...` .

ملخص الفصل

- minikube بيعمل Cluster فيه Node واحدة للتجربة، وبيستخدم Docker كـ Driver.
- `minikube start` بيقوم نفس الـ Cluster كل مرة، ولو عايز أكتر من Cluster استخدم `--profile` .
- `kubectl apply -f pod.yaml` بيعمل Pod من ملف، و `kubectl run` من Image جاهزة.
- READY = الشغال فعلاً / الكلي، و STATUS بتقولك الـ Pod في أنهي مرحلة.
- في الشغل الحقيقي مش بنعمل Pods لوحدها، بنستخدم Deployment.

أسئلة مراجعة

1. إيه الأمر اللي بيقوم Cluster تاني بـ minikube؟
2. لو READY مكتوب فيها `1/2` ده معناه إيه؟
3. اكتب أمر يعمل Pod اسمه `web` من Image اسمها `nginx:1.27` .
4. ليه الطريقتين اللي جربناهم مش هما اللي بنستخدمهم في الشغل؟

صح ولا غلط؟

1. في minikube الـ Control Plane والـ Worker على نفس الـ Node. صح / غلط
2. ينفع تستخدم Tabs في ملفات YAML. صح / غلط
3. `kubectl describe` بيوريك الـ Events بتاعة الـ Pod. صح / غلط

05

المرحلة 2 · الـ Pod من جوه

الفصل الخامس

الـ Probes: هل الـ Container بخير؟

بما إننا عاملين الـ Pod عشان نقدر نتحكم في الـ Containers، فلازم نستخدم حاجات تعرّفنا حالة الـ Containers: شغال ولا لأ؟ ولو شغال، هل شغال بشكل فعّال ولا شغال اسماً بس؟ Kubernetes بيوفرلنا حاجة اسمها الـ Probes.

بعد الفصل ده هتكون عارف

- يعني إيه Healthy Container
- الـ Mechanisms: httpGet و exec و tcpSocket
- الـ Parameters وقيمها الافتراضية
- الفرق بين Liveness و Readiness و Startup
- ترتيب تنفيذ الـ 3 Probes مع بعض

5.1 ليه محتاجين Probes؟

بما إننا عاملين الـ Pod عشان نقدر نتحكم في الـ Containers، فلازم نستخدم حاجات تعرّفنا حالة الـ Containers سواء شغال ولا لأ، ولو شغال فهل شغال بشكل فعّال ولا شغال اسماً بس (ودي اسمها الـ **healthy**، إنه هل الـ Container ده healthy ولا لأ)، وكمان نحدد إيه اللي يحصل في حالة إنه وقع: نقومه إزاي وإمتى ونحاول كام مرة وما إلى ذلك. فـ Kubernetes بيوفرلنا حاجة اسمها الـ **Probes**.

- كل Probe فيهم بيعتمد على **Mechanism** معين عشان يقدر يقدّملنا معلومة معينة.
- كل Probe فيهم بيستخدم شوية **Parameters** معينة عشان يحدد الوقت وعدد المرات اللي هيعمل فيها وظيفته.
- بنكتب الكود بتاع كل Probe فيهم جوه الـ Configuration بتاعة الـ Pod سواء في ملف (`pod.yaml`) أو (`deployment.yaml`)، جوه الـ tag بتاع الـ `containers`.

الشكل العام للكود اللي بنكتبه عشان نستخدم أي Probe منهم:

الشكل العام · Probe

1. اسم الـ Probe: `livenessProbe` أو `readinessProbe` أو `startupProbe`.

```
NameOfProbe:
```

2. الـ Mechanism المستخدم: واحد بس منهم.

```
  httpGet:
    path: path/to/health_endpoint
    port: port
  #---OR----
  exec:
    command: command
```

```
initialDelaySeconds: time(s)
periodSeconds: time(s)
failureThreshold: times(n)
successThreshold: times(n)
timeoutSeconds: time(s)
```

3 الـ Parameters اللي بتحدد التوقيت وعدد المرات.

5.2 الـ Mechanisms: هيعرف الحالة إزاي؟

الـ Mechanism	بيعمل إيه؟	بنستخدمه إمتى؟
`httpGet` `path:` `port:`	بتقول لـ Kubelet: اعمل HTTP GET request على الـ path والـ port دول، وشوف كود الرد اللي هيجيلك، وبناءً عليه حدد حالة الـ Container: • **200** (أو أي كود من 200 لـ 399): شغال (healthy) • أي كود تاني: مش شغال (unhealthy) • مفيش رد خالص (time out): يبقى واقع أصلاً	لما تطبيقك عنده HTTP server: APIs، web apps، microservices.
`exec` `command:`	بتقول لـ Kubelet: "شغّل الأمر ده جوه الـ Container كأنك فاتح terminal جواه، وشوف الـ exit code اللي هيطلعلك بعد ما تـ run الـ command": • **0**: شغال (healthy) • أي كود غير 0: مش شغال (unhealthy)	لما مفيش HTTP server: databases، workers، sidecars.
`tcpSocket` `port:`	إضافة بيحاول يفتح اتصال TCP على الـ port. لو الاتصال اتفتح يبقى healthy.	خدمات مش HTTP بس بتسمع على port، زي Redis أو أي TCP server.

جدول 5.1 الـ Mechanisms المتاحة

> **لاحظ**
>
> حتة إنه هل الـ Container ده healthy ولا لأ: إنت اللي بتحطلها معيار محدد تقدر من خلاله تحدد الحالة، إنما Kubernetes نفسه ميقدرش يعرف ده. فبالتالي تختار الطريقة اللي إنت متأكد إنها تقدر تعرّفك الحالة بتاعته وتستخدمها في الـ Mechanism اللي هتختاره جوه الـ Probe.
>
> - فمثلاً إنت عارف إن فيه endpoint لو بعت عليها request وجالك رد منها، هتعرف الحالة بناءً على قيمة الكود اللي طلع في الرد، فتستخدم `httpGet`.
> - أو إنت عارف إن فيه command معين لو اتـ run جوه الـ Container واشتغل واداّك exit code معين، يبقى هتعرف الحالة، فتستخدم `exec`.

5.3 الـ Parameters: إمتى وكام مرة؟

الـ Parameter	معناه	الـ Default

`initialDelaySeconds`	كام ثانية Kubernetes يستنى بعد ما الـ Container نفسه يبدأ (قبل ما يطبّق عليه الـ Probe). **تستخدمه إمتى:** لو تطبيقك بياخد وقت يبدأ (JVM، loading config، connecting to DB). لو مستخدمتهوش هيبدأ الـ Probe فوراً، ولو تطبيقك لسه بيبدأ هيفشل وهيعمل restart.	**0**
`periodSeconds`	كل كام ثانية Kubernetes يعيد تطبيق الـ Probe. **إمتى تقلله:** لو عايز تكتشف المشكلة بسرعة (5 ثواني). **إمتى تكبره:** لو الـ Probe بياخد resources أو DB query (30 ثانية).	**10**
`failureThreshold`	Kubernetes يستنى كام مرة فشل متتالية قبل ما ياخد action: • لـ liveness: بعد الفشل يعمل restart • لـ readiness: بعد الفشل يشيله من الـ load balancer **إمتى تكبره:** لو تطبيقك بيعمل spikes مؤقتة، عشان ما يعملش restart بسبب مشكلة لحظية.	**3**
`successThreshold`	كام مرة نجاح متتالية بعد الفشل عشان يرجع يعتبره healthy. **إمتى تكبره:** لو عايز تتأكد إن التطبيق مش بس بيرد مرة وبعدين بيوقع تاني. لـ liveness و startup لازم يكون **1** دايماً، و Kubernetes بيرفض أي قيمة تانية.	**1**
`timeoutSeconds`	لو الـ Probe مردّش في كام ثانية يعتبره فشل: يعني Kubelet بعت request أو شغّل command وبعد وقت معين معدّش رد، يبقى فشل. **إمتى تزوده:** لو الـ health endpoint بتاعك بيعمل DB query بتاخد وقت. لو صغير أوي: هتاخد false failures بدون سبب. الأحسن تستخدمه وتخليه 3.	**1**

جدول 5.2 الـ Parameters وقيمها الافتراضية

5.4 الأنواع التلاتة للـ Probes

Liveness Probe (1)

بتسأل: هل الـ Container شغال صح ولا وقع في مشكلة جواه؟

Readiness Probe (2)

بتسأل: هل الـ Container جاهز يستقبل traffic دلوقتي؟

Startup Probe (3)

بتسأل: هل التطبيق خلّص بدأ ووصل للحالة الأولانية؟

Liveness Probe الـ (1)

- بتسأل هل الـ Container شغال صح ولا وقع في مشكلة جواه؟
- التطبيق ممكن يبقى في process شغالة بس متجمدة، زي: deadlock في الـ threads، أو memory leak، و Kubernetes مش عارف من غير ما تحددله الطريقة اللي يقدر يعرف بيها هو healthy ولا لأ زي ما قلنا فوق.
- **لو فشل:** Kubernetes بيوقف الـ Container، بيمسحه، وبيعمله restart من أول وجديد.

livenessProbe

```yaml
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 15
  failureThreshold: 3
  successThreshold: 1
  timeoutSeconds: 5
```

> **خلي بالك**
>
> متحطش في الـ Liveness check على DB أو services خارجية، عشان لو وقعوا متعملش restart لكل الـ Pods دفعة واحدة (والمشكلة أصلاً مش فيهم).

(2) الـ Readiness Probe

- بتسأل هل الـ Container جاهز يستقبل traffic دلوقتي؟
- التطبيق ممكن يكون شغال بس لسه بيحمّل cache أو بيتصل بـ DB أو تحت ضغط.
- هنا تمام تعمل check على الـ DB والـ external services، لأن الفشل مش بيوقف الـ Container، هو بس بيشيله من الـ load balancer.
- ضروري جداً في الـ rolling deployments عشان zero downtime.
- **لو فشل:** Kubernetes بيشيل الـ Pod من الـ endpoints، مش بيعمل restart، بس مبيجيلوش traffic.

readinessProbe

```yaml
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
  failureThreshold: 3
  successThreshold: 2
```

(3) الـ Startup Probe

- بتسأل هل التطبيق خلّص بدأ ووصل للحالة الأولانية؟
- طول ما هي شغالة، الـ liveness والـ readiness بيتوقفوا خالص، ولأنها بعد ما تنجح مرة واحدة بس بتوقف وتسيب المجال للتانيين.
- بتستخدمها مع التطبيقات اللي بتاخد وقت طويل تبدأ زي JVM أو legacy apps، بدلاً من إنك تحط `initialDelaySeconds` كبيرة على الـ liveness.

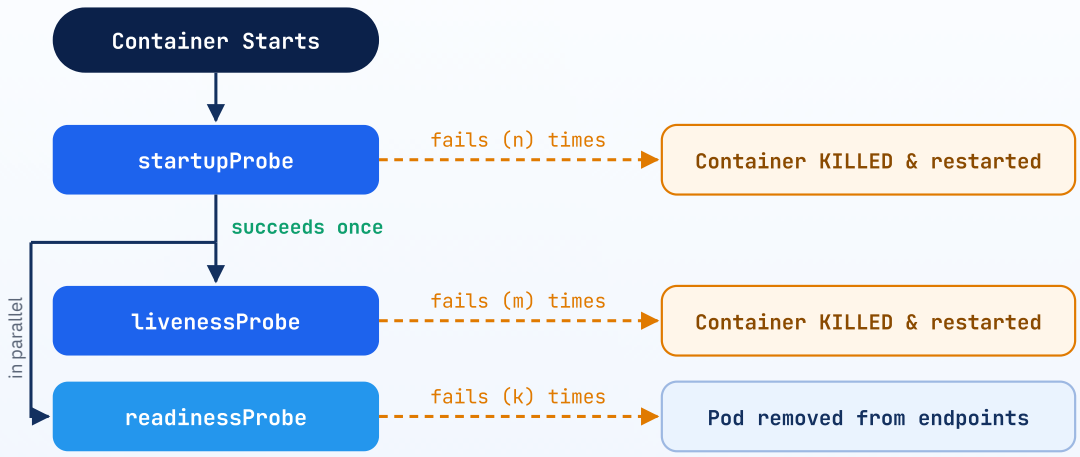
- **لو فشل أكتر من** `failureThreshold` : بيعمل restart زي الـ liveness.

```yaml
startupProbe:
  httpGet:
    path: /healthz
    port: 8080
  failureThreshold: 30
  periodSeconds: 10
```

> **معلومة مهمة**
>
> هنا التطبيق قدامه لحد **30 × 10 = 300 ثانية** (5 دقايق) عشان يبدأ. أول ما ينجح مرة، الـ Startup Probe بتقف والـ Liveness تبدأ شغلها.

5.5 الـ 3 Probes مع بعض

لاحظ إن ممكن Pod تحتاج نعملها Probe واحدة من التلاتة، أو اتنين، أو التلاتة كلهم. ده شكل الـ flow بتاع التنفيذ بتاع الـ 3 Probes مع بعض:



شكل 5.1 الـ startupProbe الأول، ولما تنجح مرة يشتغل الـ liveness والـ readiness مع بعض (بالتوازي).

	Liveness	Readiness	Startup
السؤال	عايش ولا متجمّد؟	جاهز يستقبل Traffic؟	خلّص بدأ ولا لسه؟
لو فشل	restart لـ Container	يتشال من الـ endpoints (من غير restart)	restart لـ Container
check على DB؟	لأ	أيوه، تمام	حسب التطبيق
بيشتغل إمتى	بعد نجاح الـ Startup، طول الوقت	بعد نجاح الـ Startup، طول الوقت	في الأول بس، لحد أول نجاح

جدول 5.3 مقارنة سريعة بين الـ 3 Probes

ملخص الفصل

- الـ Probes بتعرّف Kubernetes إذا كان الـ Container healthy ولا لأ، بمعيار إنت اللي بتحدده.
- الـ Mechanisms: `httpGet` (كود الرد)، `exec` (الـ exit code)، `tcpSocket` (الاتصال).
- الـ Defaults: initialDelay 0، period 10، failure 3، success 1، timeout 1.
- Liveness فشل = restart. Readiness فشل = مفيش traffic. Startup بيحمي التطبيقات البطيئة في البداية.
- متحطش checks على DB في الـ Liveness.

أسئلة مراجعة

1. عندك Worker مفيهوش HTTP server، هتستخدم أنهي Mechanism؟
2. ليه مينفعش تعمل check على الـ DB في الـ Liveness Probe؟
3. تطبيق بياخد دقيقتين يبدأ: إيه الأفضل، `initialDelaySeconds: 120` على الـ liveness ولا Startup Probe؟ وليه؟
4. Readiness Probe فشلت 3 مرات، إيه اللي هيحصل للـ Pod بالظبط؟

صح ولا غلط؟

1. لو الـ Readiness Probe فشلت، الـ Container بيعمل restart. صح / غلط
2. ينفع تخلي `successThreshold: 3` في الـ Liveness Probe. صح / غلط
3. طول ما الـ Startup Probe شغالة، الـ Liveness مش بتشتغل. صح / غلط

06

المرحلة 2 · الـ Pod من جوه

الفصل السادس

الـ Requests والـ Limits: موارد الـ Pod

لما بتشتغل مع Kubernetes، في الآخر عندك Nodes (سيرفرات فعلية أو سحابية)، وعليها بتشتغل الـ Pods. الـ Node عندها موارد محدودة (CPU و Memory) والـ Pods بتتشارك فيهم. إزاي Kubernetes يقرر يحط الـ Pod على أنهي Node؟ وإزاي يمنع Pod واحد ياخد كل الموارد؟

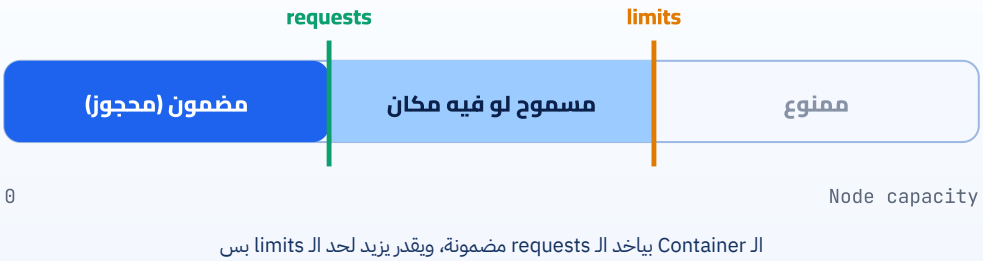
بعد الفصل ده هتكون عارف

- الفرق بين الـ Requests والـ Limits
- إيه اللي بيحصل لو الـ Container عدّى الـ Limit
- وحدات الـ CPU (m) والـ Memory (Mi و Gi)
- ليه الـ Requests مهمة للـ Scheduler
- الـ QoS Classes

6.1 المشكلة: موارد محدودة متشاركة

لما بتشتغل مع Kubernetes، في الآخر عندك **Nodes** (سيرفرات فعلية أو سحابية)، وعليها بتشتغل الـ **Pods** (اللي جواها التطبيقات بتاعتك). الـ Node عندها موارد محدودة (CPU و Memory) والـ Pods بتتشارك فيهم.

إزاي Kubernetes يقرر يحط الـ Pod على أنهي Node؟ وإزاي يمنع Pod واحد ياخد كل الموارد؟ الإجابة هي نظام الـ **Requests & Limits**.



الـ Container بياخد الـ requests مضمونة، ويقدر يزيد لحد الـ limits بس

شكل 6.1 الـ requests هي الأرضية المضمونة، والـ limits هي السقف.

6.2 (1) الـ Limits: السقف

- من اسمه هو حد (limit)، يعني بتحدد **الحد الأقصى** من الموارد اللي ممكن يستخدمها الـ Container ومينفعش يزيد عنها. العملية بتتم في الـ Kernel Level عن طريق الـ **cgroups**.
- لو الـ Container حاول يعدّي الـ limit بتاع الـ **CPU**: هيفضل شغال بس هيتم تبطيئه (Throttling). إنما لو حاول يعدّي الـ limit بتاع الـ **RAM**: هيتم قفله تماماً (kill)، وده بيتم عن طريق الـ **OOM** kernel's killer (Out-of-Memory)، وهتلاقي حالته `OOMKilled`.

- بنستخدمه لما نكون عايزين نحمي الـ Node اللي عليها الـ Pod ده، وغيره من الـ Pods، من إن Pod يستهلك أغلب الموارد لوحده ويأثر على الباقي. وده في حالة إننا عارفين إن فيه services أو حاجات في الـ app اللي هنقومه صعب نتوقع استهلاكها الحقيقي، وفي نفس الوقت ممكن تستهلك موارد كتير.
- **طريقة الاستخدام:** جوه الـ tag بتاع الـ `containers` اللي جوه الـ Pod بنعمل tag اسمها `resources` وتحتها بنحط `limits`.

limits

```yaml
resources:
  limits:
    cpu: "number of cores"    # x: x cores  //  (x)m: x millicores
    memory: "size of memory"  # (x)Ki / Mi / Gi
```

6.3 وحدات الموارد

الوحدة	اسمها	قيمتها
`Ki`	Kibibyte	1,024 bytes (مش 1000 زي الـ kilobyte)
`Mi`	Mebibyte	1,048,576 bytes (مش 1,000,000 زي الـ megabyte)
`Gi`	Gibibyte	1,073,741,824 bytes (مش 1,000,000,000 زي الـ gigabyte)

جدول 6.1 وحدات الـ Memory

القيمة	معناها
`"1"`	Core كامل (أو vCPU واحد)
`"500m"`	نص Core (500 millicores)
`"100m"`	عُشر Core
`"0.25"`	نفس `"250m"`

جدول 6.2 وحدات الـ CPU إضافة

خلي بالك: M غير Mi

لو كتبت `memory: 128M` (من غير i) ده معناه 128 ميجا بالنظام العشري (128,000,000 byte)، ولو كتبت `128m` بـ m صغيرة ده معناه **0.128 byte**! استخدم دايماً `Mi` و `Gi` للـ Memory.

دقة: لو مفيش Limits

لو سبت الـ tag من الاتنين (`memory` أو `cpu`) فاضي من غير قيمة، ساعتها هيتفهم إنه ملهوش حدود، ويقدر يستخدم الموارد المتاحة على الـ Node لحد آخرها، وده ممكن يأثر على كل الـ Pods اللي معاه على نفس الـ Node. لكن Kubernetes مش هيقفل باقي الـ Pods عشان خاطره: لو الـ Memory خلصت، الـ kubelet بيبدأ يطرد (Evict) Pods، وبيبدأ باللي ملهاش requests ولا limits، وممكن الـ OOM killer يقتل الـ Container اللي بياكل الرامات نفسه. فالنتيجة فوضى، وعشان كده حط حدود دايماً.

6.4 (2) الـ Requests: الأرضية

- بتحدد **الحد الأدنى** من الموارد اللي محتاجها الـ Container ده عشان يقوم بدوره بشكل طبيعي، والـ Scheduler بيحجزهاله.

- الـ Container يقدر يزيد عن الحد الأدنى ده عادي طالما فيه إمكانية (مساحة يعني)، ولحد الـ limit لو موجود.
- لازم نستخدمه دايماً، عشان الـ Kubernetes **Scheduler** لما ييجي يقوم الـ Pod اللي فيها الـ Container ده يكون عارف هي عايزة موارد قد إيه (كحد أدنى)، عشان يشوف أنهي Node متوفر عليها المساحة دي أو أكتر فيحطها عليها.
- لاحظ إن عدم استخدامك لـ requests هيخليه يتعامل معاه إنه مش محتاج موارد، فممكن يتحط على Node عليها حمل عالي مفيهاش مكان، فمتشتغلش وتعمل مشاكل.
- **طريقة الاستخدام:** جوه الـ tag بتاع الـ `containers` بنعمل tag اسمها `resources` وتحتها بنحط `requests`.

requests

```yaml
resources:
  requests:
    cpu: "number of cores"    # x: x cores  //  (x)m: x millicores
    memory: "size of memory"  # (x)Ki / Mi / Gi
```

مثال كامل

pod.yaml · resources

```yaml
spec:
  containers:
  - name: api
    image: my-api:1.0
    resources:
      requests:
        cpu: "250m"
        memory: "128Mi"
      limits:
        cpu: "500m"
        memory: "256Mi"
```

الـ Scheduler هيدوّر على Node فاضي فيها ربع Core و 128Mi على الأقل. والـ Container يقدر يوصل لنص Core (بعدها throttling) و 256Mi (بعدها OOMKilled).

6.5 الـ QoS Classes: مين يتطرد الأول؟ إضافة

على حسب إنت كاتب requests و limits إزاي، Kubernetes بيدّي كل Pod "درجة" اسمها **QoS Class**. لما الـ Node تتزنق في الـ Memory، بيطرد الأقل درجة الأول.

الـ Class	بيتحسب إمتى؟	أولوية الطرد
Guaranteed	كل Container عنده requests و limits للـ CPU والـ Memory، والـ requests = الـ limits.	آخر واحد يتطرد
Burstable	فيه requests أو limits لكن مش متساويين أو مش كاملين.	في النص
BestEffort	مفيش requests ولا limits خالص.	أول واحد يتطرد

جدول 6.3 الـ QoS Classes

Terminal

```
kubectl get pod my-pod -o jsonpath='{.status.qosClass}'
kubectl top pod          # الاستهلاك الحقيقي (محتاج metrics-server)
```

تشريح الأمر

```
kubectl get pod my-pod -o jsonpath='{.status.qosClass}'
        1   2      3  4
```

1. `get pod my-pod` هات الـ Pod اللي اسمه my-pod بالذات.
2. `-o jsonpath=` بدل الجدول، طلّع قيمة حقل معين من الـ JSON بتاع الـ Object.
3. `'` علامات التنصيص المفردة بتمنع الـ shell إنه يفسّر الأقواس `{}` بطريقته.
4. `{.status.qosClass}` المسار جوه الـ Object: الـ `.` الأولى هي الـ Object نفسه، وبعدين الحقل `status` وجواه `qosClass`.

تشريح الأمر

```
kubectl top pod
        1   2
```

1. `top` اعرض الاستهلاك اللحظي للـ CPU والـ Memory (زي أمر top في Linux).
2. `pod` للـ Pods. وينفع `kubectl top node` للـ Nodes. محتاج metrics-server (في minikube: `minikube addons enable metrics-server`).

ملخص الفصل

- الـ requests: الحد الأدنى المحجوز، والـ Scheduler بيستخدمه عشان يختار الـ Node.
- الـ limits: السقف. عدّى الـ CPU = throttling، عدّى الـ Memory = OOMKilled.
- الـ CPU بالـ cores أو millicores (`500m`)، والـ Memory بـ `Mi` و `Gi`.
- من غير حدود الـ Container يقدر ياكل موارد الـ Node، وده بيعمل Evictions ومشاكل.
- الـ QoS: Guaranteed أحسن من Burstable أحسن من BestEffort.

؟ أسئلة مراجعة

1. Container عدّى الـ Memory limit، وواحد تاني عدّى الـ CPU limit، إيه اللي هيحصل لكل واحد؟
2. ليه الـ requests مهمة للـ Scheduler بالذات؟
3. إيه الفرق بين `256Mi` و `256M` ؟
4. Pod فيه limits = requests لكل الموارد، يبقى QoS بتاعه إيه؟

صح ولا غلط؟

1. `cpu: "500m"` معناها نص Core. صح / غلط
2. الـ Container مينفعش يستخدم أكتر من الـ requests بتاعته أبداً. صح / غلط
3. الـ Pods اللي ملهاش requests ولا limits هي أول حاجة بتتطرد. صح / غلط

07

المرحلة 3 · إدارة الـ Workloads

الفصل السابع

الـ Labels والـ Selectors

أكيد أنا هيبقى عندي أكتر من Pod في الـ App بتاعي، إزاي هفرق أو أميز بينهم؟ بمعنى إني أعرّف Kubernetes الـ Pods الموجودة عشان لما أديله أمر يعمل حاجة على Pod معينة منهم يعرف يميزها ويشتغل عليها. الإجابة هي الـ Labels والـ Selectors.

بعد الفصل ده هتكون عارف

- يعني إيه Label وشكلها key: value
- الفرق بين الـ name والـ Label
- يعني إيه Selector وبيتكتب فين
- الـ matchLabels و الفلترة بـ kubectl -l

7.1 الـ Label (1)

من اسمها كده، الـ **Label** شبه الـ Tag أو الـ Sticker اللي بنلزقه على أي حاجة ونكتب عليه وصف معين عشان نعرف نميّز الحاجة دي بسهولة.

الـ Label بتكون عبارة عن: `key: value` ، وهدفها إنها تساعدنا نصنّف ونرتّب الـ Objects الموجودة في Kubernetes زي الـ Pods والـ Deployments والـ Services وغيرهم.

مجموعة الـ Backend

فمثلاً لو عندي كذا Pod شغالين كـ Backend، ممكن أحط عليهم كلهم Label بالشكل ده: `type: backend` . كده أي Pod عليها الـ Label دي هتبقى ضمن مجموعة الـ Backend.

تمييز أدق

ولو فيه Pod معينة مسؤولة عن خدمة الطباعة مثلاً، أقدر أضيف لها Label تانية: `service: print` . وبكده أقدر أميّزها عن باقي الـ Backend Pods.

بالنسبة للتسمية، فأنت حر تماماً تختار أي أسماء للـ Key أو الـ Value، مفيش أسماء إجبارية. المهم إنك تختار أسماء واضحة ومنظمة عشان تبقى فاهم الـ Objects بتاعتك بسهولة، وكمان عشان Kubernetes يقدر يتعامل معاها بشكل صحيح.

معلومة مهمة

الـ Labels مش الغرض منها إنها تكون اسم أو ID للـ Pod، لكن الغرض منها التصنيف والتجميع. Kubernetes بيستخدمها بعد كده مع الـ Selectors عشان يختار مجموعة معينة من الـ Pods ويتعامل معاها، زي مثلاً لما Service تربط نفسها بمجموعة Pods معينة.

إيه هو Syntax الـ Labels؟

الـ Labels بتتكتب جوه الـ `metadata` اللي بتكون موجودة مع الـ `apiVersion` والـ `kind` والـ `spec` . الـ metadata غالباً بيكون فيها حاجتين مهمين: `name` و `labels` .

الشكل العام

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: name_of_pod_itself
  labels:
    key_1: value_1
    key_2: value_2
```

مثال

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: frontend-pod
  labels:
    environment: production
    app: website
```

7.2 الفرق بين الـ name والـ Label

يعني إيه name؟ هو الاسم اللي Kubernetes بيعرف بيه الـ Pod دي. ولما تيجي تتعامل مع الـ Pod عن طريق Kubernetes API أو أوامر `kubectl` هتستخدم الاسم ده. **لازم كمان يكون الاسم فريد جوه نفس الـ Namespace.**

> **مثال من الحياة**
>
> عشان نفرق أكتر بين الـ name والـ label، خلينا ناخد مثال: لنفترض إنت موظف في شركة، ليك ID أو اسم خاص بيك، وفي نفس الوقت إنت تبع Team معين. هل لما بيتحوّلك المرتب الشركة هتبعته للـ Team كله ولا ليك أنت شخصياً؟؟ أكيد ليك أنت، باستخدام اسمك أو الـ ID بتاعك، لأن دي عملية موجهة لشخص واحد.
>
> لكن لو الشركة عايزة تبعت email لكل أعضاء فريق معين أو تعمل اجتماع للـ Team، مش هتختار كل شخص بالاسم، لكنها هتحدد الـ Team كله وكل أعضائه هيتضمّنوا في العملية.

	name	labels
بيحدد	Object واحد بعينه	مجموعة Objects ليهم نفس الصفة أو الدور
فريد؟	أيوه، جوه نفس الـ Namespace	لأ، نفس الـ Label على Objects كتير
بنستخدمه في	نتعامل مع Object مباشرة	نستهدفهم كلهم مرة واحدة

جدول 7.1 name مقابل labels

7.3 الـ Selectors (2)

من اسمها كده هي حاجة بتشاور أو بتختار الـ Labels اللي إنت عملتها وحطيتها على الـ Pods.

بمعنى إن عندك Object أو Service معينة بتحتاج تكلّم مجموعة الـ Pods بتوع الـ Backend مثلاً، هتروح جوه الـ Configuration file بتاع الـ Object أو الـ Service دي وتحدد الـ Labels اللي عايز تستهدفهم عن طريق الـ **Selector**. بتكون عبارة عن `key: value`، ولازم تكون بنفس الأسامي والقيم زي الـ Labels اللي إنت عملتها.

إيه هو Syntax الـ Selector؟

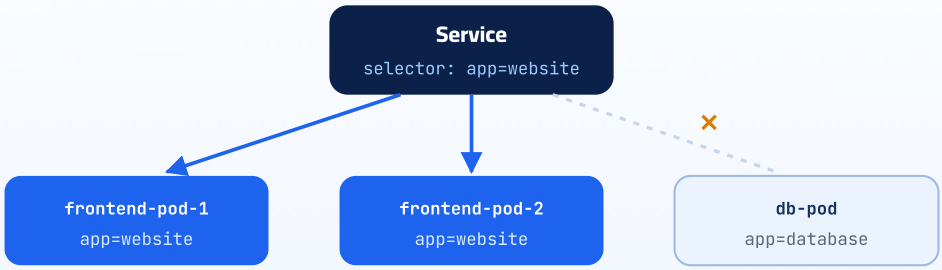
الـ Selector بتتكتب جوه الـ `spec` اللي بتكون موجودة مع الـ `apiVersion` والـ `kind`:

الشكل العام

```yaml
apiVersion: v1
kind: Service
spec:
  selector:
    key_1: value_1   # same as label_1
    key_2: value_2   # same as label_2
```

مثال

```yaml
apiVersion: v1
kind: Service
spec:
  selector:
    environment: production
    app: website
```



شكل 7.1 الـ Selector بيختار كل الـ Pods اللي عليها نفس الـ Label، واللي مش مطابق بيتجاهله.

إضافة: matchLabels

الـ Service بتكتب الـ selector على طول (`key: value`). لكن الـ Objects الأحدث زي الـ Deployment والـ ReplicaSet والـ DaemonSet بتكتبه جوه `matchLabels`، وكمان تقدر تستخدم `matchExpressions` لشروط زي `In` و `NotIn`. هتشوفها في الفصول الجاية.

Deployment selector

```yaml
selector:
  matchLabels:
    app: website
```

الـ Labels من الـ Terminal إضافة

`kubectl get pods --show-labels`	الـ Pods ومعاها كل الـ Labels.
`kubectl get pods -l app=website`	فلترة: الـ Pods اللي عليها الـ Label دي بس.
`kubectl get pods -l 'env in (dev,prod)'`	فلترة بشرط (set-based).
`kubectl label pod my-pod tier=backend`	إضافة Label لـ Pod موجودة.
`kubectl label pod my-pod tier-`	مسح الـ Label (علامة `-` في الآخر).

تشريح الأمر

```
kubectl get pods -l 'env in (dev,prod)'
```

1. `-l` اختصار `--selector` : فلتر بالـ Labels.
2. `env` الـ key بتاع الـ Label.
3. `in (dev,prod)` الشرط: القيمة تكون dev أو prod. وفيه كمان `notin` ، و `!env` (الـ key مش موجود).
4. `'` التنصيص عشان الـ shell ميفهمش الأقواس والمسافات غلط.

تشريح الأمر

```
kubectl label pod my-pod tier=backend --overwrite
```

1. `label` ضيف أو عدّل Labels على Object موجود.
2. `pod my-pod` النوع والاسم.
3. `tier=backend` الـ Label بصيغة `key=value` (في الـ Terminal بـ `=` مش `:`).
4. `--overwrite` لازم لو الـ key موجود قبل كده وعايز تغيّر قيمته، وإلا هيديك Error.

وعشان تمسح Label اكتب اسم الـ key وبعده علامة `-` لازقة فيه: `kubectl label pod my-pod tier-` .

ملخص الفصل

- الـ Label = `key: value` للتصنيف والتجميع، وبتتكتب في `metadata.labels` .
- الـ name فريد ويحدد Object واحد، والـ Labels بتجمع Objects كتير.
- الـ Selector بيختار الـ Objects بالـ Labels، وبيتكتب في `spec.selector` .
- الأسامي والقيم في الـ Selector لازم تطابق الـ Labels بالظبط (وحساسة لحالة الحروف).

أسئلة مراجعة

1. ليه منستخدمش الـ name عشان الـ Service تختار الـ Pods؟
2. اكتب الـ Labels لـ Pod تبع الـ backend في بيئة الـ staging.
3. Service الـ selector بتاعها `app: Web` والـ Pods عليها `app: web` ، هتشتغل؟
4. اكتب أمر يعرض الـ Pods اللي عليها `tier=backend` بس.

صح ولا غلط؟

1. ينفع اتنين Pods في نفس الـ Namespace يبقى ليهم نفس الـ name. صح / غلط
2. ينفع اتنين Pods يبقى عليهم نفس الـ Labels. صح / غلط
3. الـ Labels بتتكتب جوه الـ spec. صح / غلط

08

المرحلة 3 · إدارة الـ Workloads

الفصل التامن

الـ ReplicaSets: عدد ثابت من النسخ

فاكر لما قلنا إن Kubernetes بيشتغل بمبدأ الـ Desired State، واللي هي إن دوره يتأكد إن فيه عدد Pods معين شغال وقايم، ولو حصل أي مشكلة وحاجة منهم وقعت هيقوم بدالها؟ هنتكلم عن الحاجة اللي بتعمل كده، ألا وهي الـ ReplicaSets.

بعد الفصل ده هتكون عارف

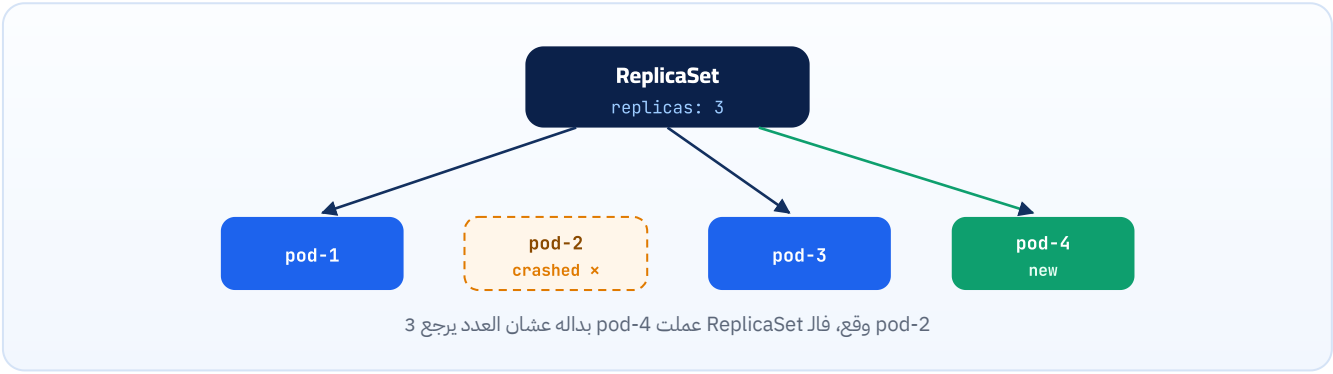
- يعني إيه ReplicaSet ودورها
- ليه بنشغّل كذا نسخة من نفس الـ Pod
- إيه اللي بيحصل لو النسخ زادت أو قلّت
- الـ YAML الصح للـ ReplicaSet

8.1 يعني إيه ReplicaSet؟

هي الحاجة اللي بتتأكد إن فيه عدد معين من الـ Pods شغال، ولكن لاحظ إنها هنا بتتأكد من إن فيه عدد نسخ محدد من **Pod معينة** قايمة. يعني لنتخيل عندنا Pod للـ frontend اسمها `frontend-pod`، لو حطيت إن الـ ReplicaSet بتاعها 3 مثلاً، فكده Kubernetes هيفهم إنه لازم يكون فيه 3 نسخ منها قايمين شغالين.

طب ليه ممكن نحتاج نقوم كذا نسخة من نفس الـ Pod؟

ببساطة عشان نقدر نستقبل traffic أعلى، وكمان عشان لو واحدة فيهم وقعت يبقى فيه بدالها يستقبل الـ traffic على ما الـ Pod اللي وقعت تقوم تاني. لكن تخيل لو مقومين نسخة واحدة من كل Pod ووقعت.. على ما تقوم تاني الموقع هيفضل واقع. فالهدف إنه يكون فيه نسخ احتياطية عشان لو حصل أي مشاكل زي كده، عقبال ما تتحل، الـ Pods تفضل موجودة وبتستقبل traffic بشكل طبيعي لحد ما المشاكل تتصلح.



شكل 8.1 الـ ReplicaSet بتحافظ على العدد المطلوب طول الوقت.

لاحظ

- لو بطريقة ما بقى فيه نسخ أكتر من المطلوب من نفس الـ Pod، الزيادات هتتمسح.
- لما Pod بيقع وبييجي يقوم بداله، فهو بيقوم Pod زي المكتوب في الـ Configuration (نسخة من الـ template).
- لو زودت أو قللت عدد الـ replicas فجأة، فهيزوّد أو هيقلل النسخ اللي كانت شغالة عشان تبقى زي المكتوب.

8.2 إيه هو Syntax الـ ReplicaSet؟

الفكرة كلها في tag اسمها `replicas` بتحطها جوه الـ tag بتاع الـ `spec`. الشرح الأصلي كتبها كده عشان يوضّح الفكرة:

الفكرة (للتوضيح)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: frontend-pod
spec:
  replicas: 3
  containers:
    ........
```

الشكل العام (للتوضيح)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: name_of_pod_itself
spec:
  replicas: integer_value
  containers:
    ........
```

دقة: الـ YAML الحقيقي

الـ `kind: Pod` مفيهوش حقل اسمه `replicas`، ولو كتبته Kubernetes هيرفض الملف. الـ `replicas` بتتكتب في Object من نوع `ReplicaSet` (أو `Deployment`)، والـ Pod نفسه بيتوصف جواه في `template`، وبيتربطوا ببعض بالـ **Labels** والـ **Selector** اللي لسه شايفينهم. ده الشكل الصح:

replicaset.yaml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: frontend-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: frontend
```

1. الـ ReplicaSet في الـ API group اسمه `apps/v1`.
2. عدد النسخ المطلوب.
3. الـ Pods اللي الـ ReplicaSet مسؤولة عنها.

```yaml
template:
  metadata:
    labels:
      app: frontend
  spec:
    containers:
    - name: web
      image: nginx:1.25
```

4 الـ template: شكل الـ Pod اللي هيتنسخ. الـ labels هنا لازم تطابق الـ selector.

Terminal · توضيح للناتج

```
moaz@Linux:~$ kubectl apply -f replicaset.yaml
replicaset.apps/frontend-rs created
moaz@Linux:~$ kubectl get rs
NAME          DESIRED   CURRENT   READY   AGE
frontend-rs   3         3         3       20s
moaz@Linux:~$ kubectl delete pod frontend-rs-7xk2p
pod "frontend-rs-7xk2p" deleted
moaz@Linux:~$ kubectl get pods
NAME                READY   STATUS              RESTARTS   AGE
frontend-rs-4mz9q   1/1     Running             0          40s
frontend-rs-9wq1d   1/1     Running             0          40s
frontend-rs-b8c3t   0/1     ContainerCreating   0          2s
```

تشريح الأمر

```
kubectl get rs
          ①
```

1 `rs` اختصار `replicasets`. الأعمدة: `DESIRED` العدد المطلوب، `CURRENT` الموجود فعلاً، `READY` الجاهز يستقبل traffic.

ولاحظ إن أسامي الـ Pods بقت **اسم الـ ReplicaSet + كود عشوائي** (زي `frontend-rs-7xk2p`)، لأنها هي اللي بتعملهم من الـ template. والـ Pod الجديدة `b8c3t` اتعملت أوتوماتيك بعد ما مسحنا واحدة.

Best Practice

في الشغل الحقيقي نادراً ما بتكتب ReplicaSet بنفسك. بتكتب **Deployment** وهو اللي بيعمل الـ ReplicaSet ويديرها، وده اللي هنشوفه بعد فصل الـ Namespaces.

ملخص الفصل

- الـ ReplicaSet بتضمن إن فيه عدد ثابت من نسخ نفس الـ Pod شغال طول الوقت.
- النسخ الزيادة بتتمسح، والناقصة بتتعمل من الـ template.
- الـ YAML الصح: `kind: ReplicaSet` فيها `replicas` و `selector` و `template`.
- في الشغل بنستخدم الـ Deployment اللي بيدير الـ ReplicaSet بدالنا.

أسئلة مراجعة

1. ليه نشغّل 3 نسخ من نفس الـ Pod بدل نسخة واحدة؟
2. مسحت Pod تبع ReplicaSet عددها 3، إيه اللي هيحصل؟
3. إيه علاقة الـ `selector` بالـ `template.metadata.labels` ؟

صح ولا غلط؟

1. ينفع تكتب `replicas` جوه `kind: Pod` . صح / غلط
2. لو بقى فيه Pods أكتر من المطلوب، الـ ReplicaSet بتمسح الزيادة. صح / غلط
3. الـ ReplicaSet بتحدّث الـ Pods القديمة لو غيرت الـ image في الـ template. صح / غلط

09

المرحلة 3 · إدارة الـ Workloads

الفصل التاسع

الـ Namespaces: تقسيم الـ Cluster

الـ Namespace عبارة عن عزل أو تقسيم الـ Cluster الكبير اللي شايل كل حاجة لمجموعة Clusters صغيرة، ولكن بشكل افتراضي (Virtually). ومعاها هنشوف إزاي نحط حدود للموارد بتاعة كل Namespace بالـ ResourceQuota والـ LimitRange.

بعد الفصل ده هتكون عارف

- يعني إيه Namespace وليه بنستخدمه
- إزاي الـ Pods بتكلم بعض عبر Namespaces
- الـ 4 Namespaces اللي بتتعمل تلقائي
- الـ ResourceQuota والـ LimitRange
- إيه الـ Objects اللي بتتحط في Namespace وإيه لأ

9.1 يعني إيه Namespace؟

عبارة عن عزل أو تقسيم الـ Cluster الكبير اللي شايل كل حاجة لمجموعة Clusters صغيرة، ولكن بشكل افتراضي (**Virtually**). وده بنعمله عشان لو عندنا أكتر من Team بيشتغلوا في نفس الـ Cluster، نفصل بينهم وشغلهم ميدخلش على بعض ولا يأثر على بعض. وفي الأغلب أشهر استخدام ليها إنك تعمل كذا environment جوه نفس الـ Cluster، وكل واحدة فيهم ليها غرض ما.

dev

معمولة عشان الـ Developers يكون ليهم بيئة خاصة بيهم يجربوا فيها كل حاجة محتاجينها للـ app، بعيداً عن الـ production عشان ميأثرش عليه (كأنها مسودة بيتجرب فيها قبل ما يطلع الشكل النهائي للـ app على الـ production).

prod

البيئة الحقيقية اللي المستخدمين شغالين عليها، ومحدش يلمسها غير بحساب.

وممكن يتعمل أكتر من Namespace زي ما نحب، المهم إنه لازم يتحددلها القواعد الخاصة بيها (**Roles**) ومقدار استهلاكها للموارد.

9.2 الـ Namespaces اللي بتتعمل تلقائي

لما بتنشئ Cluster، فـ Kubernetes بشكل تلقائي بيعمل 4 Namespaces ألا وهم:

الـ Namespace	دورها
`default`	ده الـ Namespace الافتراضي، وأي Object بتنشئه (من غير ما تحدد Namespace) بيتعمل جواه. متاح ليه يستخدم كل الموارد اللي جوه الـ Cluster. تجنّب تستخدمه كبيئة للـ production.
`kube-system`	ده الـ Namespace اللي جواه كل الـ Objects الأساسية بتاعة Kubernetes زي kube-dns (CoreDNS)، kube-proxy، الـ API server، controller manager وغيرهم. متحطش فيه شغلك أبداً.
`kube-public`	Namespace عام يقدر يشوفه أي حد حتى من غير ما يكون logged in. بيتستخدم لمعلومات الـ cluster اللي المفروض تكون متاحة للجمهور. في الغالب مش هتحتاج تتعامل معاه.
`kube-node-lease`	ده بيحتوي على ملفات صغيرة اسمها **Lease**، واحدة لكل Node. الـ Node بتبعت heartbeat كل شوية لـ Kubernetes تقوله "أنا لسه شغال وبخير". ده بيساعد الـ cluster يكتشف بسرعة لو Node وقعت.

جدول 9.1 الـ Namespaces الافتراضية

9.3 إيه اللي بيتحط في Namespace وإيه لأ؟

جوه Namespace معين

- Pods
- Services
- Deployments
- ReplicaSets
- ConfigMaps
- Secrets

على مستوى الـ Cluster كله

- الـ Nodes نفسها، مش بتتحط جوه مساحة افتراضية.
- الـ Persistent Volumes: الهاردات المتوصلة بالـ Cluster.
- الـ Namespace نفسه، مش جواه Namespace تاني!
- ClusterRoles و ClusterRoleBindings: الصلاحيات اللي على مستوى الـ Cluster كله.

ملحوظة

لو عايز تعرف أي Object في Kubernetes تبع أنهي نوع، تقدر تستخدم الأمر ده:

```
Terminal
kubectl api-resources --namespaced=true
kubectl api-resources --namespaced=false
```

لو `true` يبقى هيجيبلك الحاجات اللي جوه Namespace، ولو `false` الحاجات اللي على مستوى الـ Cluster.

9.4 أوامر الـ Namespaces

عشان نعمل Namespace عندنا طريقتين:

Terminal

```
kubectl create namespace dev
# أو اختصاراً
kubectl create ns dev
```

namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: dev
```

`kubectl get namespaces`	تعرض كل الـ Namespaces اللي عندك (أو `kubectl get ns` اختصاراً).
`kubectl get pods -n dev`	الـ Pods جوه Namespace اسمها dev (أو `--namespace=dev`).
`kubectl get pods -A`	الـ Pods في كل الـ Namespaces مرة واحدة (اختصار `--all-namespaces`).
`kubectl delete namespace dev`	تمسح الـ Namespace **وكل اللي جواه**. خلي بالك!

تشريح الأمر

```
kubectl get pods -n dev
        1    2  3
```

1. `get pods` اعرض الـ Pods.
2. `-n` اختصار `--namespace`: نفّذ الأمر جوه Namespace معينة. ينفع مع أي أمر تقريباً (apply و delete و logs و ...).
3. `dev` اسم الـ Namespace.

لاحظ إن أي أمر بتكتبه فهو بيتنفذ جوه الـ Namespace الافتراضية اللي هي الـ `default` (فكّر فيها كأنها users بتبدّل بينهم في نظام Linux). فلو عايز تغيّر الـ Namespace اللي إنت جواها لواحدة تانية عشان تكتب أوامر جواها، بدل ما كل مرة تكتب الأمر اللي عايزه وفي آخره تحدد الـ Namespace اللي عايزها (`-n dev`):

Terminal

```
# الأمر ده بيثبت الـ dev كـ Namespace افتراضية للـ context الحالي
kubectl config set-context --current --namespace=dev
```

تشريح الأمر

```
kubectl config set-context --current --namespace=dev
        ①      ②          ③          ④
```

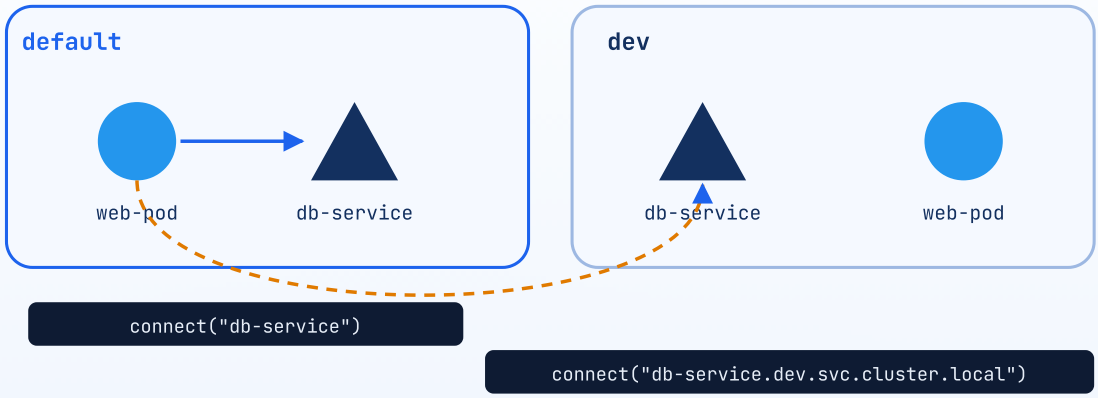
① `config` أوامر بتعدّل ملف الإعدادات بتاع kubectl (`~/.kube/config`).

② `set-context` عدّل الـ Context. الـ Context = (Namespace + User + Cluster) افتراضية.

③ `--current` عدّل الـ Context اللي إنت شغال بيه دلوقتي.

④ `--namespace=dev` خلي الـ Namespace الافتراضية dev. تتأكد بـ `kubectl config view --minify | grep namespace`.

9.5 إزاي الـ Pods بتكلم بعضها؟



شكل 9.1 في نفس الـ Namespace بالاسم على طول، ومن Namespace تانية بالعنوان الكامل.

لو في نفس الـ Namespace

الـ `web-pod` اللي جوه الـ default عايز يكلم الـ `db-service` اللي معاه في نفس الأوضة؟ هينادي عليه باسمه علطول:

```
mysql.connect("db-service")
```

لو في Namespace تانية

الـ `web-pod` اللي جوه الـ default عايز يكلم الـ `db-service` اللي في dev؟ لازم يكتب العنوان بالكامل عشان الـ DNS يوجهه صح:

```
"db-service.dev.svc.cluster.local"
```

تشريح الأمر

```
db-service.dev.svc.cluster.local
```
(1) (2) (3) (4)

1	`db-service`	اسم الـ Service.
2	`dev`	الـ Namespace اللي الـ Service فيها.
3	`svc`	ثابتة: معناها إن الاسم ده بتاع Service.
4	`cluster.local`	الـ Domain الافتراضي بتاع الـ Cluster.

معلومة مهمة

الـ Namespaces بتعزل **الأسامي والصلاحيات والموارد**، لكن مش بتعزل الشبكة: أي Pod يقدر يكلم أي Pod في أي Namespace طالما عارف العنوان. لو عايز تمنع ده بتستخدم حاجة اسمها **NetworkPolicy**.

9.6 الـ ResourceQuota: تحجيم الموارد للفرق

من غير حدود، فريق واحد ممكن يعمل Deploy لحاجة تاكل كل الـ CPU والـ RAM، ويوقّع بقية الفرق في الـ Cluster. الـ ResourceQuota هو "عسكري المرور" اللي بيمسك الـ Namespace ويقولها: "ليكِ حدود مش هتعديها".

شكل الـ YAML file اللي بنعمل بيها الكلام ده:

resource-quota.yaml

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: backend-team-quota
  namespace: my-team
spec:
  hard:
    pods: "20"
    services: "10"
    configmaps: "30"
    secrets: "30"
    persistentvolumeclaims: "10"
    requests.cpu: "8"
    limits.cpu: "16"
```

(1) اسم الـ ResourceQuota (بنسميه براحتنا).

(2) حددنا الـ Namespace المستهدفة.

(3) أقصى عدد Pods مسموح بيه هو 20. و `hard` يعني حد صارم مينفعش يتعدّى.

(4) أقصى عدد Services هو 10.

(5) هيتشرحوا بعدين... متقلقش.

(6) مجموع حجز الـ CPU للـ Pods ميزيدش عن 8 cores.

(7) مجموع سقف الـ CPU ميزيدش عن 16 cores: بعد ما كل Pod اتوفرله الـ cores اللي محتاجها بشكل مبدئي وابتدوا يشتغلوا، فآخرهم كلهم في الاستهلاك هو ده.

8 نفس فكرة الـ CPU بالظبط بس لـ Memory.	`requests.memory: 16Gi` `limits.memory: 32Gi`

> **⚠ خلي بالك**
>
> أول ما تحط ResourceQuota فيها `requests.cpu` أو `limits.cpu` على Namespace، أي Pod جديد فيها **لازم** يكون كاتب الـ requests والـ limits دي، وإلا هيترفض. وده بالظبط اللي الـ LimitRange بيحلّه.

Terminal

```
kubectl apply -f resource-quota.yaml
kubectl describe resourcequota backend-team-quota -n my-team    # المستخدم مقابل الحد
```

9.7 الـ LimitRange: القيم الافتراضية للـ Containers

لو الـ Developer نسي يكتب الـ resources (الـ requests والـ limits) جوه الـ YAML بتاع الـ Pod بتاعه، الـ Container ده ممكن يسرح في الـ Node وياكل الأخضر واليابس. الحل: الـ LimitRange بيحط "كتالوج افتراضي" للـ Namespace. لو الـ Developer نسي يحدد، الـ LimitRange بينزل عليه بالقيم الافتراضية أوتوماتيك، وكمان بيحددله سقف وأرضية للطلبات بتاعته.

limit-range.yaml

1 الاسم والـ Namespace اللي هيتطبق عليها.	`apiVersion: v1`
	`kind: LimitRange`
	`metadata:`
	`  name: default-container-limits`
	`  namespace: my-team`

2 القواعد دي لكل Container.	`spec:`
	`  limits:`
	`  - type: Container`

3 القيم الافتراضية للـ **limits** اللي هتنزل أوتوماتيك لو الـ developer نسي يحط حاجة.	`  default:`
	`  cpu: "500m"`
	`  memory: "256Mi"`

4 القيم الافتراضية للحجز المبدئي (**requests**) لو نسيها.	`  defaultRequest:`
	`  cpu: "200m"`
	`  memory: "128Mi"`

5 هنا هو منسيش يحط القيم، هو حطها بالفعل، ولكن إحنا بنحددله range معين يختار فيه: أقصى حد مسموح لـ Container واحد يطلبه، عشان مفيش Pod واحد يستهبل ويطلب الـ quota كلها.	`  max:`
	`  cpu: "4"`
	`  memory: "4Gi"`

6 أقل حد مسموح بيه، عشان ميكونش الـ Pod صغير لدرجة إنه ملوش لازمة ويستهلك أسماء ع الفاضي.	`  min:`
	`  cpu: "50m"`
	`  memory: "32Mi"`

	ResourceQuota	LimitRange
بيحدد	المجموع الكلي للـ Namespace كلها	الحدود لكل Container (أو Pod) لوحده
بيحط Defaults؟	لأ	أيوه (`default` و `defaultRequest`)
مثال	الفريق كله ميعدّيش 16 core	كل Container بين m50 و 4 cores

جدول 9.2 ResourceQuota مقابل LimitRange

معلومة مهمة

فاضل آخر حاجة في الـ Namespaces ألا وهي الـ **RBAC** (Role-Based Access Control)، ولكن دي هنأجلها لبعدين في الآخر عشان نقدر نفهمها كويس.

ملخص الفصل

- الـ Namespace تقسيم افتراضي للـ Cluster، أشهر استخدام ليه dev و prod أو فرق مختلفة.
- فيه 4 تلقائي: default و kube-system و kube-public و kube-node-lease.
- الـ Nodes والـ PVs والـ ClusterRoles على مستوى الـ Cluster، مش جوه Namespace.
- `-n` لـ Namespace معينة، `-A` لكلهم، و `set-context --current --namespace` لتغيير الافتراضي.
- من Namespace تانية: `service.namespace.svc.cluster.local`.
- الـ ResourceQuota للمجموع، والـ LimitRange للـ Defaults والحدود لكل Container.

أسئلة مراجعة

1. Pod في `dev` عايز يكلم Service اسمها `api` في `prod`، هيكتب العنوان إزاي؟
2. ليه مينفعش تحط شغلك في `kube-system`؟
3. حطيت ResourceQuota فيها `limits.memory` وبعدين عملت Pod من غير resources، إيه اللي هيحصل؟ وإيه الحل؟
4. اكتب الأمر اللي يخلي `staging` هي الـ Namespace الافتراضية.

صح ولا غلط؟

1. الـ Nodes بتتحط جوه Namespace. صح / غلط
2. مسح Namespace بيمسح كل اللي جواها. صح / غلط
3. الـ Namespaces بتمنع الـ Pods إنها تكلم بعض بالشبكة. صح / غلط

10

المرحلة 3 · إدارة الـ Workloads

الفصل العاشر

الـ Deployment: إدارة الـ Pods باحتراف

في الحاجات اللي فاتت شفنا مجموعة Options بتتعمل على الـ Pod في الـ YAML بتاعها زي: ReplicaSet و Probes و Volumes و Resources (Requests & Limits) وغيرها. مع كتر الحاجات دي بنحتاج ننظم الـ YAML بتاع الـ Pod أكتر، وكمان محتاجين مرونة عالية في التحكم في الـ Pod دي، فعشان كده موجود الـ Deployment.

بعد الفصل ده هتكون عارف

- ليه الـ Deployment مع إن الـ ReplicaSet موجودة
- مشكلة تحديث الـ image من غير Deployment
- العلاقة Deployment ← ReplicaSet ← Pods
- الـ Rolling Update والـ Rollback
- الـ Scaling بأمر واحد

10.1 يعني إيه Deployment؟

هو عبارة عن Object بيدير الـ Pod اللي جواه، بالإضافة لأي Options أو خلينا نقول Objects مرتبطة بالـ Pod دي.

تسأل سؤال وتقولي: ما أنا أقدر أكتب YAML للـ Pod ومعاها كل الـ Objects اللي محتاجاها عادي، إيه لازمته إني أحطها جوه Object جديد اسمه Deployment؟

لسببين: الأول هو فكرة الـ ReplicaSet، وإنه عشان نضمن إن فيه عدد X نسخ من الـ Pod دي شغالة، ولو حاجة وقعت تقوم تاني. ولكن النقطة دي مش جوهرية أوي، لأن زي ما قلنا عادي أحط ReplicaSet للـ Pod وخلاص، ودي حقيقة، ومن هنا ننتقل للسبب التاني.

10.2 السبب الحقيقي: الـ Update

تخيل السيناريو ده: عايز تغيّر الـ image اللي بيتعمل منها الـ Container اللي جوه الـ Pod من `nginx:1.25` لـ `nginx:1.26`.

- لو عدّلت الـ image في الـ YAML بتاع الـ ReplicaSet وعملت (`kubectl apply`)، مفيش حاجة هتحصل للـ Pods الموجودة. اللي اتعملت وشغالة من الـ YAML القديم هتفضل معمولة من الـ Image القديمة.
- وهنا هتظهر الحاجة للـ Deployment، لأنه حتى لو حاطط ReplicaSet في الـ YAML فده مش كفاية، لأنه بيراقب بس **"العدد المطلوب"**، مش بيتأكد من إن "النسخ اللي شغالة دي مطابقة للي مكتوب في الـ YAML ولا لأ (لو اتحدّث يعني)".
- فكده الـ Pods القديمة هتفضل شغالة بـ `nginx:1.25` للأبد، والتحديث مش هيتطبق إلا لو Pod جديدة اتعملت (يعني لو واحدة ماتت بالصدفة).
- أو هتحتاج تحدّث إنت بإيدك بنفسك، يعني هتمسح كل الـ Pods يدوياً عشان الـ ReplicaSet يعمل جداد بالـ image الجديدة. وده معناه **downtime** فعلي لو مسحتهم كلهم مع بعض وتستنى الـ ReplicaSet تقومهم تاني. شوف الوقت اللي هتستغرقه

العملية دي وتأثيرها على أي Traffic هييجي على الـ Pods دي وهي مش شغالة.

10.3 الـ Structure: مين بيدير مين؟



شكل 10.1 إنت بتتعامل مع الـ Deployment بس، هو اللي بيدير الـ ReplicaSet، وهي اللي بتدير الـ Pods.

حاجة أخيرة بيقدّمها وهي الـ **Scaling**: زيادة أو تقليل عدد الـ Pods (في الـ ReplicaSet) بأمر واحد، بدل ما تعدّل في الـ YAML وتعمل Save وبعدها تعمل (`kubectl apply`)، هو بيختصرلك العملية دي.

10.4 شكل الـ YAML بتاعه

شكل الـ YAML بتاعه مفهوش جديد، زي ما قلتلك هو تجميع للي فات:

deployment.yaml

1 الـ Deployment في الـ API group `apps/v1`.	`apiVersion: apps/v1` `kind: Deployment`

2 اسم الـ Deployment، والـ Namespace، و Label على الـ Deployment نفسه.	`metadata:`
	`  name: webapp`
	`  namespace: default`
	`  labels:`
	`  app: webapp`

3 عدد النسخ (الـ ReplicaSet).	`spec:` `  replicas: 3`

4 الـ Pods اللي الـ Deployment مسؤول عنها، بالـ Labels.	`  selector:`
	`  matchLabels:`
	`  app: webapp`

5 بداية شكل الـ Pod. الـ labels هنا **لازم** تطابق الـ selector.	`  template:`
	`  metadata:`
	`  labels:`
	`  app: webapp`

6 نفس الـ spec بتاع الـ Pod اللي اتعلمناه. علامة `-` معناها عنصر في list.	`  spec:`
	`  containers:`
	`  - name: webapp`
	`  image: nginx:1.25`
	`  ports:`
	`  - containerPort: 80`

7 الـ Requests والـ Limits زي فصل 6. وهنا كمان مكان الـ Probes.	`resources:`
	`  requests:`
	`  cpu: "100m"`
	`  memory: "128Mi"`
	`  limits:`
	`  cpu: "200m"`
	`  memory: "256Mi"`

10.5 الـ Rolling Update: تحديث من غير Downtime

إضافة

أحلى حاجة في الـ Deployment إنه لما تغيّر الـ image، مش بيمسح كل الـ Pods مرة واحدة. بيعمل **ReplicaSet جديدة** بالـ image الجديدة، ويزوّدها Pod Pod، وفي نفس الوقت يقلّل القديمة Pod Pod، لحد ما الجديدة تبقى 3 والقديمة 0. وطول الوقت فيه Pods شغالة بتستقبل Traffic.



شكل 10.2 الـ Rolling Update: الـ ReplicaSet الجديدة بتكبر والقديمة بتصغر تدريجياً.

deployment.yaml · strategy

1 الـ Default. البديل `Recreate`: يمسح الكل الأول وبعدين يعمل الجديد (فيه downtime).	`spec:`
	`  strategy:`
	`  type: RollingUpdate`

2 أقصى عدد Pods **زيادة** عن الـ replicas وقت التحديث (الـ Default 25%).	`  rollingUpdate:` `  maxSurge: 1`
3 أقصى عدد Pods مسموح يبقوا مش جاهزين وقت التحديث. `0` = مفيش أي نقص في الخدمة (الـ Default 25%).	`  maxUnavailable: 0`

Best Practice

الـ Rolling Update بيعتمد على الـ **Readiness Probe** عشان يعرف إن الـ Pod الجديدة جاهزة قبل ما يكمل ويمسح قديمة. من غيرها ممكن يمسح القديم ويحوّل الـ Traffic على Pods لسه مش جاهزة.

تحديث الـ image ومتابعته

تشريح الأمر

```
kubectl set image deployment/webapp webapp=nginx:1.26
          ①       ②              ③      ④
```

1. `set image` — غيّر الـ image بتاعة Container جوه Object موجود (من غير ما تفتح الـ YAML).
2. `deployment/webapp` — النوع والاسم، والـ `/` بتفصل بينهم. ينفع تكتبها `deploy/webapp`.
3. `webapp=` — اسم الـ Container (من `containers.name`) وبعده `=`.
4. `nginx:1.26` — الـ image الجديدة بالـ tag.

خلي بالك

الأحسن في الشغل الحقيقي إنك تعدّل الـ image في ملف الـ YAML نفسه وتعمل `kubectl apply -f deployment.yaml`، عشان الملف (اللي غالباً في Git) يفضل مطابق للي شغال. `set image` سريع للتجربة بس.

`kubectl rollout status deploy/webapp`	تابع التحديث لحد ما يخلص (أو يفشل).
`kubectl rollout history deploy/webapp`	النسخ (Revisions) اللي اتعملت قبل كده.
`kubectl rollout undo deploy/webapp`	ارجع للنسخة اللي قبلها (Rollback).
`kubectl rollout undo deploy/webapp --to-revision=2`	ارجع لـ Revision معينة برقمها من الـ history.
`kubectl rollout restart deploy/webapp`	اعمل restart لكل الـ Pods بالتدريج (مفيد بعد تغيير ConfigMap مثلاً).
`kubectl rollout pause deploy/webapp`	وقّف التحديث مؤقتاً (و `resume` تكمّله).

Terminal · توضيح للناتج

```
moaz@Linux:~$ kubectl rollout status deploy/webapp
Waiting for deployment "webapp" rollout to finish: 1 out of 3 new replicas have been
updated...
Waiting for deployment "webapp" rollout to finish: 2 out of 3 new replicas have been
updated...
deployment "webapp" successfully rolled out
moaz@Linux:~$ kubectl get rs
NAME               DESIRED   CURRENT   READY   AGE
webapp-5d8f7c9b6   0         0         0       10m
webapp-7b9c4d2f1   3         3         3       40s
```

لاحظ إن الـ ReplicaSet القديمة لسه موجودة بس بـ 0 Pods، وده اللي بيخلي الـ `rollout undo` سريع: بيكبّرها تاني ويصغّر الجديدة.

10.6 الـ Scaling بأمر واحد إضافة

تشريح الأمر

```
kubectl scale deployment webapp --replicas=5
```

1. `scale` غيّر عدد النسخ.
2. `deployment webapp` النوع والاسم (بمسافة هنا بدل `/`، الاتنين شغالين).
3. `--replicas=5` العدد الجديد. هيزود أو يقلل الـ Pods فوراً.

معلومة مهمة

لو عملت scale بالأمر وبعدين عملت `apply` للملف القديم اللي فيه `replicas: 3`، العدد هيرجع 3. وعشان كده لو عايز Scaling أوتوماتيك على حسب الضغط، فيه الـ **HPA** اللي هنشوفه في المرحلة الأخيرة.

تشريح الأمر

```
kubectl create deployment web --image=nginx --replicas=3 --dry-run=client -o yaml > deployment.yaml
```

1. `create deployment web` اعمل Deployment اسمه web.
2. `--image=nginx` الـ image بتاعة الـ Container.
3. `--replicas=3` عدد النسخ.
4. `--dry-run=client` متعملوش بجد في الـ Cluster، جرّب بس على جهازك.
5. `-o yaml` طلّع الناتج بصيغة YAML.
6. `>` علامة الـ redirect في الـ shell: احفظ الناتج في ملف بدل ما يظهر على الشاشة (وتمسح أي حاجة كانت فيه).
7. `deployment.yaml` اسم الملف. كده عملت قالب جاهز تعدّل عليه بدل ما تكتب من الصفر.

ملخص الفصل

- الـ Deployment بيدير ReplicaSet، والـ ReplicaSet بتدير الـ Pods.
- الـ ReplicaSet لوحدها مش بتحدّث الـ Pods القديمة لما الـ image تتغير.
- الـ Deployment بيعمل Rolling Update من غير downtime، وتقدر ترجع بـ `rollout undo`.
- `maxSurge` و `maxUnavailable` بيتحكموا في سرعة وأمان التحديث.
- `kubectl scale` للـ Scaling السريع، والملف هو مصدر الحقيقة.

؟ أسئلة مراجعة

1. ليه الـ ReplicaSet لوحدها مش كفاية عشان تحدّث الـ image؟
2. اشرح اللي بيحصل للـ ReplicaSets وقت الـ Rolling Update.
3. عملت تحديث والنسخة الجديدة فيها bug، إيه الأمر اللي يرجّعك؟
4. إمتى تستخدم `strategy: Recreate` بدل `RollingUpdate` ؟

صح ولا غلط؟

1. الـ Deployment بيمسح كل الـ Pods القديمة مرة واحدة وهو بيحدّث. صح / غلط
2. `maxUnavailable: 0` معناها إن عدد الـ Pods الجاهزة مش هيقل عن المطلوب. صح / غلط
3. الـ `--dry-run=client` بيعمل الـ Object في الـ Cluster. صح / غلط

11

المرحلة 3 · إدارة الـ Workloads

الفصل الحادي عشر

الـ DaemonSet: Pod على كل Node

شبه الـ Deployment، بس الفرق إن الـ DaemonSet بتكون لـ Pods ليها مهام متعلقة بالـ Node نفسها، زي الـ Logs والمراقبة وما إلى ذلك. ومعاها هنتعلم إزاي نختار Nodes معينة بالـ Node Selector والـ Node Affinity.

بعد الفصل ده هتكون عارف

- الفرق بين الـ DaemonSet والـ Deployment
- الـ Images المشهورة للـ DaemonSets
- إزاي الـ Pod يقرا ملفات الـ Node بـ hostPath
- تحديد Nodes معينة بـ nodeSelector و nodeAffinity
- حل المشاكل الشائعة (taints و tolerations)

11.1 يعني إيه DaemonSet؟

شبه الـ Deployment، بس الفرق إن الـ DaemonSet بتكون لـ Pods ليها مهام متعلقة بالـ Node نفسها، زي الـ Logs والمراقبة وما إلى ذلك.

يعني كأنك عملت Pods عادية بس شغلها مرتبط بالـ Node اللي عليها، فبنحطها جوه Object اسمه DaemonSet، هتشغّلها على **كل** الـ Nodes اللي موجودة (أو اللي ممكن تتعمل مستقبلاً) في الـ Cluster بشكل تلقائي، كأنها حاجة افتراضية على كل Node.

- ممكن أحدد Nodes معينة بس يشتغل عليها في حالات محدودة.
- ممكن أعمل كذا DaemonSet على كل Node. يعني مثلاً أنا محتاج على كل Node مجموعة DaemonSets، كل واحدة فيهم جواها Pod بتعمل حاجة مختلفة عن الـ Pod اللي جوه الـ DaemonSet التانية: واحدة بتجمع الـ Logs، وواحدة بتراقب استهلاك الـ Resources، ....



شكل 11.1 الـ Deployment بيوزع عدد معين على أي Nodes، والـ DaemonSet بيحط Pod واحدة بالظبط على كل Node.

11.2 شكل الـ YAML بتاعه

نيجي لشكل الـ YAML بتاعه (شبه الـ Deployment جداً، ولكن الفرق إنه مفيش `replicas`، لأن العدد = عدد الـ Nodes):

daemonset.yaml

1 اسم الـ DaemonSet نفسها (من اختيارك)، في أنهي Namespace، و Label على الـ DaemonSet نفسها.	`apiVersion: apps/v1`
	`kind: DaemonSet`
	`metadata:`
	`  name: log-collector`
	`  namespace: monitoring`
	`  labels:`
	`  app: log-collector`

2 إيه الـ Pods اللي الـ DaemonSet مسؤول عنها. لازم يطابق labels الـ Pod تحت.	`spec:`
	`  selector:`
	`  matchLabels:`
	`  app: log-collector`

3 مواصفات الـ Pod اللي هيعمله. هنختار Image ليها علاقة بالهدف بتاعنا من الـ Pod اللي هنقومها (الجدول الجاي).	`  template:`
	`  metadata:`
	`  labels:`
	`  app: log-collector`
	`  spec:`
	`  containers:`
	`  - name: fluentd`
	`  image: fluentd:v1.14`

4 هنحط حدود للـ resources اللي مسموح للـ Pod دي تستخدمها، ومنطقي نخلي استهلاكها قليل، لأنه مش دورها تبع الـ App نفسه فتحتاج تستهلك كتير، لا هي بتراقب بس.	`  resources:`
	`  requests:`
	`  cpu: "50m"`
	`  memory: "64Mi"`
	`  limits:`
	`  cpu: "100m"`
	`  memory: "128Mi"`

5 `volumeMounts` جوه الـ containers: المسار **جوه الـ Container** اللي الملفات اللي هناخدها من الـ Node هتظهر فيه.	`  volumeMounts:`
	`  - name: varlog`
	`  mountPath: /var/log`

6 `volumes` جوه الـ spec (مش الـ containers): مكان الملفات **على الـ Node** اللي هنجيب منه. والـ `name` هو اللي بيربط الاتنين ببعض.	`  volumes:`
	`  - name: varlog`
	`  hostPath:`
	`  path: /var/log`

زي ما قلنا فوق، الـ Pod أو الـ Image تحديداً هتحتاج تاخد بيانات من الـ Node نفسها، وده منطقي، لأن كل البيانات اللي هتراقبها أو هتعرضها بتكون عبارة عن ملفات جوه الـ Node نفسها، فلازم أخليها تكلم الملفات دي على الـ Node عشان تاخد البيانات اللي محتاجاها، فأكيد هنستخدم الـ **Volumes**. والـ DaemonSet في الغالب بيستخدم volumes من نوع `hostPath` عشان يوصل مباشرة لملفات الـ Node نفسها.

الـ Image	بتعمل إيه؟
(Node Exporter) Prometheus	مراقبة الـ Resources (CPU، Memory، Disk) + ميزة Alerts لما الاستهلاك يوصل للـ Max يبلّغك.
Filebeat / Fluentd	جمع الـ Logs من الـ Node اللي هي عليها.
Weave / Calico	إدارة الشبكة.

جدول 11.1 Images مشهورة للـ DaemonSets

> **دقة**
>
> اللي بيشتغل كـ DaemonSet في المراقبة هو الـ **node-exporter** (بيطلّع أرقام كل Node)، والـ Prometheus نفسه بيشتغل غالباً كـ Deployment أو StatefulSet ويجمع من كل الـ exporters. وفي الشبكات، Weave اتوقف تطويره، والمشهور النهارده Calico و **Cilium**.

زي ما إنت لاحظت، أنا غالباً هحتاج أعمل الـ 3 حاجات مع بعض، مش هختار مراقبة الـ Resources بس مثلاً، عشان كده قلت في الأول ممكن أعمل أكتر من DaemonSet، وتبقى كل واحدة لحاجة فيهم، ويشتغلوا كلهم على كل Node موجودة عادي. مش هندخل في تفاصيل الحتة دي أوي، ولكن يكفينا حالياً نبقى عارفين الـ Images دي واستخدامها. أما بقى هتبلّغك إزاي أو هتبعتلك التقارير إزاي، فيكفينا نعرف إنه بيبقى ليها Configurations خاصة بيها بتظبطها، وبتبعتلك التقارير دي عن طريق Dashboard خاصة بيها. إنما هنا إنت بس بتظبط الـ Pod من ناحيتك إنت كـ Cluster: آخرها فين كاستهلاك، وهتستخدم أنهي Image، وهتاخد بياناتها منين.

Terminal

```
kubectl apply -f daemonset.yaml
kubectl get daemonset -n monitoring      # أو ds اختصاراً
```

11.3 تحديد Nodes معينة

عايزين نتكلم على جزئية إننا إزاي نحدد Nodes معينة نشغّل عليها DaemonSet وغيرها لأ. ممكن نعمل ده بطريقة من اتنين: **(1) Node Selector** أو **(2) Node Affinity**.

Node Selector (1)

بتخلي الـ DaemonSet تشتغل على الـ Nodes اللي عندها الـ label معينة. بيعمل Match من الآخر.

(1) لازم الأول تعمل label على الـ Nodes اللي عايزها، بالطريقة دي:

Terminal

```
kubectl label nodes <node-name> <key>=<value>
kubectl label nodes node-1 hardware=gpu
kubectl label nodes node-2 hardware=gpu
```

تشريح الأمر

```
kubectl label nodes node-1 hardware=gpu
```
(1) (2) (3)

1. `label nodes` — ضيف Label على Node (نفس فكرة الـ Pods بالظبط).
2. `node-1` — اسم الـ Node زي ما بيظهر في `kubectl get nodes`. والـ `<>` في الشكل العام معناها "حط القيمة بتاعتك هنا" ومش بتتكتب.
3. `hardware=gpu` — الـ Label: key هو hardware و value هو gpu.

عشان تتأكد إنه اتضاف:

Terminal
```
moaz@Linux:~$ kubectl get nodes --show-labels
NAME     STATUS   LABELS
node-1   Ready    hardware=gpu,...
node-2   Ready    hardware=gpu,...
node-3   Ready    ...
```

(2) هتضيف الجزء ده في الـ YAML:

YAML
```
  template:
    spec:
      nodeSelector:
        hardware: gpu       # name of selected label/s of the nodes
```

لو ضفت Node جديدة وحطيت عليها الـ label دي، الـ DaemonSet بيتعمل عليها أوتوماتيك.

وإزاي تشيل الـ label لو غلطت:

Terminal
```
kubectl label nodes <node-name> <label>-
kubectl label nodes node-1 hardware-
```

Node Affinity (2)

بتديك نفس نتيجة الـ nodeSelector، بس بإمكانيات أكتر، يعني تقدر تقول اختار الـ Node دي "أو" دي، وتحدد نطاقات. وزي الـ nodeSelector بالظبط، لو الشرط مش متحقق، الـ Pod مش بيتعمل خالص.

nodeAffinity

1 بنحدد قواعد اختيار الـ Node.	`template:`
	`  spec:`
	`  affinity:`
	`  nodeAffinity:`

2 الاسم الطويل ده معناه: الشرط **إجباري** وقت الجدولة، ولو الـ Label اتغيرت بعد ما الـ Pod اشتغل، متطردوش. (فيه نوع تاني `preferred...` يعني "يُفضَّل" مش إجباري).	`requiredDuringSchedulingIgnoredDuringExecution:`
3 الـ `nodeSelectorTerms` بتاخد List من الـ `matchExpressions`، واللي هي تعتبر شرط.	`nodeSelectorTerms:` `- matchExpressions:`

4 الـ `key` هو اسم الـ label اللي هنختاره، والـ `operator` هو الشرط نفسه، والـ `values` القيم اللي هيتعمل عليها الشرط (بندوّر فيها). في المثال ده بنقوله: لو قيمة الـ label بتاعة الـ Node في (In) القيم دي، اختارها.	`- key: hardware`
	`operator: In`
	`values:`
	`- gpu`
	`- gpu-v2`

الـ Operator	معناه
`In`	القيمة موجودة في القايمة (values).
`NotIn`	القيمة مش موجودة في القايمة (values).
`Exists`	الـ key موجود أصلاً بشكل عام (إن فيه label بتاعة الـ Node اسمها كده)، أياً كانت قيمتها. ومش بتكتب values.
`DoesNotExist`	الـ key مش موجود خالص أصلاً.
`Lt` / `Gt`	إضافة أكبر من / أصغر من (للقيم الرقمية).

جدول 11.2 الـ Operators

بعد ما تخلص كتابة الملف احفظه واعمل: `kubectl apply -f daemonset.yaml`

معلومة مهمة

الـ `nodeSelector` والـ `nodeAffinity` مش خاصين بالـ DaemonSet بس، بتقدر تستخدمهم في أي `template.spec` بتاع Deployment أو Pod عادي.

11.4 مشاكل شائعة

مشكلة 1 الـ Pod مش بيتعمل على الـ Nodes الـ control plane

بشكل افتراضي، الـ DaemonSet مش بيشتغل على الـ Nodes الـ control plane (master) لأنها عندها `taint` خاص بيمنع الـ Pods العادية. والـ taint زي "يافطة ممنوع الدخول" على الـ Node، والـ toleration هي "التصريح" اللي بيخلي Pod معينة تعدّي.

الحل لو محتاجه يشتغل عليها:

YAML

```yaml
spec:
  template:
    spec:
      tolerations:
      - key: node-role.kubernetes.io/control-plane
        operator: Exists
        effect: NoSchedule
```

مشكلة 2 الـ READY مختلف عن الـ DESIRED

يعني فيه Node أو أكتر الـ Pod مش شغال عليها. الأسباب الشائعة:

- الـ Node عندها taint والـ DaemonSet معندوش toleration ليها.
- مشكلة في الـ image أو الـ resources.
- الـ nodeSelector مش مطابق.

Terminal

```
kubectl describe daemonset log-collector    # شوف Events
kubectl get pods -o wide                    # شوف على أنهي Node الـ Pod مش موجود
```

مشكلة 3 موارد زيادة على الـ Nodes

لأن الـ DaemonSet بيشغّل Pod على كل Node، لو نسيت تحط resource limits، Pod واحد ممكن ياكل موارد الـ Node كلها ويأثر على باقي التطبيقات.

الحل: دايماً حط `resources.requests` و `resources.limits`.

نصيحة: الـ DaemonSets في Kubernetes نفسها

Kubernetes نفسها بتستخدم DaemonSets داخلياً، زي `kube-proxy` اللي بيشتغل على كل Node ومسؤول عن الـ networking. لو عملت `kubectl get daemonset -n kube-system` هتلاقيه هناك.

ملخص الفصل

- الـ DaemonSet بيشغّل Pod واحدة على كل Node (أو Nodes مختارة)، ومفيهوش replicas.
- بيستخدم في الـ Logs والمراقبة والشبكات، وغالباً بـ `hostPath` volumes.
- `volumeMounts` = المسار جوه الـ Container، و `volumes` = المصدر على الـ Node.
- `nodeSelector` للـ match البسيط، و `nodeAffinity` للشروط (In و NotIn و Exists و ...).
- الـ taints بتمنع، والـ tolerations بتسمح.

أسئلة مراجعة

1. ليه الـ DaemonSet مفيهوش `replicas` ؟
2. اشرح الفرق بين `volumeMounts` و `volumes` في مثال الـ log-collector.
3. عايز Pod تشتغل على Nodes عليها `disk=ssd` أو `disk=nvme` ، هتستخدم إيه؟ اكتب الشرط.
4. DaemonSet شغال على 2 Nodes بس من 3، إيه أول حاجة تعملها؟

صح ولا غلط؟

1. لو ضفت Node جديدة للـ Cluster، الـ DaemonSet بيعمل عليها Pod أوتوماتيك. صح / غلط
2. الـ operator `Exists` محتاج values. صح / غلط
3. kube-proxy شغال كـ DaemonSet. صح / غلط

12

المرحلة 3 · إدارة الـ Workloads

الفصل التاني عشر

الـ Jobs والـ CronJobs: شغل يخلص ويقفل

الـ Deployment معمول لحاجة المفروض تفضل شغالة على طول، زي موقع أو API. طب لو عندي شغلانة تتعمل مرة وتخلص، زي Backup أو Migration أو إرسال تقرير؟ هنا بييجي دور الـ Job، ولو عايزها تتكرر على مواعيد يبقى CronJob.

بعد الفصل ده هتكون عارف

- الفرق بين Job و Deployment
- كتابة CronJob وفهم الـ schedule
- الـ completions والـ parallelism والـ backoffLimit
- تنضيف الـ Jobs القديمة

إضافة

الشرح الأصلي ذكر الـ Jobs كعنوان من غير محتوى، فالفصل ده **إضافة** علشان الكتاب يبقى كامل، ومكتوب بنفس الأسلوب.

12.1 يعني إيه Job؟

تخيل إنك عايز تعمل Backup للـ Database. لو حطيته في Deployment، أول ما الـ Backup يخلص والـ Container يقفل، الـ Deployment هيفتكره وقع ويقومه تاني، ويفضل يعمل Backup ورا Backup للأبد! المشكلة إن الـ Deployment فاكر إن "الطبيعي" إن الحاجة تفضل شغالة.

الـ **Job** عكسه: الطبيعي عنده إن الـ Pod يخلص شغله ويقفل بنجاح (exit code 0). لو فشل يعيد المحاولة، ولو نجح يقف ويقول "خلصت".

job.yaml

1 الـ Jobs في الـ API group اسمه `batch/v1`.	`apiVersion: batch/v1`
	`kind: Job`
	`metadata:`
	`  name: db-backup`

2 لازم الشغلانة تنجح كام مرة عشان الـ Job يعتبر نفسه خلص (الـ Default 1).	`spec:` `  completions: 1`
3 كام Pod يشتغلوا في نفس الوقت (الـ Default 1).	`  parallelism: 1`
4 لو فشل، يحاول كام مرة قبل ما يستسلم ويعلن فشل الـ Job (الـ Default 6).	`  backoffLimit: 3`
5 أقصى وقت للـ Job كله. لو عدّاه يتقفل ويعتبر فشل.	`  activeDeadlineSeconds: 600`

6 امسح الـ Job والـ Pods بتاعته أوتوماتيك بعد ساعة من ما يخلص.	`ttlSecondsAfterFinished: 3600`

7 لازم تكون `OnFailure` أو `Never`، لأن `Always` (بتاعة الـ Deployment) مش مسموحة هنا.	`template:`
	`  spec:`
	`  restartPolicy: OnFailure`

8 الأمر اللي هيتنفذ. `sh -c` معناها "نفّذ النص ده كأمر shell"، عشان علامة `>` (احفظ الناتج في ملف) تشتغل.	`  containers:`
	`  - name: backup`
	`  image: postgres:16`
	`  command: ["sh", "-c", "pg_dump -h db -U app appdb > /backup/db.sql"]`

Terminal · توضيح للناتج

```
moaz@Linux:~$ kubectl apply -f job.yaml
job.batch/db-backup created
moaz@Linux:~$ kubectl get jobs
NAME        STATUS     COMPLETIONS   DURATION   AGE
db-backup   Complete   1/1           12s        30s
moaz@Linux:~$ kubectl get pods
NAME              READY   STATUS      RESTARTS   AGE
db-backup-x7k2m   0/1     Completed   0          30s
```

لاحظ إن الـ STATUS بتاعة الـ Pod بقت `Completed` مش Running، والـ READY `0/1`، وده الطبيعي لـ Job خلص.

`kubectl logs job/db-backup`	الـ logs بتاعة الـ Pod اللي الـ Job عمله.
`kubectl describe job db-backup`	تفاصيل ومحاولات الفشل في الـ Events.
`kubectl delete job db-backup`	تمسح الـ Job والـ Pods بتاعته.

completions: 5 و parallelism: 1

5 مرات ورا بعض، Pod Pod. مفيد لو عندك 5 دفعات بيانات لازم تتعالج بالترتيب.

completions: 5 و parallelism: 5

5 Pods في نفس الوقت. أسرع، بس بيستهلك موارد أكتر.

12.2 الـ CronJob: Job على مواعيد

لو عايز الـ Backup يتعمل كل يوم الساعة 2 بالليل، مش هتصحى كل يوم تعمل `kubectl apply`. الـ **CronJob** بيعمل Job جديد أوتوماتيك على الـ schedule اللي تحدده، بنفس فكرة الـ cron في Linux.

فاكر الـ cron؟

صيغة الـ `schedule` هي نفس الخمس خانات بتاعة الـ cron اللي شرحناها في كتاب الـ Terminal (فصل 12): دقيقة، ساعة، يوم الشهر، الشهر، يوم الأسبوع. فـ `"0 2 * * *"` = كل يوم الساعة 2:00.

cronjob.yaml

1 نفس الـ API group.	`apiVersion: batch/v1`
	`kind: CronJob`
	`metadata:`
	`  name: nightly-backup`

2 الميعاد: كل يوم الساعة 2:00 الصبح (شوف التشريح تحت).	`spec:` `  schedule: "0 2 * * *"`
3 المنطقة الزمنية. لو مكتبتهاش هيستخدم توقيت الـ Cluster (غالباً UTC).	`  timeZone: "Africa/Cairo"`
4 لو الـ Job اللي فات لسه مخلصش، متبدأش واحد جديد. (`Allow` الـ Default، و `Replace` يوقف القديم ويبدأ الجديد).	`  concurrencyPolicy: Forbid`
5 احتفظ بآخر 3 Jobs نجحوا وآخر واحد فشل، والباقي يتمسح.	`  successfulJobsHistoryLimit: 3` `  failedJobsHistoryLimit: 1`

6 `jobTemplate` = نفس الـ spec بتاع الـ Job اللي فوق بالظبط.	`  jobTemplate:`
	`  spec:`
	`  template:`
	`  spec:`
	`  restartPolicy: OnFailure`
	`  containers:`
	`  - name: backup`
	`  image: postgres:16`
	`  command: ["sh", "-c", "pg_dump -h db -U app appdb > /backup/db.sql"]`

تشريح الأمر

```
0 2 * * *
1 2 3 4 5
```

1. `0` الدقيقة (0 لـ 59).
2. `2` الساعة (0 لـ 23).
3. `*` يوم الشهر (1 لـ 31). علامة `*` = أي قيمة (كل يوم).
4. `*` الشهر (1 لـ 12).
5. `*` يوم الأسبوع (0 لـ 6، والـ 0 = الأحد).

الـ schedule	معناه
`*/15 * * * *`	كل 15 دقيقة (`/` = كل كام).
`0 * * * *`	كل ساعة على رأس الساعة.
`30 9 * * 0-4`	9:30 الصبح من الأحد للخميس (`-` = من لـ).
`0 0 1 * *`	أول كل شهر الساعة 12 بالليل.

جدول 12.1 أمثلة schedules

`kubectl get cronjobs`	الـ CronJobs، ومعاها آخر مرة اشتغلت (LAST SCHEDULE).
`kubectl create job test-run --from=cronjob/nightly-backup`	شغّل الـ CronJob دلوقتي حالاً للتجربة من غير ما تستنى الميعاد.
`kubectl patch cronjob nightly-backup -p '{"spec":{"suspend":true}}'`	وقّف الـ CronJob مؤقتاً (`patch` بيعدّل حقل واحد، و `-p` فيه التعديل بصيغة JSON).

خلي بالك

الـ CronJob ممكن (نادراً) يشغّل نفس الميعاد مرتين أو يفوّته لو الـ Cluster كان واقع. عشان كده خلي الشغلانة **Idempotent**، يعني لو اتنفذت مرتين النتيجة متبوظش (زي إن الـ Backup يكتب على نفس الملف بدل ما يكرر البيانات).

ملخص الفصل

- الـ Deployment لحاجة تفضل شغالة، والـ Job لحاجة تخلص وتقفل.
- الـ Job: `completions` و `parallelism` و `backoffLimit` ، و `restartPolicy` لازم OnFailure أو Never.
- الـ CronJob بيعمل Jobs على schedule بصيغة cron (دقيقة ساعة يوم شهر يوم-أسبوع).
- `concurrencyPolicy: Forbid` يمنع التداخل، و `ttlSecondsAfterFinished` للتنضيف.

أسئلة مراجعة

1. ليه مينفعش تعمل الـ Backup بـ Deployment؟
2. اكتب schedule لـ CronJob يشتغل كل يوم جمعة الساعة 11 بالليل.
3. Job فشل 3 مرات و `backoffLimit: 3` ، إيه اللي هيحصل؟
4. إزاي تجرّب CronJob دلوقتي من غير ما تستنى ميعاده؟

صح ولا غلط؟

1. ينفع تكتب `restartPolicy: Always` في Job. صح / غلط
2. `* * * * */5` معناها كل 5 دقايق. صح / غلط
3. الـ Pod بتاع Job اللي خلص بيبقى STATUS بتاعه Completed. صح / غلط

13

المرحلة 4 · الشبكات

الفصل التالت عشر

الـ Services: عنوان ثابت للـ Pods

كل Pod بياخد IP جديدة كل ما يتعمل من أول وجديد، فإزاي الـ Frontend يلاقي الـ Database لو عنوانها بيتغير؟ الحل بنفس مبدأ رقم تليفون الشركة: رقم ثابت مش مهم مين الموظف اللي بيرد النهارده. ده بالظبط الـ Service.

بعد الفصل ده هتكون عارف

- المشاكل التلاتة اللي الـ Service بتحلها
- إزاي الـ Service بتلاقي الـ Pods بالـ Labels والـ Endpoints
- الفرق بين port و targetPort و nodePort
- الأنواع الأربعة: ClusterIP و NodePort و LoadBalancer و ExternalName
- الـ DNS جوه الـ Cluster وحل المشاكل الشائعة

13.1 المشكلة: الـ IP بيتغير

- تخيل إن عندك تطبيق فيه Frontend بيتكلم مع Database. الـ Frontend محتاج يعرف عنوان الـ Database عشان يتكلم معاها.
- في Kubernetes، كل Pod بياخد IP جديدة كل ما يتعمل من أول وجديد. يعني لو الـ Pod بتاعة الـ Database اتحذفت واتعملت تاني هتاخد IP جديد، والـ Frontend يفضل بيدوّر على العنوان القديم وبيرفض يشتغل.
- الحل هيكون بنفس مبدأ رقم تليفون الشركة: زي ما الشركة عندها رقم تليفون ثابت، مش مهم مين الموظف اللي بيرد النهارده، الرقم مابيتغيرش.

الـ **Service** هي نفس الفكرة: عنوان ثابت أمام الـ Pods اللي ممكن تتغير في أي وقت. في الحقيقة مش مشكلة واحدة، في تلاتة، والـ Service بتحل التلاتة دول مرة واحدة:

المشكلة	الـ Service بتعمل إيه؟
الـ IP بتاع الـ Pod بيتغير كل ما تتعمل من الأول.	بتديك IP ثابت تقدر من خلاله تكلم مجموعة Pods (بيكونوا مختارين جوه الـ Service).
لو عندك 3 نسخ من نفس الـ Pod، مين يوزع الطلبات عليهم؟	بتوزع الحمل على الـ Pods اللي هي بتخدم عليهم.
لو الـ Pod ماتت، مفيش حاجة تمنع إنه يتبعت طلب لـ IP فاضية (الـ Pod الميتة).	بتعرف لوحدها الـ Pods الشغالة من الميتة.

جدول 13.1 المشاكل التلاتة وحل الـ Service

13.2 إزاي الـ Service بتشتغل فعلاً؟

الـ Service زي أي Object تاني في Kubernetes: بتكتبها في YAML وبتعملها apply. بتاخد IP ثابتة اسمها **ClusterIP**، ودي بتفضل زي ما هي طول عمر الـ Service.

الـ Service مش بتحتفظ بقايمة IPs مكتوبة لمجموعة Pods. لأ، هي بتقول "جيبلي كل الـ Pods اللي عندها label معينة"، زي `app: web` مثلاً.

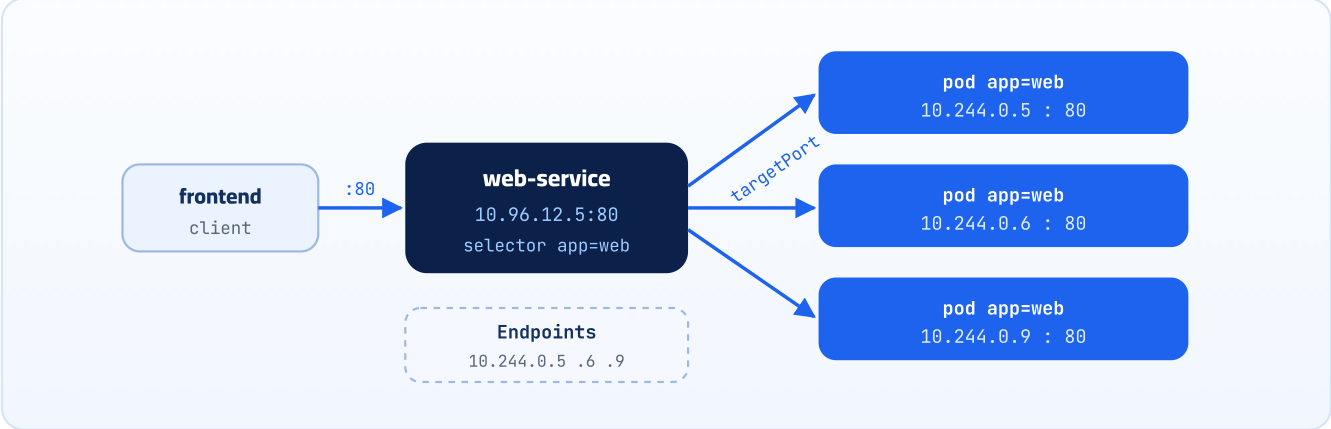
- أي Pods بتتخلق بنفس الـ label، الـ Service بتلاقيها علطول.
- أي Pods تموت، بتشيلها من القايمة (الخدمة يعني).

service.yaml

1 اسم الـ Service، وده نفسه اللي هيبقى اسم الـ DNS بتاعها.	`apiVersion: v1`
	`kind: Service`
	`metadata:`
	`  name: web-service`

2 هيختار مجموعة الـ Pods اللي عليهم الـ Label ده.	`spec:`
	`  selector:`
	`  app: web`

3 `port` : الـ Port اللي الـ Service بتسمع عليه (بتستقبل عليه) أي حاجة جيالها.	`  ports:` `  - port: 80`
4 `targetPort` : الـ Port اللي الـ App نفسه جوه الـ Pod بيسمع عليه (بيستقبل عليه) أي حاجة جياله.	`  targetPort: 80`



شكل 13.1 الـ Service ليها IP ثابت، وبتوزع الطلبات على الـ Pods اللي في الـ Endpoints، واللي بتتحدّث أوتوماتيك بالـ selector.

ورا كل Service فيه Object اسمه **Endpoints** (وفي النسخ الحديثة **EndpointSlices**)، هو ده اللي فيه الـ IPs الـ Pods الشغالة حالياً، و Kubernetes بيعمله update تلقائي. يعني هو وظيفته يروح على الـ Pods اللي تبع الـ Label اللي الـ Service عايزاه ويجيب الـ IPs بتاعة الـ Pods اللي شغالة فيهم (والـ Ready بس، وهنا بتبان فايدة الـ Readiness Probe).

الـ Service بتتولى هي مسؤولية توزيع الـ Traffic (بشكل عشوائي تقريباً) على الـ Pods اللي وراها.

13.3 الأنواع الأربعة لـ Services

عندنا 4 أنواع لـ Services في Kubernetes:

ClusterIP

ده الـ default: يعني لو ماكتبتش `type` خالص في الـ YAML، Kubernetes هيعتبره ClusterIP تلقائي. فكرته إنه بيديك IP داخلية بس، مفيش حاجة من بره الـ Cluster تقدر توصلها. ده اللي بتستخدمه لكل حاجة المفروض تفضل داخلية: الـ database، الـ cache، الـ backend API. أي حاجة مش عايز العالم يشوفها مباشرة. بيستخدم في 90% من الحالات، وباقي الحالات بنسمح بإن حاجة بره الـ Cluster تكلم اللي جواه، ولكن بيكون فيه قيود Security وغيرها.

```yaml
spec:
  type: ClusterIP    # أو سيبها فاضية
  selector:
    app: backend
  ports:
  - port: 8080
    targetPort: 8080
```

NodePort

بيفتح Port على كل Node في الـ Cluster (في الـ Range: **30000–32767**)، وبيوصّل الـ traffic الجاي عليه للـ Service. يعني من بره تقدر توصل للتطبيق عن طريق `Node_IP:30080` مثلاً. والجميل إنك تقدر تستخدم IP أي Node حتى لو الـ Pod مش شغالة عليها، ساعتها Kubernetes بيعمل routing علطول لأي Node تانية.

```yaml
spec:
  type: NodePort
  selector:
    app: web
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30080
```

LoadBalancer

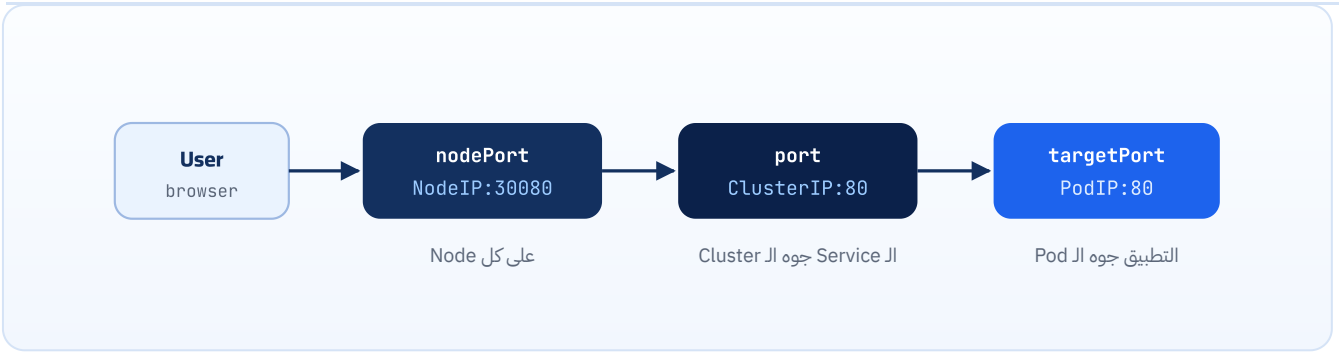
ده اللي بتستخدمه للـ production على الـ Cloud. لما بتعمل Service بـ LoadBalancer، ساعتها Kubernetes بيتكلم مع الـ Cloud ويعملك Load Balancer حقيقي أوتوماتيك، وبتاخد IP أو DNS من بره حقيقية.

```yaml
spec:
  type: LoadBalancer
  selector:
    app: frontend
  ports:
  - port: 80
    targetPort: 3000
```

ExternalName

من اسمه هو بيتعامل مع حاجة خارجية. وفعلاً وظيفته يوصّل حاجة جوه الـ Cluster لحاجة بره، مثلاً فيه Pod عايزة تكلم عنوان معين بره الـ Cluster. بيعمل **DNS alias** لاسم خارج الـ Cluster. التطبيق بيتكلم مع اسم Service عادي، والـ DNS بيوديه للـ endpoint الخارجي.

```yaml
spec:
  type: ExternalName
  externalName: my-db.us-east-
1.rds.amazonaws.com
```



شكل 13.2 رحلة الطلب في NodePort: من الـ nodePort، للـ port بتاع الـ Service، للـ targetPort بتاع الـ Pod.

13.4 ملاحظات على أنواع الـ Services

النوع	الملاحظة
NodePort	النوع ده مش مناسب للـ production، لأنك بتكشف Port غريب على كل ماكينة، والمستخدم محتاج يعرف IP الـ Node والـ Port بتاعها. مش الأسهل تقوله "روح على myapp.com" وخلاص؟ لأ، هتضطر تخليه كل مرة يكتب: "Node_IP:Port"، طب ما ممكن ينسى، وغير كده دي مش طريقة احترافية. كمان الـ IPs الـ Nodes ممكن تتغير، فهو هيعرف منين ساعتها؟ استخدمه للـ dev والاختبار، أو لو عندك load balancer خارجي إنت بتديره.
LoadBalancer	حاجة مهمة جداً، الـ local clusters: لو اشتغلت على minikube أو kind أو أي cluster محلي وكتبت `type: LoadBalancer`، مش هيحصل error، بس الـ EXTERNAL-IP هتفضل `<pending>` للأبد. الحل: شغّل `minikube tunnel` في Terminal تاني.
ExternalName	فايدته إيه؟ بدل ما تكتب الـ IP بتاع المكان (اللي بره الـ Cluster) اللي عايز تكلمه في كل حتة في الكود، بتكتب اسم الـ Service، ولو الـ IP اتغير بعدين، بتغير الـ YAML بس وخلاص، التطبيق مش محتاج يتغير.

جدول 13.2 ملاحظات مهمة على كل نوع

> **نصيحة: الـ port هو الوحيد الإلزامي**
>
> في الـ NodePort Service، الوحيد الـ required هو `port`. لو مكتبتش `targetPort`، هيبقى زي الـ `port`. ولو مكتبتش `nodePort`، Kubernetes بيختار رقم free من الرينج أوتوماتيك.

> **نصيحة: الـ ports هي array**
>
> الـ Service الواحدة تقدر تفتح بورتات كتير في نفس الوقت. بس حط dash (`-`) قبل كل entry، ولازم كل واحد يبقى ليه `name`. مفيد لو التطبيق بيـ serve HTTP على 80 و gRPC على 9090 من نفس الـ Pods.

13.5 إزاي الـ Pods بتلاقي بعضها؟ (DNS)

كل Service بتتعمل ليها **DNS record** أوتوماتيك، من غير أي إعداد منك. Kubernetes بيشغّل **CoreDNS** جوه الـ Cluster، وهو اللي بيحفظ أسماء كل الـ Services. عشان تكلم أي Service بتنده عليها باسمها بالشكل ده:

service-name

تستخدم الاسم علطول لو هي في نفس الـ Namespace اللي إنت فيه.

<service-name>.<namespace>.svc.cluster.local

دي الطريقة الأطول والأعم (في حالة إنت كنت في Namespace تانية غير الـ Namespace اللي الـ Service دي موجودة فيها).

تخيلها زي الـ Contacts اللي عندك في التليفون: مش محتاج تحفظ الأرقام، بتدوّر على الاسم وتلاقي الرقم. في Kubernetes بيعمل نفس الحاجة بالظبط.

13.6 أوامر kubectl السريعة للـ Services

`kubectl get services`	شوف كل الـ Services (أو `svc` اختصاراً).
`kubectl get endpoints`	شوف الـ IPs الـ Pods ورا كل Service.
`kubectl describe service <name>`	تفاصيل كاملة: events + selector.
`kubectl expose deploy <name> --port=80`	اعمل Service من Deployment بسرعة.
`kubectl port-forward svc/<name> 8080:80`	اختبار محلي من غير NodePort.

تشريح الأمر

```
kubectl expose deployment webapp --port=80 --target-port=8080 --type=NodePort
```

1. `expose` — اعمل Service لـ Object موجود (بياخد الـ selector من الـ labels بتاعته أوتوماتيك).
2. `deployment webapp` — الـ Deployment اللي عايز تعمله Service.
3. `--port=80` — الـ `port` بتاع الـ Service.
4. `--target-port=8080` — الـ `targetPort`: البورت اللي التطبيق بيسمع عليه جوه الـ Pod.
5. `--type=NodePort` — نوع الـ Service. لو مكتبتهوش يبقى ClusterIP.

تشريح الأمر

```
kubectl port-forward svc/web-service 8080:80
```

1. `port-forward` — افتح نفق (tunnel) من جهازك لحاجة جوه الـ Cluster. بيفضل شغال لحد ما تدوس `Ctrl+C`.
2. `svc/web-service` — الهدف: Service اسمها web-service (ينفع `pod/name` أو `deploy/name`).
3. `8080` — الـ Port على جهازك إنت (localhost).
4. `:80` — الـ Port بتاع الـ Service. فلما تفتح `http://localhost:8080` هتوصل للـ Service على 80.

جرّبها على minikube

```
Terminal
moaz@Linux:~$ kubectl create deployment web --image=nginx --replicas=2
moaz@Linux:~$ kubectl expose deployment web --port=80 --type=NodePort
moaz@Linux:~$ kubectl get svc web
NAME   TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
web    NodePort   10.96.144.20   <none>        80:31234/TCP   5s
moaz@Linux:~$ minikube service web --url
http://192.168.49.2:31234
```

في عمود `PORT(S)`، الـ `80:31234/TCP` معناها: الـ port بتاع الـ Service هو 80، والـ nodePort اللي اتختار أوتوماتيك هو 31234، والبروتوكول TCP. و `minikube service --url` بيديك الرابط الجاهز.

13.7 مشاكل وأخطاء شائعة

مشكلة 1 **الـ Service مش بتروّح لأي Pod**

عملت الـ Service بس الطلبات بتتضيّع. أول حاجة اعملها:

```
Terminal
moaz@Linux:~$ kubectl get endpoints my-service
NAME         ENDPOINTS   AGE
my-service   <none>      5m
```

`<none>` = فاضية = مفيش Pod اتلاقت. ده في 95% من الحالات بيبقى الـ selector مش مطابق للـ labels. افتح الاتنين وقارن:

```
Terminal
kubectl get pods --show-labels
kubectl describe service my-service | grep Selector
```

خلي بالك: الـ labels حساسة لحالة الحروف. `app: Web` مش زي `app: web`.

تشريح الأمر

```
kubectl describe service my-service | grep Selector
        1                2          3
```

1. `describe service my-service` — طلّع كل تفاصيل الـ Service كنص طويل.
2. `|` — الـ **Pipe**: خد الناتج بتاع الأمر اللي على شمالي وابعته كـ input للأمر اللي على يميني.
3. `grep Selector` — من كل النص ده، اعرض بس السطور اللي فيها كلمة Selector.

مشكلة 2 **الـ targetPort غلط**

الـ Service شغالة بس الطلبات بتفشل. الغلطة الكلاسيكية:

```yaml
ports:
  - port: 80          # ✓ العميل بيتكلم على 80
    targetPort: 80    # ✗ بس التطبيق داخل الـ Pod بيسمع على 8080
```

الحل: دايماً تحقق من البورت الفعلي اللي التطبيق بيسمع عليه جوه الـ Pod، وطابقه مع الـ `targetPort`.

ملخص الفصل

- الـ Service = IP واسم ثابتين قدام مجموعة Pods متغيرة، بتختارهم بالـ selector.
- الـ Endpoints فيها IPs الـ Pods الجاهزة، وبتتحدث أوتوماتيك.
- `port` بتاع الـ Service، `targetPort` بتاع التطبيق، `nodePort` على كل Node (30000–32767).
- ClusterIP داخلي (الافتراضي)، NodePort للتجربة، LoadBalancer للـ Cloud، ExternalName لـ alias خارجي.
- الـ DNS: `name` في نفس الـ Namespace، أو `name.ns.svc.cluster.local`.

أسئلة مراجعة

1. ليه منكتبش IP الـ Pod في إعدادات الـ Frontend على طول؟
2. الـ Endpoints بتاعة Service طلعت `<none>`، إيه أول حاجة تشك فيها؟
3. التطبيق بيسمع على 3000 وعايز الناس توصله على 80 جوه الـ Cluster، اكتب الـ ports.
4. عملت LoadBalancer على minikube والـ EXTERNAL-IP `<pending>`، ليه؟ وإيه الحل؟

صح ولا غلط؟

1. لو مكتبتش `type`، الـ Service هتبقى NodePort. صح / غلط
2. الـ nodePort ممكن يبقى أي رقم زي 8080. صح / غلط
3. Pod مش Ready مش بيتحط في الـ Endpoints. صح / غلط

14

المرحلة 4 · الشبكات

الفصل الرابع عشر

الـ Ingress: باب واحد لكل الـ Traffic

تخيل إنك عملت تطبيقك بـ LoadBalancer Service زي ما اتعلمنا. تمام، شغال. بعدين الفريق ضاف admin dashboard، وبعدين API منفصل. كل جزء محتاج Load Balancer لوحده وفاتورة لوحده. الـ Ingress بيحل المشكلة دي بباب رئيسي واحد بيوزّع على الكل.

بعد الفصل ده هتكون عارف

- المشكلة اللي الـ Ingress بيحلها (التكلفة)
- الفرق بين Ingress Resource و Ingress Controller
- تركيب nginx-ingress على minikube وبـ Helm
- يعني إيه Routing: الـ Host والـ Path
- كتابة Ingress بالـ Path وبالـ Host

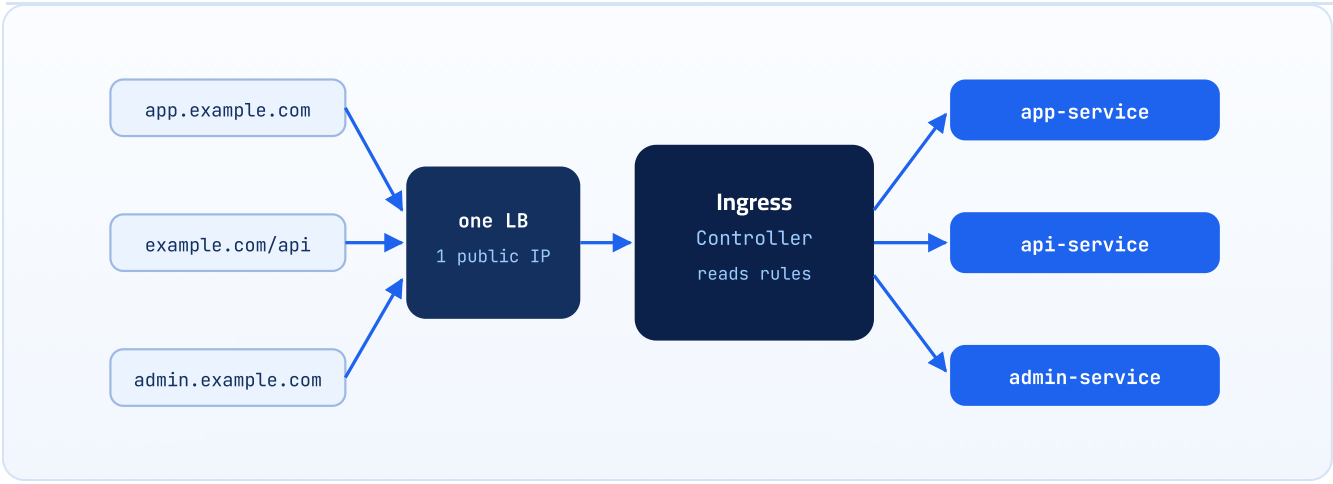
14.1 المشكلة: Load Balancer لكل حاجة

- تخيل عملت تطبيقك بـ LoadBalancer Service زي ما اتعلمنا. تمام، شغال. بعدين الفريق ضاف admin dashboard. وبعدين API منفصل.
- كل جزء في الـ App محتاج LB خاص بيه، يعني كل واحد بياخد cloud load balancer لوحده، و IP خارجي لوحده، وسطر منفصل في فاتورة الـ cloud.
- طب لما تعرف إن الـ Load Balancer الواحد بيكلف تقريباً من 20 لـ 50 دولار شهرياً. يعني لو عندك 5 أجزاء في الـ App بتاعك، هتدفع 100-250 دولار شهرياً بس في الـ load balancers، قبل ما تحسب تكلفة باقي الحاجات أصلاً.

الـ **Ingress** بيحل المشكلة دي بطريقة ذكية: بيعمل **Load Balancer واحد بس** على الـ Cluster من بره، وكل الـ Traffic بيعدي منه. الـ Ingress بعد كده بيوزع كل request للمكان الصح بناءً على الـ **hostname** أو الـ **path**، وفاهم HTTP/HTTPS بالكامل، وفيه **TLS termination** مدمج.

تخيله كده

زي مكتب الاستقبال في مبنى فيه شركات كتير. مش كل شركة محتاجة باب خاص بيها من الشارع: فيه باب رئيسي واحد، والريسيبشن (الـ Ingress) بيوجه كل زائر للدور والمكتب الصح حسب اسمه أو طلبه.



شكل 14.1 Load Balancer واحد، والـ Ingress Controller بيوزّع على الـ Services حسب الـ Host أو الـ Path.

14.2 الـ Ingress عنده جزئين

Ingress Resource

الـ YAML اللي إنت بتكتبه، وبيبقى فيه القواعد اللي هيتعمل بيها routing للـ Traffic.

Ingress Controller

البرنامج الفعلي اللي هيقرا القواعد دي ويبتدي ينفذها ويوزع الـ Traffic. هي مجرد أداة هتسطّبها بس، وتخليها تقرا ملف الـ YAML، وهي هتتعامل مع نفسها، مش محتاج تعمل حاجة تانية في الحتة دي.

> **لاحظ**
>
> Kubernetes مش بييجي معاه Ingress Controller افتراضي. لازم تركّبه إنت بنفسك. لو عملت Ingress Resource من غير ما تركّب Controller، هيتعمل الـ Object آه، بس مش هيحصل أي routing فعلي.

يعني الـ flow هو: إنت بتكتب القواعد في YAML، والـ Controller أياً كان نوعه (nginx، traefik، أو غيره) بيراقب التغييرات دي ويظبط نفسه أوتوماتيك، وبعد كده هو اللي بيوجه الـ Traffic فعلياً للـ Services الصح.

14.3 تركيب الـ Ingress Controller

الأشهر والأكتر استخداماً هو **ingress-nginx**. خلينا نركّبه.

لو شغال على Minikube (للتجربة محلياً)

Terminal

```
moaz@Linux:~$ minikube addons enable ingress
# للتأكد إنه شغال:
moaz@Linux:~$ kubectl get pods --namespace=ingress-nginx
NAME                                        READY   STATUS    AGE
ingress-nginx-controller-5c8d66c76d-v7jkp   1/1     Running   30s
```

تشريح الأمر

```
minikube addons enable ingress
```

1. `addons` — إضافات جاهزة بتيجي مع minikube (ingress، metrics-server، dashboard ...). تشوفها كلها بـ `minikube addons list`.
2. `enable` — فعّل الإضافة (و `disable` تقفلها).
3. `ingress` — اسم الإضافة: بتركّب ingress-nginx controller.

لو على Cluster حقيقي (Helm)

Terminal

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
helm install ingress-nginx ingress-nginx/ingress-nginx \
  --namespace ingress-nginx \
  --create-namespace \
  --set controller.replicaCount=2
# هات الـ IP/hostname الخارجي
kubectl get service ingress-nginx-controller --namespace=ingress-nginx
# NAME                       TYPE           EXTERNAL-IP
# ingress-nginx-controller   LoadBalancer   a1b2c3.elb.amazonaws.com
```

تشريح الأمر

```
helm install ingress-nginx ingress-nginx/ingress-nginx --namespace ingress-nginx --
create-namespace --set controller.replicaCount=2
```

1. `helm install` — سطّب Chart (حزمة جاهزة من ملفات YAML). هنشرح Helm بالتفصيل في آخر فصل.
2. `ingress-nginx` — اسم الـ Release (النسخة المتسطبة عندك، اسمها من اختيارك).
3. `ingress-nginx/ingress-nginx` — اسم الـ Chart: `repo/chart`، يعني Chart اسمها ingress-nginx من الـ repo اللي ضفناه بنفس الاسم.
4. `--namespace ingress-nginx` — سطّبه في الـ Namespace دي.
5. `--create-namespace` — اعمل الـ Namespace لو مش موجودة.
6. `--set controller.replicaCount=2` — غيّر قيمة إعداد في الـ Chart: شغّل نسختين من الـ Controller.

معلومة مهمة

علامة `\` في آخر السطر معناها "الأمر لسه مكمّل في السطر اللي تحت"، عشان الأوامر الطويلة تبقى مقروءة. ولو بتنسخ الأوامر خلي بالك إن `--namespace` بشرطتين عاديتين `--`، مش شرطة طويلة `-` (بتحصل كتير لما تنسخ من Word).

خلي بالك: ingress-nginx اتقفل

مشروع ingress-nginx (التابع لـ Kubernetes) اتعمله **Retirement** رسمي في 24 مارس 2026: مفيش أي releases أو security patches جديدة بعد التاريخ ده. اللي متسطّب لسه شغال، والـ Helm charts والـ images لسه موجودة، وعشان كده لسه بنستخدمه هنا للتعلم لأنه أسهل حاجة على minikube. لكن لمشروع حقيقي جديد، الـ Kubernetes نفسها بتنصح بالـ **Gateway API** (هتلاقي نبذة عنه في آخر الفصل) أو Controller تاني متصان زي Traefik أو HAProxy. الأفكار اللي في الفصل ده (Host و Path و TLS) هي هي في كل الحلول دي.

قبل ما نكمّل في أي تفاصيل عملية، زي إننا هنكتب ملف الـ YAML بتاع الـ Ingress Resource وإزاي نخلي الـ Controller يتصرف على أساسه، عايزين نفهم Concept مهم جداً ألا وهو الـ **Routing**.

14.4 الـ Routing: الطلب ده يروح فين؟

الـ Routing معناها بكل بساطة: **"الطلب ده يروح فين؟"**. الـ Ingress Controller بيستقبل آلاف الطلبات من كل حتة في الدنيا، وكل طلب لازم يوصل للـ Service الصح. إزاي بيقرر؟ بيبص على حاجتين بس في الطلب اللي جاله:

1. **الـ Host**: يعني إيه Domain اللي المستخدم كتبه؟ `app.example.com` ولا `api.example.com`؟
2. **الـ Path**: يعني إيه الـ Path بعد الدومين؟ `/app` ولا `/api`؟

خلينا نقسّم أي رابط (URL) لأجزائه عشان تفهم بالظبط الـ Ingress بيبص على إيه:

تشريح الأمر

```
1       2              3
https://app.example.com/api/users
```

1	`https://`	البروتوكول (Scheme). الـ s معناها إن الاتصال متشفّر بـ TLS.
2	`app.example.com`	ده الـ **Domain (Host)**، اللي هو اسم الموقع، ولو فيه أي إضافة عليه زي app أو api (اسمها subdomain).
3	`/api/users`	ده الـ **Path**: مسار جوه الـ Domain بيبقى صفحة معينة جوه الموقع. يعني مثلاً `example.com/login` معناه إنك عايز صفحة تسجيل الدخول اللي جوه الموقع ده.

من الآخر نقدر نستخدم الـ Domain أو الـ Path لنفس الهدف (استهداف أجزاء معينة من الموقع):

بالـ Domain

- `www.app.example.com`
- `www.api.example.com`

بالـ Path

- `www.example.com/app`
- `www.example.com/api`

ولكن ممكن نستغرب أول طريقة دي، إنه ممكن أعمل path جوه الـ 2 Domains المختلفة: `app.example.com/path` أو `api.example.com/path`. فكده إيه لازمة الطريقة الأولى طالما أقدر أستخدم الـ Path في الحالتين؟

دي ملاحظة كويسة، وفعلاً ممكن يبان إنه مفيش فرق وفكرة التفريق بالـ Domain ملهاش لازمة، ولكن في الحقيقة الـ Domain هنا بيستخدم لغرض أشمل وأعم... إزاي؟

خلينا نتخيل إن الموقع بتاعنا فيه 50 صفحة مختلفين، والصفح دي متقسمة، بمعنى إنها مش بتظهر لكل مستخدم: فمثلاً فيه صفح للمستخدم العادي، وفيه صفح للـ Admin، وهكذا. وكل مجموعة صفح من دول مختلفة عن التانية.

طريقة الـ Path

`www.example.com/path1`
`www.example.com/path2`
`www.example.com/path3`
`www.example.com/path1/path2`
...

هي صح وتمام، ولكن شايف كم التعقيد؟ والدنيا داخلة في بعضها إزاي (ده في حالة إنه فيه مسارات كتير جداً، لكن لو حاجات معقولة فعادي)، خصوصاً مع كتر الصفحات وبالتالي طول الكتابة لكل link هتكتبه، فممكن تغلط وإنت بتكتب.

طريقة الـ Domain

`www.app.example.com/path1`
`www.app.example.com/path2`
`www.admin.example.com/path1`
`www.admin.example.com/path2`
...

هنا إنت عملت تصنيف من الأول للخدمات أو الصفح: `app` بيستهدف كل الصفح/الخدمات المتاحة للـ user العادي، و `admin` بيستهدف كل الصفح/الخدمات المتاحة للـ admin. ولكل Domain منهم تقدر تكتب path الخدمة اللي عايزها، فتعتبر طريقة الـ Domain هي mix بين الطريقتين.

فالموضوع هنا بقى أسهل: كتابة أقل، أخطاء أقل... ولكن هل ده معناه إننا نستخدم طريقة واحدة بس دوناً عن التانية؟ الإجابة لأ، الطريقتين صح وتمام، والاختيار بيتحدد بناءً على شكل الموقع بتاعك إنت، وأنهي طريقة تخدم الموقع بتاعك أفضل.

14.5 (1) على حسب الـ Path (Path-based)

نرجع بقى للـ Ingress: الـ Ingress بيقدر يوزع الطلبات بأي طريقة من الاتنين.. إنت اللي بتحدد تخليه يستخدم أنهي. هنا إحنا هنعتمد على Paths مختلفة تحت نفس الـ Domain:

ingress.yaml · Path-based

1 حالياً ممكن نحفظ دي كـ Format ثابت عشان نعمل Ingress، والحاجة الوحيدة اللي بنغيرها هي الاسم `name: my-app-ingress`، إنما باقي الحاجات هنثبتها.

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-app-ingress
```

2 إعداد خاص بـ nginx: بيشيل الـ `/app` من الطلب قبل ما يبعته للـ Service، فالتطبيق يستلمه كأنه `/`.

```yaml
  annotations:

nginx.ingress.kubernetes.io/rewrite-
target: /
```

3 الـ `ingressClassName` بتكتب فيها اسم الـ Ingress Controller اللي إنت عايزه ينفذ الـ YAML ده أصلاً. ولاحظ لازم يكون متسطّب من قبلها على الـ Cluster، وإلا مفيش Traffic هيتوزع على الـ Services، يعني الـ Ingress هيتعمل بس مش هيوزع حاجة.	`spec:` `  ingressClassName: nginx`
4 الـ `rules` بتاخد list من الـ hosts (عشان لو عندك أكتر من Domain)، ولكن في حالتنا هنعتمد على Paths مختلفة تحت نفس الـ Domain. والـ `host` بياخد الـ Domain بتاعك (لينك الموقع).	`  rules:` `  - host: www.example.com`
5 بنستخدم جوه الـ `http` tag تانية اسمها `paths`، ودي بتستقبل list من الـ paths.	`    http:` `      paths:`
6 الـ `path` بتحدد فيها المسار نفسه، بمعنى إنك بتقوله: لو لقيت بعد `www.example.com` الـ `/app`، هتعمل إيه أو هتروح فين.	`      - path: /app`
7 طريقة الـ Matching: لو عايزه ينفذ لما يلاقي `/app` بس بالظبط، تكتب `Exact`. ولو لقى أي path أوله `/app` وخلاص، أياً كان فيه حاجة بعدها ولا لأ (زي `/app/settings`)، تكتب `Prefix`.	`        pathType: Prefix`
8 جوه الـ `backend` فيه `service`، بتكتب جواها بيانات الـ Service اللي هتكلمها. مكتوبة هنا بصيغة مختصرة (Flow style) زي الـ JSON.	`        backend:` `          service: { name: app-service, port: { number: 80 } }`

الصيغة المختصرة `{ name: app-service, port: { number: 80 } }` هي هي بالظبط الشكل ده وهو مفرود:

```yaml
backend:
  service:
    name: app-service
    port:
      number: 80
```

اختصاراً في الكتابة بس مكتبناش path تاني، لكن عادي تكتب زيادة، كله ينطبق عليه نفس الكلام (`- path: /api` لـ `api-service` مثلاً).

14.6 (2) على حسب الـ Domain (Host-based)

هنا إحنا هنعتمد على Domains مختلفة بغض النظر عن لو هنستهدف paths جواها ولا لأ. نفس الـ Syntax الـ YAML بتاع النوع الأول، الفرق إننا هنستخدم كذا `host` جوه الـ `rules`، عشان زي ما قلنا بتاخد list من الـ hosts:

ingress.yaml · Host-based

```yaml
spec:
  ingressClassName: nginx
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend: { service: { name: app-service, port: { number: 80 } } }
  - host: api.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend: { service: { name: api-service, port: { number: 8080 } } }
```

هنا إحنا عملنا 2 Domains مختلفين (`app.example.com` و `api.example.com`) بنوزع عليهم الـ Traffic أياً كان الـ Path اللي بعدهم، عشان كتبنا `/` بس في الـ path، وكمان اخترنا `Prefix`، فكده إحنا بنقول أياً كان مكتوب إيه بعد الـ Domain، حتى لو مفيش حاجة.

التشغيل والتأكد

بمجرد ما تحفظ ملف الـ YAML بتاعك (وليكن هنسميه `ingress.yaml`) هتكتب الأمر ده عشان تشغّل الـ Ingress اللي كتبته الـ YAML، ولو الملف سليم ومفيش مشاكل هيظهر كده:

Terminal

```
moaz@Linux:~$ kubectl apply -f ingress.yaml
ingress.networking.k8s.io/my-app-ingress created
moaz@Linux:~$ kubectl get ingress
NAME             CLASS   HOSTS             ADDRESS        PORTS   AGE
my-app-ingress   nginx   www.example.com   192.168.49.2   80      40s
```

مهم تتأكد إن الـ Ingress Controller (اللي هو Nginx في حالتنا) شاف الملف واستلمه صح. ولاحظ إن أول ما بتشغّل الأمر، ممكن تلاقي خانة الـ `ADDRESS` فاضية. استنى ثواني وجرّب الأمر تاني لحد ما يظهر فيها عنوان الـ IP الفعلي. ظهور الـ IP معناه إن الـ Nginx استلم الـ Ingress وربطه بنجاح.

جرّب من غير DNS

معندكش Domain حقيقي؟ ابعت الطلب على الـ IP وقوله إنت عايز أنهي Host:

```
Terminal
curl -H "Host: www.example.com" http://192.168.49.2/app
```

`-H` بتضيف Header للطلب، والـ Ingress بيقرا الـ Host من الـ Header ده بالظبط.

14.7 مقارنة الـ Controllers وأوامر سريعة

Controller	الأنسب لـ	ملاحظات
ingress-nginx	عام / general	الأكتر استخداماً، annotations كتير
Traefik	cloud-native	بيكتشف الـ services لوحده، فيه dashboard
HAProxy	أداء عالي	مميزات enterprise
AWS ALB Controller	AWS EKS بس	متكامل مع AWS ALB
Istio Gateway	service mesh	إدارة ترافيك متقدمة

جدول 14.1 مقارنة الـ Ingress Controllers

`kubectl get ingress`	يسرد كل الـ Ingress resources (أو `ing` اختصاراً).
`kubectl describe ingress <name>`	قواعد الـ routing والـ events.
`kubectl get ingressclass`	الـ Controllers المتاحة (الأسامي اللي تكتبها في `ingressClassName`).
`kubectl logs -n ingress-nginx deploy/ingress-nginx-controller --tail=50`	لوجز الـ Controller: آخر 50 سطر.

14.8 مشاكل شائعة

مشكلة 1 مفيش Ingress Controller متركّب

عملت Ingress بس عمود ADDRESS فاضي للأبد:

```
Terminal
moaz@Linux:~$ kubectl get ingress
NAME    CLASS   HOSTS             ADDRESS   PORTS   AGE
myapp   nginx   app.example.com             80      10m
```

الحل: ركّب Controller (ingress-nginx، traefik، أو غيره). الـ Ingress Resource مايعملش حاجة من غيره.

مشكلة 2 الـ DNS مش موجّه للـ LoadBalancer

`curl https://app.example.com` يرجع `Could not resolve host`.

```
Terminal
kubectl get service ingress-nginx-controller -n ingress-nginx
# انسخ EXTERNAL-IP واعمل A record أو CNAME عليه
```

الحل: عند الشركة اللي اشتريت منها الـ Domain، اعمل **A record** (لو IP) أو **CNAME** (لو hostname) بيشاور على الـ EXTERNAL-IP.

مشكلة 3 404 من nginx بس الـ Service شغالة

الـ `port-forward` للـ Service بيشتغل عادي، بس الـ Ingress بيرجع 404. الأسباب المحتملة:

- `ingressClassName` مش مطابق للـ Controller المتركّب.
- الـ path أو الـ pathType غلط.
- اسم الـ Service أو البورت غلط.

```
Terminal
kubectl describe ingress my-ingress
# شوف قسم Rules و Events
kubectl logs -n ingress-nginx deployment/ingress-nginx-controller --tail=50
# شوف اللوجز عشان تعرف القرارات اللي بياخدها
```

14.9 الـ Gateway API: البديل الحديث للـ Ingress إضافة

الـ Gateway API هي النسخة الأحدث والأقوى من فكرة الـ Ingress، ومعمولة بحيث كل فريق يمسك الجزء بتاعه: فريق الـ Infrastructure يعمل الـ **Gateway** (الباب نفسه)، وكل فريق تطبيق يعمل الـ **HTTPRoute** بتاعه (قواعد التوجيه). والـ Controller هنا برضه لازم يتسطّب (زي Envoy Gateway أو Istio أو Traefik أو NGINX Gateway Fabric).

gateway.yaml

1 الباب نفسه. الـ `gatewayClassName` زي الـ `ingressClassName`: بيحدد مين الـ Controller.

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: main-gateway
spec:
  gatewayClassName: envoy
```

2 الـ Gateway بيسمع على أنهي بروتوكول وبورت.

```yaml
  listeners:
  - name: http
    protocol: HTTP
    port: 80
```

httproute.yaml

1 الـ Route دي متعلّقة على أنهي Gateway.

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: app-route
spec:
  parentRefs:
  - name: main-gateway
```

2 نفس فكرة الـ Ingress بالظبط: Host و Path والـ Service اللي هيروحلها.

```yaml
  hostnames: ["www.example.com"]
  rules:
  - matches:
    - path: { type: PathPrefix, value: /app }
    backendRefs:
    - name: app-service
      port: 80
```

ملخص الفصل

- الـ Ingress = Load Balancer واحد بيوزع على Services كتير، بدل LB لكل Service.
- الـ Ingress Resource (القواعد) محتاج Ingress Controller (المنفّذ) متركّب.
- الـ Routing بالـ Host (الدومين) أو بالـ Path (المسار)، أو الاتنين مع بعض.
- `pathType` : Exact للمسار بالظبط، Prefix لأي مسار يبدأ بيه.
- ADDRESS فاضي = Controller مش شغال أو لسه بيستلم.

أسئلة مراجعة

1. ليه الـ Ingress بيوفّر فلوس مقارنة بـ LoadBalancer لكل Service؟
2. إيه الفرق بين الـ Ingress Resource والـ Ingress Controller؟
3. عايز `/docs` و `/docs/intro` يروحوا لنفس الـ Service، تستخدم أنهي pathType؟
4. اكتب rule تبعت `shop.example.com` لـ `shop-service` على 80.

صح ولا غلط؟

1. Kubernetes بييجي معاه Ingress Controller جاهز. صح / غلط
2. الـ Ingress بيقرر على حسب الـ Host والـ Path. صح / غلط
3. `pathType: Exact` مع `/app` هيطابق `/app/users` . صح / غلط

15

المرحلة 4 · الشبكات

الفصل الخامس عشر

الـ TLS و HTTPS :cert-manager ببلاش

من ضمن الحاجات اللي الـ Ingress بيقدمها هي الـ TLS Termination. فخلينا الأول نفهم إيه هو الـ TLS بشكل مفصّل، وبعدين نشوف إزاي نجيب شهادات مجانية من Let's Encrypt ونجددها أوتوماتيك بـ cert-manager.

بعد الفصل ده هتكون عارف

- ليه الاتصال لازم يتشفّر
- الأهداف التلاتة للـ TLS
- التشفير غير المتماثل (الصندوق والقفل)
- خطوات الـ HTTPS ودور الـ Certificate والـ CA
- يعني إيه TLS Termination في الـ Ingress
- تركيب cert-manager وربطه بالـ Ingress

15.1 ليه محتاجين TLS؟

مبدئياً، أي مستخدم بيكتب اسم موقع في أي متصفح زي Google مثلاً، فهو بيبعت طلب إنه يكلم الموقع ده، أو بمعنى أدق عايز يكلم السيرفر اللي عليه الموقع ويشوف الموقع نفسه. وفي الحالة دي السيرفر بيرد عليه ببيانات، اللي بدورها بتظهرله الموقع ويبتدي يتعامل معاه.. العملية أو الاتصال ده بقى لازم يكون متشفّر أو متأمّن، عشان لو بطريقة ما حد اتدخل في النص، ممكن يشوف التواصل ده ويسرق بيانات.

يعني تخيل مثلاً حد فاتح موقع بنك وبيحط بياناته: لو الاتصال بينه وبين موقع البنك مش متشفّر، ساعتها المخترق هيشوف الحساب والبيانات الحساسة كلها ويسرق المستخدم ده.

الـ **TLS (Transport Layer Security)** هو بروتوكول بيعمل "قناة مشفرة" بين طرفين (متصفح المستخدم والسيرفر بتاعك) عشان البيانات تمشي فيه بأمان.

15.2 الـ TLS بيحقق 3 أهداف رئيسية

التشفير (Encryption)

البيانات بتتحول لطلاسم، محدش يقدر يفهمها لو اعترضها في السكة (زي الباسوردات أو بيانات الفيزا).

التوثيق (Authentication)

بيتأكد إن السيرفر اللي إنت بتكلمه هو فعلاً السيرفر الحقيقي (عن طريق حاجة اسمها Certificate)، عشان محدش ينتحل شخصية الموقع.

سلامة البيانات (Integrity)

بيتأكد إن البيانات متمّش التلاعب بيها أو تعديلها أثناء نقلها.

15.3 الفكرة التأسيسية: التشفير غير المتماثل

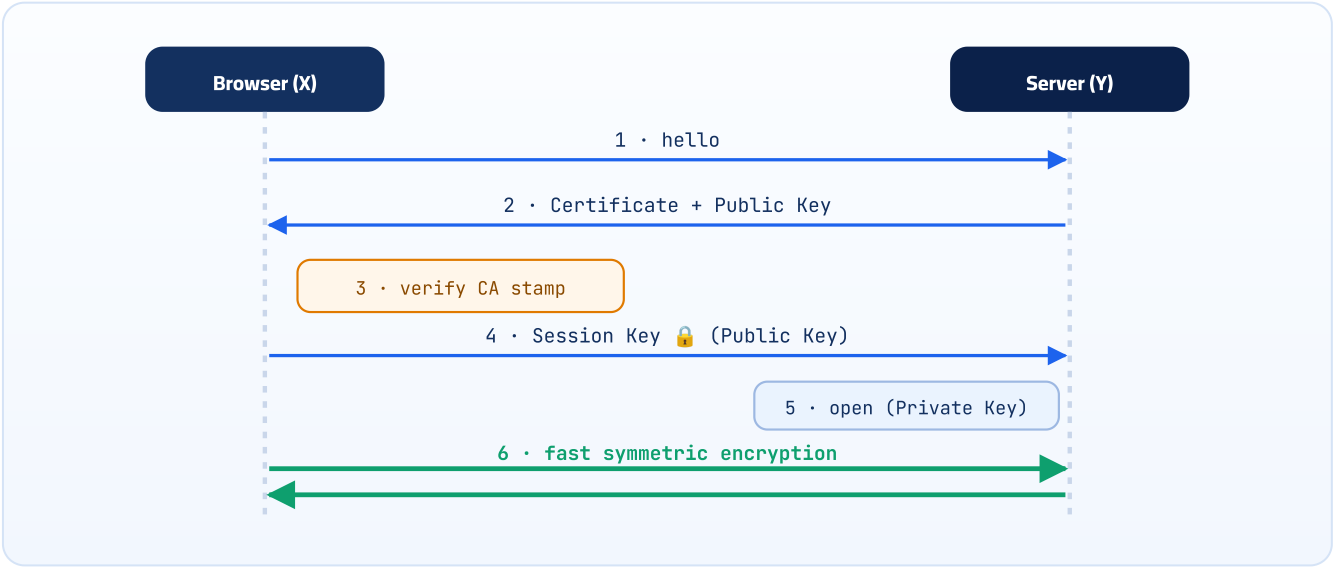
طب إزاي بيشتغل؟.. في الأول محتاجين نعرف أبسط شكل لعمليات التشفير: الـ **Asymmetric Encryption**. في البداية، فكّروا في طريقة **الصندوق والقفل**:

1. **التجهيز:** الطرف المستقبِل، وليكن اسمه Y اللي هو السيرفر، بيجيب صندوق مفتوح وقفل، وبيخلي "المفتاح المعدني" (الخاص) معاه هو بس.
2. **التبادل:** Y بيبعت "القفل المفتوح" (**Public Key**) لأي حد عايز يبعتله بيانات.
3. **الإرسال:** وليكن الطرف اللي هيبعت اسمه X، اللي هو المتصفح اللي استخدمه الـ user عشان يوصل للموقع (السيرفر)، بيحط البيانات في الصندوق، بيقفله بـ "القفل المفتوح"، وبيبعته لـ Y.
4. **الاستقبال:** الصندوق بيوصل لـ Y مقفول، وبما إنه الوحيد اللي معاه المفتاح المعدني (**Private Key**)، بيقدر يفتحه ويقرأ البيانات.

> ⚠ **المشكلة هنا**
>
> الطريقة دي (اللي بتعتمد على قفل وفتح الصناديق في كل مرة) بتستهلك "مجهود حسابي" ضخم جداً من المعالج (CPU). لو استخدمناها في كل حرف بتبعته، السيرفر بتاعك هيبطّأ جداً وهيفشل في خدمة آلاف المستخدمين في نفس الوقت.

15.4 التطور الحديث: نظام الـ HTTPS والـ TLS

عشان نحل مشكلة البطء، المهندسين عملوا "خلطة سحرية" بتدمج بين الأمان العالي والسرعة القصوى. إزاي؟ عن طريق فصل مرحلة "التعارف" عن مرحلة "نقل البيانات". العملية بتتم في أجزاء من الثانية:



شكل 15.1 المراحل: التعارف، التحقق، تبادل السر بالصندوق مرة واحدة، وبعدين تشفير سريع بمفتاح مشترك.

1. **التعارف:** المتصفح بيكلم السيرفر، والسيرفر بيبعتله **Certificate** (بطاقة هويته) المختومة من جهة موثوقة.
2. **التحقق:** المتصفح بيتأكد إن الشهادة دي أصلية ومش مزوّرة، عن طريق إنه بيدوّر على ختم الجهة الموثوقة دي فيها.

3. **تبادل السر (استخدام الصندوق):** هنا بنستخدم فكرة "الصندوق" لمرة واحدة فقط! المتصفح بيولّد مفتاح سري مؤقت (**Session Key**)، بيحطه في صندوق ويقفله بالـ Public Key بتاع السيرفر، ويبعته.
4. **فتح السر:** السيرفر بيفتح الصندوق بالـ Private Key الخاص بيه، وبيطلّع المفتاح السري.
5. **الاتصال السريع (الواقع العملي):** دلوقتي الطرفين معاهم نفس "المفتاح السري". الـ HTTPS هنا بيبطّل يستخدم الصناديق والقفل (لأنه بطيء)، وبيستخدم المفتاح السري ده لتشفير/فك تشفير كل البيانات المتبادلة بشكل متماثل (**Symmetric Encryption**)، وده أسرع بمراحل!

> **دقة: TLS 1.3**
>
> الطريقة اللي فوق هي الفكرة الأساسية وكانت بتحصل كده بالظبط في النسخ القديمة (RSA key exchange). في **TLS 1.3** (المستخدم النهارده) المتصفح والسيرفر بيتفقوا على الـ Session Key بطريقة رياضية اسمها **ECDHE** من غير ما المفتاح نفسه يتبعت أصلاً في الصندوق، والـ Private Key بتاع السيرفر بيستخدم إنه "يمضي" ويثبت إنه هو صاحب الشهادة. النتيجة هي هي: تعارف تقيل مرة واحدة، وبعدين تشفير متماثل سريع.

مهم جداً نفهم حتة الـ Certificate دي

- الـ Certificate دي بطاقة هوية (Identity Document) رقمية.
- هي بتثبت إن الـ Public Key ده بتاع الموقع الفلاني فعلاً، مش بتاع واحد نصّاب.
- اللي بيضمن ده هو الطرف الثالث الموثوق (**CA**: Certificate Authority) اللي ختم الـ Certificate بختمه الرقمي.
- المتصفح بيثق في السيرفر لأنه واثق في الـ CA اللي ختم الـ Certificate نفسها.

15.5 يعني إيه TLS Termination في الـ Ingress؟

بعد ما فهمنا القصة الكبيرة دي بقى.. الـ Ingress يقدر يعمل **TLS termination**، يعني الـ HTTPS بيتفك تشفيره عند الـ Ingress Controller، وبعد كده الـ Traffic الداخلي للـ Services بيبقى HTTP عادي (من غير تشفير، لأنه جوه الـ Cluster أصلاً).



شكل 15.2 التشفير من المستخدم لحد الـ Ingress، وجوه الـ Cluster HTTP عادي.

15.6 إزاي نعمل الموضوع ده بـ cert-manager

في الـ production، بنستخدم حاجة اسمها **cert-manager** عشان يجيب شهادات مجانية من **Let's Encrypt** ويجدّدها لوحده، مش هتلمس الشهادات بإيدك تاني أبداً.

(1) تسطيب الـ cert-manager

أسهل طريقة هي استخدام Helm. نفّذ الأوامر دي في الـ Terminal:

Terminal

```
# إضافة الـ Repository
helm repo add jetstack https://charts.jetstack.io
helm repo update
# تنصيب الـ cert-manager
helm install cert-manager jetstack/cert-manager \
  --namespace cert-manager \
  --create-namespace \
  --set crds.enabled=true
```

تشريح الأمر

```
helm repo add jetstack https://charts.jetstack.io
           1    2       3
```

1. `repo add` — ضيف مستودع Charts جديد لـ Helm.
2. `jetstack` — اسم مختصر إنت بتختاره للمستودع، وبتستخدمه بعد كده في `jetstack/cert-manager`.
3. `https://charts.jetstack.io` — عنوان المستودع. وبعدها `helm repo update` بينزّل آخر قايمة Charts منه.

معلومة مهمة

الـ `--set crds.enabled=true` بيسطّب الـ **CRDs** (Custom Resource Definitions): دي أنواع Objects جديدة بيضيفها cert-manager على Kubernetes، زي `ClusterIssuer` و `Certificate`، عشان تقدر تكتبهم في YAML زي الـ Pod والـ Service بالظبط.

(2) تعريف "جهة الإصدار" (Issuer)

الـ cert-manager محتاج يعرف مين اللي هيصدر الشهادات. هننشئ ملف اسمه `issuer.yaml` عشان يربطنا بـ Let's Encrypt:

issuer.yaml

1 `ClusterIssuer`: جهة إصدار متاحة للـ Cluster كله (و `Issuer` لـ Namespace واحدة).	`apiVersion: cert-manager.io/v1`
	`kind: ClusterIssuer`
	`metadata:`
	`  name: letsencrypt-prod`

2 الـ ACME هو البروتوكول اللي Let's Encrypt بيستخدمه، وده عنوان السيرفر الحقيقي (production).	`spec:`
	`  acme:`
	`  server: https://acme-v02.api.letsencrypt.org/directory`

3 اكتب إيميلك هنا عشان يبعتولك تنبيه لو الشهادة قربت تنتهي.	`  email: your-email@example.com`
4 اسم الـ Secret اللي هيتحفظ فيه المفتاح الخاص بحسابك عند Let's Encrypt.	`  privateKeySecretRef:` `  name: letsencrypt-prod-key`

5 طريقة إثبات ملكية الدومين: HTTP-01 Challenge عن طريق الـ Ingress بتاع nginx.	`solvers:`
	`- http01:`
	`ingress:`
	`class: nginx`

بعد ما تخلص كتابة الملف احفظه واعمل: `kubectl apply -f issuer.yaml`

(3) تعديل الـ Ingress بتاعك (الربط السحري)

دلوقتي هنروح لملف الـ Ingress اللي عملناه قبل كده، ونضيف عليه "Annotation" عشان الـ cert-manager يفهم إنه هو اللي مسؤول عن الشهادة دي:

ingress.yaml · TLS

1 السطر ده هو اللي بيكلم الـ cert-manager: "اعمل شهادة للـ Ingress ده من الـ ClusterIssuer ده".	`apiVersion: networking.k8s.io/v1`
	`kind: Ingress`
	`metadata:`
	`name: my-app-ingress`
	`annotations:`
	`cert-manager.io/cluster-issuer: "letsencrypt-prod"`

2 زي ما هي من الفصل اللي فات.	`nginx.ingress.kubernetes.io/rewrite-target: /`
	`spec:`
	`ingressClassName: nginx`

3 الدومينات اللي عايز لها HTTPS، و `secretName` هو الاسم اللي الـ cert-manager هيحفظ جواه الشهادة.	`tls:`
	`- hosts:`
	`- myapp.example.com`
	`secretName: my-app-tls-secret`

4 نفس قواعد الـ routing. لاحظ إن الـ host هنا لازم يطابق اللي في `tls.hosts`.	`rules:`
	`- host: myapp.example.com`
	`http:`
	`paths:`
	`- path: /`
	`pathType: Prefix`
	`backend:`
	`service: { name: app-service, port: { number: 80 } }`

15.7 إيه اللي هيحصل في الخلفية؟

السحر ده بيحصل في ثواني:

1 **الاكتشاف:** بمجرد ما تعمل apply للـ Ingress، الـ cert-manager بيشوف الـ Annotation.

2. **التحدي (HTTP-01 Challenge):** الـ cert-manager بيبعت طلب لـ Let's Encrypt يقولهم: أنا عايز شهادة للدومين `myapp.example.com`.
3. **إثبات الملكية:** بترد بـ "تحدي" (ملف صغير لازم يكون متاح على الموقع بتاعك عشان نثبت إنك صاحب الدومين). الـ cert-manager بيعدّل الـ Ingress مؤقتاً عشان يخلي التحدي ده يمر، وبعدها بيمسحه.
4. **الاستلام:** بمجرد ما Let's Encrypt تتأكد، بتبعت الشهادة للـ cert-manager، اللي بيحفظها جوه Secret اسمه `my-app-tls-secret`.
5. **التفعيل:** الـ Ingress Controller (Nginx) بيشوف الـ Secret ده وبيبدأ يستخدمه فوراً لتشفير أي اتصال يجيلك!

> ⚠ **ملاحظات**
>
> - Let's Encrypt ليها حدود في عدد الطلبات (Rate Limits). يفضّل في البداية تستخدم سيرفر الـ staging عشان لو غلطت في حاجة متتحظرش من الموقع:
>
> ```
> server: https://acme-staging-v02.api.letsencrypt.org/directory
> ```
>
> - الـ HTTP-01 محتاج الدومين يكون موجّه فعلاً للـ Ingress، والبورت 80 مفتوح من النت. على minikube من غير دومين حقيقي مش هينفع.

تقدر دايماً تتأكد إن الشهادة طلعت صح بالأمر ده:

Terminal

```
moaz@Linux:~$ kubectl get certificate
NAME                READY   SECRET              AGE
my-app-tls-secret   True    my-app-tls-secret   2m
moaz@Linux:~$ kubectl describe certificate my-app-tls-secret
```

الـ `READY = True` معناها الشهادة اتصدرت واتحفظت. ولو `False` افتح الـ describe وشوف الـ Events.

15.8 مشاكل شائعة

مشكلة 1 مشاكل في الشهادة (TLS)

الشهادة مش بتطلع أو READY فاضلة False. افحص بالترتيب:

Terminal

```
kubectl describe certificate my-app-tls-secret
kubectl describe certificaterequest
kubectl get events --sort-by='.lastTimestamp'
```

أشهر الأسباب: الـ DNS مش موجّه للـ Ingress، أو الـ `class` في الـ solver مش مطابق للـ Controller، أو اتحظرت من الـ Rate Limits (جرّب staging).

تشريح الأمر

```
kubectl get events --sort-by='.lastTimestamp'
```

1. `get events` اعرض الأحداث اللي حصلت في الـ Namespace (Pod اتعمل، Image اتسحبت، Error ...).
2. `--sort-by=` رتّب النتايج على حسب حقل معين.
3. `'.lastTimestamp'` الحقل: آخر وقت الحدث حصل فيه، فالأحدث يبقى تحت.

ملخص الفصل

- الـ TLS بيحقق: تشفير، توثيق، سلامة بيانات.
- التشفير غير المتماثل (Public/Private Key) تقيل، فبيستخدم مرة في التعارف، وبعدها تشفير متماثل سريع بـ Session Key.
- الـ Certificate بطاقة هوية مختومة من CA موثوقة.
- الـ TLS Termination: الـ HTTPS بيتفك عند الـ Ingress، وجوه الـ Cluster HTTP.
- cert-manager + ClusterIssuer + annotation في الـ Ingress = شهادات Let's Encrypt بتتجدد لوحدها.

أسئلة مراجعة

1. ليه مبنستخدمش التشفير غير المتماثل في كل البيانات؟
2. المتصفح بيثق في الـ Certificate بتاعة السيرفر ليه؟
3. إيه دور الـ annotation `cert-manager.io/cluster-issuer`؟
4. ليه نبدأ بسيرفر الـ staging بتاع Let's Encrypt؟

صح ولا غلط؟

1. الـ Private Key بيتبعت للمتصفح عشان يفك التشفير. صح / غلط
2. مع TLS Termination الـ Traffic جوه الـ Cluster بيبقى HTTP. صح / غلط
3. cert-manager بيحفظ الشهادة جوه Secret. صح / غلط

16

المرحلة 5 · البيانات والإعدادات

الفصل السادس عشر

الـ Volumes والتخزين: البيانات اللي متضيعش

الـ Container بطبيعته مؤقت: لو اتمسح أو اتعمله Restart، أي ملف اتكتب جواه بيضيع. طب لو عندي Database؟ الفصل ده بيشرح إزاي نخلّي البيانات تعيش أطول من الـ Pod، من أبسط Volume لحد التخزين الدائم PV و PVC.

بعد الفصل ده هتكون عارف

- ليه البيانات بتضيع من الـ Container
- الفرق بين volumes و volumeMounts
- emptyDir و hostPath وإمتى أستخدم كل واحد
- الـ PersistentVolume والـ PersistentVolumeClaim
- الـ StorageClass والـ accessModes والـ reclaimPolicy
- فكرة سريعة عن الـ StatefulSet

إضافة

الشرح الأصلي استخدم الـ Volumes (hostPath) جوه الـ DaemonSet بس، من غير فصل مخصص ليها. الفصل ده **إضافة** بيكمّل الصورة بنفس الأسلوب.

16.1 المشكلة: الـ Container ذاكرته قصيرة

فاكر في فصل الـ DaemonSet لما قولنا إن الـ Pod محتاج ياخد ملفات من الـ Node، فاستخدمنا `volumeMounts` جوه الـ containers و `volumes` جوه الـ spec؟ دي كانت أول مرة نشوف الـ Volumes. تعالى نفهمها كويس.

الـ File System بتاع الـ Container بيتولد معاه ويموت معاه. لو الـ Container وقع والـ kubelet قومه تاني، هيقوم "نضيف" كأنه أول مرة. ده كويس لـ Web Server، بس كارثة لـ Database!

الـ **Volume** ببساطة هو "فولدر" بيتركّب (mount) جوه الـ Container من برّه، فالبيانات اللي بتتكتب فيه متتمسحش مع الـ Container.

volumes (في الـ Pod spec)

بتعرّف فيها **مصدر** التخزين: جاي منين؟ (فولدر على الـ Node، مساحة مؤقتة، Disk دائم، ConfigMap ...).

volumeMounts (في الـ container)

بتحدد هتركّب الـ Volume ده **فين جوه الـ Container** (الـ mountPath)، بالاسم.

16.2 أنواع الـ Volumes الأساسية

النوع	البيانات بتعيش لحد إمتى؟	بيستخدم في إيه؟

`emptyDir`	طول ما الـ Pod موجود (بتعدّي الـ Restart بتاع الـ Container، بس بتتمسح مع الـ Pod)	مساحة مشتركة بين Containers في نفس الـ Pod، Cache مؤقت
`hostPath`	على الـ Node نفسها (لو الـ Pod راح Node تانية مش هيلاقيها)	الـ DaemonSets اللي بتقرأ ملفات الـ Node زي `/var/log`
`secret` / `configMap`	مع الـ Object نفسه	تركيب ملفات إعدادات (الفصول الجاية)
`persistentVolumeClaim`	دائمة، مستقلة عن الـ Pod والـ Node	Databases وأي بيانات مهمة

جدول 16.1 أشهر أنواع الـ Volumes

مثال: emptyDir بين اتنين Containers

shared-pod.yaml

1 عرّفنا Volume اسمه `shared-data` من نوع `emptyDir`. الـ `{}` معناها "من غير إعدادات إضافية".

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: shared-pod
spec:
  volumes:
  - name: shared-data
    emptyDir: {}
```

2 الـ Container الأول بيكتب التاريخ كل 5 ثواني في `/data/log.txt`. الـ `>>` معناها "ضيف في آخر الملف" من غير ما تمسح القديم.

```yaml
  containers:
  - name: writer
    image: busybox
    command: ["sh", "-c", "while true; do date >> /data/log.txt; sleep 5; done"]
    volumeMounts:
    - name: shared-data
      mountPath: /data
```

3 الـ Container التاني شايف نفس الفولدر بس في مسار تاني، فـ nginx هيعرض الملف اللي الأول بيكتبه.

```yaml
  - name: web
    image: nginx
    volumeMounts:
    - name: shared-data
      mountPath: /usr/share/nginx/html
```

معلومة مهمة

الربط بين `volumes` و `volumeMounts` بيتم **بالاسم** (`name: shared-data`). لو الاسم مختلف حرف واحد، الـ Pod هيرفض يتعمل.

16.3 التخزين الدائم: PV و PVC

الـ hostPath مربوط بـ Node واحدة، والـ emptyDir بيموت مع الـ Pod. عشان كده Kubernetes فصل الموضوع لطرفين، زي "شقة" و"عقد إيجار":

(PV) PersistentVolume

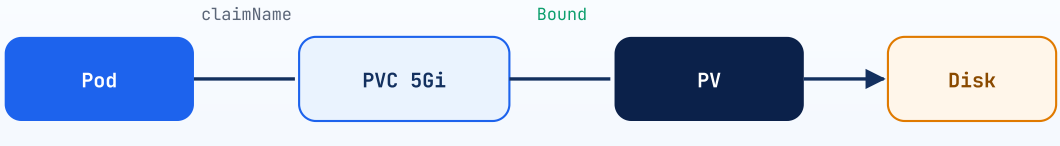
الشقة نفسها: مساحة تخزين حقيقية في الـ Cluster (Disk على Cloud، NFS، ...). بيعملها الـ Admin أو بتتعمل أوتوماتيك.

(PVC) PersistentVolumeClaim

عقد الإيجار: الـ Pod (أو إنت) بتطلب "عايز 5Gi تخزين بالمواصفات دي"، وKubernetes يدوّر على PV مناسب ويربطهم (Bound).

StorageClass

شركة المقاولات: بدل ما الـ Admin يعمل PVs بإيده، الـ StorageClass بيعمل PV جديد أوتوماتيك لكل PVC (Dynamic Provisioning).



شكل 16.1 الـ Pod بيطلب بالـ PVC، والـ PVC بيرتبط بـ PV بيمثل الـ Disk الحقيقي.

الـ PVC

db-pvc.yaml

1 طلب تخزين اسمه `db-pvc`.

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: db-pvc
```

2 طريقة الوصول (شوف الجدول تحت). `ReadWriteOnce` = قراءة وكتابة من Node واحدة.

```yaml
spec:
  accessModes:
  - ReadWriteOnce
```

3 اسم الـ StorageClass. على minikube اسمه `standard` (شوفه بـ `kubectl get storageclass`).

```yaml
  storageClassName: standard
```

4 المساحة المطلوبة: 5 GiB.

```yaml
  resources:
    requests:
      storage: 5Gi
```

استخدام الـ PVC جوه Pod

postgres-pod.yaml

```yaml
spec:
  containers:
  - name: db
    image: postgres:16
    env:
    - name: POSTGRES_PASSWORD
      value: "secret"          # في الحقيقة استخدم Secret (فصل 18)
    volumeMounts:
    - name: db-data
      mountPath: /var/lib/postgresql/data
  volumes:
  - name: db-data
    persistentVolumeClaim:
      claimName: db-pvc        # اسم الـ PVC اللي عملناه
```

دلوقتي لو مسحت الـ Pod وعملته تاني، البيانات هتفضل موجودة لأنها في الـ PV مش في الـ Container.

الـ Mode	الاختصار	المعنى
`ReadWriteOnce`	RWO	قراءة وكتابة من Node واحدة بس (الأشهر، أغلب الـ Cloud Disks)
`ReadOnlyMany`	ROX	قراءة بس من Nodes كتير
`ReadWriteMany`	RWX	قراءة وكتابة من Nodes كتير (محتاج تخزين بيدعمه زي NFS)
`ReadWriteOncePod`	RWOP	Pod واحدة بس في الـ Cluster كله

جدول 16.2 الـ accessModes

القيمة	المعنى
`Delete`	الـ PV والـ Disk يتمسحوا (الافتراضي في الـ Dynamic Provisioning). خلّي بالك!
`Retain`	الـ PV يفضل موجود بالبيانات، والـ Admin يتصرف فيه يدوي. الأأمن للـ Production.

جدول 16.3 الـ reclaimPolicy: إيه اللي يحصل للـ PV لما الـ PVC يتمسح؟

تشريح الأمر

```
kubectl get pv,pvc -A
```

1. `get` اعرض Objects.
2. `pv,pvc` نوعين مع بعض مفصولين بفاصلة من غير مسافة، فبيعرض الـ PVs والـ PVCs في أمر واحد.
3. `-A` اختصار `--all-namespaces`: من كل الـ Namespaces. (الـ PV نفسه مش تبع أي Namespace، بس الـ PVC تبع).

Terminal

```
moaz@Linux:~$ kubectl get pvc
NAME     STATUS   VOLUME              CAPACITY   ACCESS MODES   STORAGECLASS   AGE
db-pvc   Bound    pvc-3f1c...         5Gi        RWO            standard       40s
```

Pending = STATUS

لو الـ PVC فاضل `Pending`، يبقى مفيش PV مناسب ولا StorageClass تقدر تعمل واحد. اعمل `kubectl describe pvc db-pvc` وشوف الـ Events.

16.4 لمحة عن الـ StatefulSet إضافة

الـ Deployment بيعامل الـ Pods كأنهم نسخ متطابقة ممكن تستبدل أي واحد بأي واحد. ده مش مناسب لـ Database cluster لأن كل Pod ليه "هوية" وبيانات خاصة بيه. الـ **StatefulSet** بيدي كل Pod:

- اسم ثابت ومرتب: `db-0`، `db-1`، `db-2`.
- PVC خاص بيه (عن طريق `volumeClaimTemplates`) بيفضل مربوط بيه حتى لو اتعمله Restart على Node تانية.
- تشغيل وإيقاف بالترتيب، و DNS ثابت لكل Pod عن طريق Headless Service.

statefulset.yaml (مختصر)

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: db
spec:
  serviceName: db          # Headless Service (clusterIP: None)
  replicas: 3
  selector:
    matchLabels: { app: db }
  template:
    metadata:
      labels: { app: db }
    spec:
      containers:
      - name: db
        image: postgres:16
        volumeMounts:
        - { name: data, mountPath: /var/lib/postgresql/data }
  volumeClaimTemplates:     # كل Pod ياخد PVC لوحده
  - metadata:
      name: data
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 5Gi
```

ملخص الفصل

- الـ Container مؤقت، والـ Volume هو اللي بيحفظ البيانات برّه.
- `volumes` = المصدر (في الـ Pod)، و `volumeMounts` = مكان التركيب (في الـ Container)، والربط بالاسم.
- `emptyDir` بيعيش مع الـ Pod، `hostPath` مع الـ Node، والـ PVC دائم.
- الـ PVC طلب، الـ PV تخزين حقيقي، والـ StorageClass بيعمل PVs أوتوماتيك.
- الـ StatefulSet للتطبيقات اللي كل Pod فيها ليه هوية وبيانات خاصة.

أسئلة مراجعة

1. إيه الفرق بين `volumes` و `volumeMounts`؟
2. ليه الـ hostPath مش مناسب لـ Database في Cluster فيه أكتر من Node؟
3. الـ PVC حالته Pending، تعمل إيه؟
4. إمتى تختار `Retain` بدل `Delete`؟

صح ولا غلط؟

1. البيانات في emptyDir بتفضل موجودة بعد مسح الـ Pod. صح / غلط
2. الـ ReadWriteOnce معناها Pod واحدة بس في الـ Cluster كله. صح / غلط
3. الـ StatefulSet بيدي كل Pod اسم ثابت زي db-0. صح / غلط

17

المرحلة 5 · البيانات والإعدادات

الفصل السابع عشر

الـ ConfigMaps: الإعدادات برّه الكود

في خلال كتابتك للـ YAML Files هتحتاج قيم تـ configure بيها حاجات، والقيم دي مش ثابتة. الفصل ده بيشرح إزاي نجمّعها في مكان واحد اسمه ConfigMap، وإزاي الـ Pod ياخدها كـ Environment Variables أو كملفات.

بعد الفصل ده هتكون عارف

- ليه نفصل الإعدادات عن الكود
- إنشاء ConfigMap بالأوامر وبالـ YAML
- استخدامه كـ env و envFrom
- تركيبه كملفات بالـ Volume
- إيه اللي يحصل لما أعدّل الـ ConfigMap

17.1 الفكرة: متغيرات في مكان واحد

في خلال كتابتك للـ YAML Files هتحتاج تستخدم قيم عشان تـ configure حاجة، ولكن القيم دي مش ثابتة دايماً، ممكن نحتاج نغيرها. فهل من المنطقي إني كل ما أحتاج أغير فيها أدخل على كل الـ YAML Files اللي مستخدم فيها القيم دي وأغيرها بإيدي؟ أكيد لا. الأحسن إني أخليهم عبارة عن Variables (متغيرات) عشان أقدر أغير القيم دي بسهولة... إزاي برضو؟

خلينا نتخيل معايا المتغيرات دي:

المتغيرات

```
DB_HOST = "localhost"
LOG_LEVEL = "debug"
MAX_CONNECTIONS = "10"
```

بدل ما أعرّفهم جوه كل YAML File وأحط قيمهم جوه نفس الـ YAML، لا، أنا أعمل YAML File منفصل وليكن اسمه `var.yaml` مجمّع فيه كل القيم دي، ولما يكون في YAML File محتاج يستخدم قيم المتغيرات دي يكلم الـ YAML ده ياخدهم منه. بحيث لو غيرت في القيم دي في ملف `var.yaml` هبقى وفّرت مجهود إني أروح لكل الـ YAML Files وأغير بإيدي.

نفس الفكرة دي Kubernetes بيوفرهالنا بحاجة اسمها الـ **ConfigMap**.

> **التعريف**
>
> الـ ConfigMap هو Object في Kubernetes بيخزن إعدادات التطبيق منفصلة عن الكود. بدل ما تـ hardcode القيم (تكتبها بنفسك بشكل ثابت) في الـ image، بتحطها في ConfigMap، والـ Pod بياخدها منه وقت التشغيل.

إيه أنواع الحاجات اللي ممكن أخزنها في الـ ConfigMap؟

- إعدادات البيئة (dev، staging، prod).

- الـ URLs وأسماء الـ hostnames.
- الـ Feature flags.
- ملفات config كاملة زي (`nginx.conf` ، `app.properties`).

> ⚠ **خلي بالك**
>
> الـ ConfigMap للحاجات **غير السرية** بس. الباسوردات والـ Tokens مكانها الـ Secret (الفصل الجاي). وحجم الـ ConfigMap الواحد أقصاه 1 MiB.

17.2 إنشاء ConfigMap من الـ Terminal

Terminal

```
kubectl create configmap <config-name> \
    --from-literal=<key>=<value>
# مثال
kubectl create configmap app-config \
    --from-literal=APP_COLOR=blue \
    --from-literal=APP_MOD=prod
```

تشريح الأمر

```
kubectl create configmap app-config --from-literal=APP_COLOR=blue \
```

1	`create configmap`	اعمل Object جديد من نوع ConfigMap (بشكل Imperative، من غير ملف YAML).
2	`app-config`	اسم الـ ConfigMap، وهو اللي الـ Pods هتنده بيه.
3	`--from-literal=`	"خد القيمة دي مكتوبة زي ما هي" في الأمر نفسه. ممكن تكررها أكتر من مرة لأكتر من متغير.
4	`APP_COLOR=blue`	الـ key قبل علامة `=` والـ value بعدها.
5	`\`	آخر السطر معناها "الأمر لسه مكمّل في السطر اللي تحت"، بس عشان القراءة. لازم متكونش بعدها أي مسافة.

من ملف كامل إضافة

تشريح الأمر

```
kubectl create configmap nginx-conf --from-file=nginx.conf
```

1	`--from-file=`	خد ملف من جهازك وحطه جوه الـ ConfigMap.
2	`nginx.conf`	اسم الملف هيبقى هو الـ key، ومحتواه كله هو الـ value. ولو عايز key مختلف: `--from-file=default.conf=nginx.conf`.

Terminal

```
moaz@Linux:~$ kubectl get configmap app-config -o yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_COLOR: blue
  APP_MOD: prod
```

17.3 ConfigMap بالـ YAML إضافة

app-config.yaml

1 الـ ConfigMap تبع الـ `v1` زي الـ Pod، ومفيهوش `spec`.

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
```

2 بدل الـ spec بنكتب `data`: أزواج key: value. القيم دايماً نصوص، فحط الأرقام بين علامات تنصيص.

```yaml
data:
  DB_HOST: "localhost"
  LOG_LEVEL: "debug"
  MAX_CONNECTIONS: "10"
```

3 علامة `|` في YAML معناها "اللي جاي نص متعدد السطور زي ما هو". مفيد لملفات الـ config الكاملة.

```yaml
  app.properties: |
    color=blue
    mode=prod
```

17.4 إزاي الـ Pod ياخد القيم؟ إضافة

(1) متغير واحد بالاسم: valueFrom

pod · env

1 اسم الـ Environment Variable اللي هيظهر جوه الـ Container.

```yaml
    env:
    - name: LOG_LEVEL
```

2 "هات القيمة من ConfigMap اسمه `app-config`، من الـ key اللي اسمه `LOG_LEVEL`".

```yaml
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: LOG_LEVEL
```

(2) كل الـ keys مرة واحدة: envFrom

pod · envFrom

1 كل key في الـ ConfigMap بيبقى Environment Variable بنفس الاسم. أسهل، بس خلي بالك من تضارب الأسماء.	`envFrom:`
	`- configMapRef:`
	`  name: app-config`

(3) كملفات: Volume

pod · volume

1 فولدر `/etc/nginx/conf.d` جوه الـ Container هيبقى فيه ملف لكل key.	`containers:`
	`- name: web`
	`  image: nginx`
	`  volumeMounts:`
	`  - name: conf`
	`  mountPath: /etc/nginx/conf.d`

2 مصدر الـ Volume هو الـ ConfigMap `nginx-conf` (نفس فكرة الـ Volumes في الفصل اللي فات).	`volumes:`
	`- name: conf`
	`  configMap:`
	`  name: nginx-conf`

تشريح الأمر

```
kubectl exec my-pod -- printenv LOG_LEVEL
          (1)    (2)  (3)
```

1. `exec my-pod` — نفّذ أمر جوه الـ Container بتاع الـ Pod `my-pod`.
2. `--` — فاصل: كل اللي بعده هو الأمر اللي هيتنفّذ جوه الـ Container، مش options لـ kubectl.
3. `printenv LOG_LEVEL` — اطبع قيمة المتغير، عشان تتأكد إن الـ ConfigMap وصل.

17.5 لو عدّلت الـ ConfigMap، إيه اللي يحصل؟ إضافة

⚠ دقة: التغيير مش دايماً بيوصل علطول

- **لو بتستخدمه كـ env أو envFrom:** الـ Environment Variables بتتقري مرة واحدة وقت تشغيل الـ Container. التعديل **مش هيوصل** غير لما الـ Pods تتعمل من جديد، مثلاً بـ `kubectl rollout restart deployment/my-app`.
- **لو بتستخدمه كـ Volume:** الملفات بتتحدّث لوحدها بعد شوية (ممكن دقيقة أو أكتر)، بس التطبيق نفسه لازم يعيد قراية الملف.
- لو عامله `subPath` في الـ volumeMount، الملف مش بيتحدّث خالص.

Terminal

```
kubectl edit configmap app-config          # تعديل مباشر في الـ Editor
kubectl apply -f app-config.yaml           # أو عدّل الملف واعمل apply
kubectl rollout restart deployment/my-app  # عشان الـ env يتقري من جديد
```

ملخص الفصل

- الـ ConfigMap بيفصل الإعدادات (غير السرية) عن الـ image.
- بيتعمل بـ `--from-literal` أو `--from-file` أو YAML فيه `data`.
- الـ Pod بياخده بـ `valueFrom.configMapKeyRef` أو `envFrom` أو كـ Volume.
- تعديل الـ ConfigMap مش بيوصل للـ env غير بعد Restart، لكن الـ Volume بيتحدّث لوحده.

أسئلة مراجعة

1. ليه منكتبش القيم مباشرة جوه الـ image أو الـ YAML بتاع كل Pod؟
2. إيه الفرق بين `env.valueFrom` و `envFrom`؟
3. عدّلت قيمة في ConfigMap والتطبيق لسه شايف القديمة، ليه؟ وإيه الحل؟

صح ولا غلط؟

1. الـ ConfigMap مكان مناسب لباسورد الـ Database. صح / غلط
2. علامة `\` في آخر السطر معناها إن الأمر مكمّل في السطر اللي بعده. صح / غلط
3. في `--from-file=nginx.conf` اسم الملف بيبقى هو الـ key. صح / غلط

18

المرحلة 5 · البيانات والإعدادات

الفصل التامن عشر

الـ Secrets: البيانات الحساسة

الـ Secret هو أخو الـ ConfigMap، بس معمول للحاجات اللي محدش المفروض يشوفها: باسوردات، Tokens، مفاتيح، شهادات TLS. الفصل ده بيشرح بيتعمل إزاي، وبيتستخدم إزاي، وإيه حدود الأمان بتاعه بصراحة.

بعد الفصل ده هتكون عارف

- الفرق بين Secret و ConfigMap
- ليه base64 مش تشفير
- إنشاء Secret بالأوامر وبالـ YAML (data و stringData)
- استخدامه في الـ Pod كـ env وكملفات
- تأمين الـ Secrets بجد

إضافة

الشرح الأصلي ذكر الـ Secrets كعنوان بس، فالفصل ده **إضافة** بنفس الأسلوب.

18.1 يعني إيه Secret؟

نفس فكرة الـ ConfigMap بالظبط: keys و values الـ Pod بياخدها كـ env أو كملفات. الفرق إن Kubernetes بيتعامل معاه على إنه حساس: مش بيظهر في `kubectl describe`، وبيتخزن في الـ Node في الذاكرة (tmpfs) مش على الـ Disk، وتقدر تحدد بالـ RBAC مين يقدر يقراه.

⚠ أهم معلومة في الفصل: base64 مش تشفير

القيم جوه الـ Secret بتتكتب بـ **base64**، وده مجرد "ترميز" (encoding) عشان أي بيانات حتى لو Binary تتكتب كنص. أي حد يقدر يفكه في ثانية:

Terminal

```
moaz@Linux:~$ echo -n 'S3cr3t!' | base64
UzNjcjN0IQ==
moaz@Linux:~$ echo 'UzNjcjN0IQ==' | base64 -d
S3cr3t!
```

يعني اللي معاه صلاحية يقرا الـ Secret من الـ API، شايف الباسورد. الأمان الحقيقي بييجي من الـ RBAC والتشفير في الـ etcd (تحت).

تشريح الأمر

```
echo -n 'S3cr3t!' | base64
```

1. `echo` اطبع النص.
2. `-n` من غير سطر جديد في الآخر. لو نسيتها، الـ base64 هيشفّر الـ newline معاه والباسورد هتبقى غلط!
3. `'S3cr3t!'` النص بين علامات تنصيص مفردة، عشان الـ Shell ميفسّرش علامات زي `!` و `$`.
4. `|` الـ Pipe: خد ناتج الأمر اللي على الشمال وابعته كمدخل للأمر اللي على اليمين.
5. `base64` حوّل المدخل لـ base64. ومع `-d` بيرجّعه (decode) للأصل.

18.2 إنشاء Secret من الـ Terminal

تشريح الأمر

```
kubectl create secret generic db-secret --from-literal=DB_PASSWORD='S3cr3t!'
```

1. `create secret` اعمل Object من نوع Secret.
2. `generic` نوع الـ Secret: عام (keys و values عادية). فيه أنواع تانية زي `tls` و `docker-registry`.
3. `db-secret` اسم الـ Secret.
4. `--from-literal=` زي الـ ConfigMap بالظبط: value=key. والـ kubectl هو اللي بيعمل base64 لوحده.

تشريح الأمر

```
kubectl create secret tls my-tls --cert=tls.crt --key=tls.key
```

1. `tls` نوع مخصص للشهادات، بيتحفظ فيه key اسمه `tls.crt` وواحد `tls.key`. ده نفس النوع اللي الـ cert-manager بيعمله.
2. `--cert=` مسار ملف الشهادة على جهازك.
3. `--key=` مسار ملف المفتاح الخاص (Private Key).

معلومة مهمة

لو بتعمل Pull لـ images من Registry خاص، فيه نوع `docker-registry`، وبتحطه في الـ Pod تحت `imagePullSecrets`.

18.3 Secret بالـ YAML

db-secret.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
```

1 `Opaque` هو النوع العام (نفس `generic` في الأمر).

```yaml
data:
  DB_PASSWORD: UzNjcjN0IQ==
```

2 في `data` لازم تكتب القيمة **متحوّلة base64** بإيدك.

```yaml
stringData:
  DB_USER: admin
```

3 في `stringData` تكتبها نص عادي، وKubernetes هيحوّلها لوحده. أسهل وأقل غلط.

> ⚠ **خلي بالك**
>
> متعملش `git commit` لملف Secret فيه قيم حقيقية! استخدم أدوات زي **Sealed Secrets** أو **External Secrets Operator** (بيجيب القيم من Vault أو AWS Secrets Manager).

18.4 استخدام الـ Secret في الـ Pod

pod · secret

```yaml
    env:
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: DB_PASSWORD
```

1 زي `configMapKeyRef` بالظبط، بس `secretKeyRef`. الـ Container بيشوف القيمة الأصلية (مفكوكة).

```yaml
    envFrom:
    - secretRef:
        name: db-secret
```

2 أو كل الـ keys مرة واحدة.

```yaml
    volumeMounts:
    - name: creds
      mountPath: /etc/creds
      readOnly: true
  volumes:
  - name: creds
    secret:
      secretName: db-secret
```

3 أو كملفات: `/etc/creds/DB_PASSWORD`. لاحظ إنها `secretName` مش `name`. والطريقة دي أأمن من الـ env لأن الـ env ممكن يظهر في الـ logs.

تشريح الأمر

```
kubectl get secret db-secret -o jsonpath='{.data.DB_PASSWORD}' | base64 -d
```

1. `get secret db-secret` هات الـ Secret ده.
2. `-o jsonpath=` بدل الجدول، اطبع حقل واحد بس من الـ Object.
3. `'{.data.DB_PASSWORD}'` المسار: جوه `data` هات `DB_PASSWORD`. النقطة الأولى = الـ Object نفسه.
4. `| base64 -d` ابعت القيمة للـ base64 يفكها، فتشوف الباسورد الأصلي. (وده دليل إن أي حد معاه صلاحية get يقدر يشوفه).

18.5 تأمين الـ Secrets بجد

1. **RBAC:** اسمح بـ `get` و `list` على الـ secrets للي محتاجها بس (الفصل الجاي).
2. **Encryption at Rest:** بشكل افتراضي الـ Secrets في الـ etcd متخزنة base64 بس. فعّل الـ `EncryptionConfiguration` في الـ API Server (أو KMS في الـ Cloud) عشان تتشفر فعلاً.
3. **متطبعهاش:** خلي بالك من الـ logs، والـ `kubectl get -o yaml` قدام حد.
4. **Immutable:** تقدر تحط `immutable: true` للـ Secrets أو الـ ConfigMaps اللي مش هتتغير، عشان محدش يعدّلها بالغلط.

ملخص الفصل

- الـ Secret زي الـ ConfigMap بس للبيانات الحساسة.
- الـ base64 ترميز مش تشفير، والأمان الحقيقي من الـ RBAC والـ Encryption at Rest.
- `data` بقيم base64، و `stringData` بنص عادي.
- بيتستخدم بـ `secretKeyRef` أو `secretRef` أو Volume بـ `secretName`.

أسئلة مراجعة

1. ليه الـ base64 مش بيحمي الباسورد؟
2. إيه الفرق بين `data` و `stringData`؟
3. ليه `echo -n` مهمة قبل الـ base64؟
4. اذكر طريقتين لتأمين الـ Secrets في الـ Cluster.

صح ولا غلط؟

1. الـ Secrets متشفرة في الـ etcd بشكل افتراضي. صح / غلط
2. الـ Container بيشوف قيمة الـ Secret مفكوكة (مش base64). صح / غلط
3. النوع `tls` بيحتوي على `tls.crt` و `tls.key`. صح / غلط

19

المرحلة 6 · التشغيل الاحترافي

الفصل التاسع عشر

الـ RBAC: مين يقدر يعمل إيه؟

لما فريق كامل بيشتغل على نفس الـ Cluster، مش منطقي كل واحد يبقى معاه صلاحية يمسح أي حاجة.
الـ RBAC هو نظام الصلاحيات اللي بيحدد مين يقدر يعمل إيه، وفين.

بعد الفصل ده هتكون عارف

- يعني إيه RBAC وليه مهم
- الـ Role والـ RoleBinding جوه Namespace
- الـ ClusterRole والـ ClusterRoleBinding
- الـ verbs والـ resources والـ apiGroups
- الـ ServiceAccount للـ Pods
- التأكد من الصلاحيات بـ kubectl auth can-i

19.1 إيه هي الـ RBAC؟ مين يقدر يعمل إيه جوه الـ Namespace؟

الـ **RBAC (Role-Based Access Control)** ده نظام الصلاحيات. بنقسمه لخطوتين:

Role

بنحدد فيها إيه الأفعال المسموح بيها جوه الـ Namespace دي.

RoleBinding

بنربط الـ Role دي بأشخاص حقيقيين (الموظفين) أو بـ ServiceAccounts.

تخيلها كده: الـ Role هي "الكارنيه" اللي مكتوب عليه مسموح يدخل أنهي أوض، والـ RoleBinding هو إنك تسلّم الكارنيه ده لشخص معين.

معلومة مهمة

الـ RBAC في Kubernetes "مسموح فقط" (additive): مفيش قواعد "ممنوع". أي حاجة مش مسموحة صراحةً تبقى ممنوعة.

19.2 الـ Role بالتفصيل إضافة

pod-reader-role.yaml

1 الـ Role دايماً تبع Namespace واحدة (هنا `dev`)، وصلاحياتها جوهها بس.

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: dev
```

```yaml
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
```

2 الـ `""` (فاضي) = الـ core group اللي فيه pods و services و configmaps و secrets. للـ Deployments بتكتب `"apps"`.

3 على أنهي Objects. `pods/log` اسمها subresource وهي اللي بتسمح بـ `kubectl logs`.

4 الأفعال المسموحة: هنا قراية بس.

الـ verb	المعنى	مثال أمر
`get`	قراية Object واحد بالاسم	`kubectl get pod web`
`list`	عرض الكل	`kubectl get pods`
`watch`	متابعة التغييرات لايف	`kubectl get pods -w`
`create`	إنشاء	`kubectl create / apply`
`patch` / `update`	تعديل	`kubectl edit / apply / scale`
`delete`	مسح	`kubectl delete`
`*`	كل الأفعال (استخدمها بحذر)	

جدول 19.1 الـ verbs الأساسية وبتقابل أنهي أمر

19.3 الـ RoleBinding إضافة

read-pods-binding.yaml

```yaml
apiVersion:
rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: dev
```

1 لازم في نفس الـ Namespace بتاعة الـ Role.

```yaml
subjects:
- kind: User
  name: ahmed
  apiGroup: rbac.authorization.k8s.io
- kind: ServiceAccount
  name: app-sa
  namespace: dev
```

2 الـ `subjects` = مين هياخد الصلاحية: User أو Group أو ServiceAccount.

```yaml
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

3 أنهي Role هنربطها. الـ `roleRef` مينفعش يتعدّل بعد الإنشاء؛ لو عايز تغيره امسح الـ Binding واعمله تاني.

مين هو الـ User؟

Kubernetes معندوش Object اسمه User. المستخدمين بيتعرّفوا من برّه، عن طريق شهادات أو OIDC (زي Google أو Azure AD). الـ RBAC بيستقبل الاسم بس ويطبّق عليه القواعد.

19.4 ClusterRoleBinding g ClusterRole إضافة

	RoleBinding / Role	ClusterRoleBinding / ClusterRole
النطاق	Namespace واحدة	الـ Cluster كله
Objects مش تبع Namespace	مينفعش	ينفع (nodes, pv, namespaces)
مثال	مطوّر يقرا الـ Pods في dev	فريق Ops يشوف الـ Nodes كلها

جدول 19.2 Namespace مقابل Cluster

Trick مفيد

ممكن تعمل **ClusterRole** مرة واحدة وتربطه بـ **RoleBinding** في كل Namespace لوحدها، فالصلاحية تتطبق جوه الـ Namespace دي بس. وKubernetes جاي معاه ClusterRoles جاهزة زي `view` و `edit` و `admin` و `cluster-admin`.

تشريح الأمر

```
kubectl create rolebinding dev-view --clusterrole=view --user=ahmed -n dev
```

1. `create rolebinding dev-view` اعمل RoleBinding اسمه `dev-view`.
2. `--clusterrole=view` اربطه بالـ ClusterRole الجاهز `view` (قراية لكل حاجة ما عدا الـ Secrets). ولو Role عادية: `--role=pod-reader`.
3. `--user=ahmed` الـ subject. وفيه كمان `--group=` و `--serviceaccount=dev:app-sa` (namespace:name).
4. `-n dev` في الـ Namespace دي، فالصلاحية جوه dev بس.

19.5 الـ ServiceAccount: هوية الـ Pod إضافة

الـ Users للبشر، أما الـ **ServiceAccount** فهي هوية للـ Pods نفسها لما تحتاج تكلم الـ API Server (زي أداة CI/CD أو Operator). كل Namespace فيها ServiceAccount اسمه `default`، وأي Pod من غير ما تحدد بياخده.

Terminal

```
kubectl create serviceaccount app-sa -n dev
```

pod · serviceAccountName

```yaml
spec:
  serviceAccountName: app-sa      # الـ Pod هيشتغل بصلاحيات app-sa
  containers:
  - name: app
    image: my-app:1.0
```

19.6 اتأكد من الصلاحيات: kubectl auth can-i إضافة

تشريح الأمر

```
kubectl auth can-i delete pods -n dev --as=ahmed
        (1)      (2)    (3)  (4)
```

1. `auth can-i` اسأل الـ API Server: "أقدر أعمل كذا؟" والرد `yes` أو `no`.
2. `delete pods` الـ verb وبعده الـ resource.
3. `-n dev` في أنهي Namespace.
4. `--as=ahmed` اسأل كأنك المستخدم ده (Impersonation)، ومحتاجة إنت نفسك تكون Admin. ولـ ServiceAccount: `--as=system:serviceaccount:dev:app-sa`.

Terminal

```
moaz@Linux:~$ kubectl auth can-i list pods -n dev --as=ahmed
yes
moaz@Linux:~$ kubectl auth can-i delete pods -n dev --as=ahmed
no
moaz@Linux:~$ kubectl auth can-i --list -n dev --as=ahmed      # كل الصلاحيات
```

Least Privilege

إدّي كل حد أقل صلاحية يقدر يشتغل بيها. ابدأ بـ `view`، وزوّد لما يكون فيه احتياج حقيقي. وابعد عن `cluster-admin` إلا للضرورة.

ملخص الفصل

- الـ Role = إيه المسموح، والـ RoleBinding = لمين. وجوه Namespace واحدة.
- الـ ClusterRole/ClusterRoleBinding للـ Cluster كله وللـ Objects اللي مش تبع Namespace.
- القاعدة = apiGroups + resources + verbs.
- الـ ServiceAccount هوية للـ Pods، وبتتحدد بـ `serviceAccountName`.
- `kubectl auth can-i` بيختبر الصلاحية قبل ما تتفاجئ.

؟ أسئلة مراجعة

1. إيه الفرق بين Role و ClusterRole؟
2. ليه `apiGroups: [""]` فاضي في قاعدة الـ pods؟
3. عايز مطوّر يشوف الـ logs بس في Namespace واحدة، تكتب إيه في الـ resources والـ verbs؟
4. إزاي تتأكد إن ServiceAccount معين يقدر يعمل list للـ secrets؟

صح ولا غلط؟

1. الـ RBAC فيه قواعد "ممنوع" صريحة. صح / غلط
2. ينفع تربط ClusterRole بـ RoleBinding في Namespace واحدة. صح / غلط
3. Kubernetes فيه Object اسمه User بتعمله بـ kubectl create user. صح / غلط

20

المرحلة 6 · التشغيل الاحترافي

الفصل العشرين

الـ HPA: الـ Scaling الأوتوماتيك

فاكر في أول الكتاب لما قولنا إن Kubernetes بيقدر يزوّد عدد الـ Pods لما الضغط يزيد؟ الأداة اللي بتعمل كده اسمها Horizontal Pod Autoscaler. الفصل ده بيشرح هي بتقرر إزاي، ومحتاجة إيه عشان تشتغل.

بعد الفصل ده هتكون عارف

- فكرة الـ HPA ومكانه بين الـ Scaling الأنواع
- ليه الـ metrics-server و الـ requests ضروريين
- المعادلة اللي الـ HPA بيحسب بيها
- إنشاء HPA بالأمر وبالـ YAML (autoscaling/v2)
- اختبار الـ HPA بضغط حقيقي

إضافة

الفصل ده **إضافة**: الشرح الأصلي اتكلم عن الـ Scaling كفكرة في أول الكتاب (Scale Out و Scale Up)، وهنا بنطبقها أوتوماتيك.

20.1 يعني إيه HPA؟

الـ **HPA (Horizontal Pod Autoscaler)** بيراقب استهلاك الـ Pods بتاعة Deployment (مثلاً الـ CPU)، ولو الاستهلاك عدّى الهدف اللي حددته، بيزوّد الـ replicas (Scale Out). ولو الضغط قل، بيقللهم تاني لحد الحد الأدنى.

الأداة	بتعمل إيه
HPA	بتزوّد/بتقلل **عدد الـ Pods** (Horizontal)
VPA	بتعدّل **الـ requests/limits** بتاعة الـ Pod نفسه (Vertical)، add-on منفصل
Cluster Autoscaler	بيزوّد/بيقلل **عدد الـ Nodes** لما الـ Pods متلاقيش مكان

جدول 20.1 أنواع الـ Autoscaling في Kubernetes

20.2 محتاج إيه عشان يشتغل؟

1. **metrics-server:** هو اللي بيجمع أرقام الـ CPU والـ Memory من الـ kubelet على كل Node. من غيره الـ HPA أعمى. على minikube: `minikube addons enable metrics-server`.
2. **resources.requests:** الـ HPA بيحسب النسبة من الـ **requests** مش من الـ limits. لو مفيش requests للـ CPU، مش هيعرف يحسب، وهيظهر `<unknown>`.

20.3 المعادلة

معادلة الـ HPA

```
desiredReplicas = ceil( currentReplicas × currentUsage / targetUsage )
```

مثال: عندك 2 Pods، والهدف 50% CPU، والاستهلاك الحالي 100%. يبقى `ceil(2 × 100 / 50) = 4` Pods. والـ `ceil` معناها التقريب لفوق.

> **معلومة مهمة**
>
> عشان ميفضلش يزوّد ويقلل كل شوية (flapping)، الـ HPA بيستنى فترة استقرار قبل ما يقلل: افتراضياً 5 دقايق.

20.4 إنشاء HPA بالأمر

تشريح الأمر

```
kubectl autoscale deployment web --cpu-percent=50 --min=2 --max=10
```

1. `autoscale deployment web` — اعمل HPA للـ Deployment اللي اسمه `web`.
2. `--cpu-percent=50` — الهدف: متوسط استهلاك الـ CPU لكل Pod = 50% من الـ request بتاعه.
3. `--min=2` — أقل عدد Pods مهما الضغط قل.
4. `--max=10` — أقصى عدد Pods مهما الضغط زاد (عشان متصرفش موارد الـ Cluster كلها).

20.5 إنشاء HPA بالـ YAML

web-hpa.yaml

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
  minReplicas: 2
  maxReplicas: 10
```

1. الإصدار `autoscaling/v2` هو الحالي، وبيدعم أكتر من metric.
2. مين اللي هيتعمله Scale: الـ Deployment `web`.
3. الحدود.

```yaml
metrics:
- type: Resource
  resource:
    name: cpu
    target:
      type: Utilization
      averageUtilization: 50
```

4 راقب الـ CPU، والهدف متوسط 50% من الـ requests. وتقدر تضيف عنصر تاني لـ `memory`.

> **خلي بالك**
>
> لو عامل HPA للـ Deployment، شيل `replicas` من الـ YAML بتاع الـ Deployment أو متعملش له apply تاني بقيمة ثابتة، وإلا كل apply هيرجّع العدد ويتخانق مع الـ HPA.

20.6 تابعه وجرّبه

تشريح الأمر

```
kubectl get hpa -w
          1   2
```

1 `get hpa` اعرض الـ HPAs (`hpa` اختصار لـ `horizontalpodautoscaler`).

2 `-w` اختصار لـ `--watch`: متقفلش، وكل ما حاجة تتغير اطبع سطر جديد. اخرج بـ `Ctrl+C`.

```
Terminal
moaz@Linux:~$ kubectl get hpa -w
NAME      REFERENCE         TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
web-hpa   Deployment/web    cpu: 12%/50%    2         10        2          1m
web-hpa   Deployment/web    cpu: 95%/50%    2         10        2          2m
web-hpa   Deployment/web    cpu: 95%/50%    2         10        4          2m
```

ولو عايز تعمل ضغط تجريبي على الـ Service:

تشريح الأمر

```
kubectl run load --rm -it --image=busybox -- sh -c "while true; do wget -q -O- http://web; done"
```

1. `run load` شغّل Pod مؤقت اسمه `load`.
2. `--rm` امسح الـ Pod أوتوماتيك أول ما تخرج منه.
3. `-it` تفاعلي: `-i` خلي الـ input مفتوح، و `-t` اعمل Terminal.
4. `--image=busybox` image صغيرة فيها أدوات زي wget.
5. `--` كل اللي بعدها هو الأمر جوه الـ Container.
6. `sh -c "..."` شغّل الـ Shell ونفّذ الكلام اللي بين علامات التنصيص.
7. `while true; do ...; done` Loop مالهاش نهاية، و `;` بتفصل بين الأوامر.
8. `wget -q -O- http://web` اطلب الصفحة. `-q` من غير رسايل، و `-O-` اطبع الناتج على الشاشة بدل ملف. و `web` اسم الـ Service (DNS).

ملخص الفصل

- الـ HPA بيغير عدد الـ Pods حسب الاستهلاك، بين min و max.
- محتاج metrics-server و requests للـ CPU.
- المعادلة: ceil(الحالي × الاستهلاك / الهدف).
- اعمله بـ `kubectl autoscale` أو YAML بـ `autoscaling/v2`.

أسئلة مراجعة

1. الـ TARGETS ظاهرة `<unknown>/50%`، إيه السببين المحتملين؟
2. 3 Pods، الهدف 60%، الاستهلاك 90%. الـ HPA هيوصل لكام Pod؟
3. إيه الفرق بين الـ HPA والـ Cluster Autoscaler؟

صح ولا غلط؟

1. الـ HPA بيحسب النسبة من الـ limits. صح / غلط
2. `kubectl run` في `--rm` بتمسح الـ Pod بعد ما تخرج. صح / غلط
3. الـ HPA بيزوّد عدد الـ Nodes. صح / غلط

21

المرحلة 6 · التشغيل الاحترافي

الفصل الحادي والعشرين

الـ Troubleshooting: إزاي تحل أي مشكلة

أهم مهارة في Kubernetes مش إنك تكتب YAML، دي إنك تعرف تعمل إيه لما حاجة متشتغلش. الفصل ده بيديك الأوامر الأساسية للتشخيص، ومعنى كل حالة غريبة هتشوفها في عمود STATUS، وخريطة تمشي عليها خطوة بخطوة.

بعد الفصل ده هتكون عارف

- الأوامر الخمسة للتشخيص: get, describe, logs, exec, events
- معنى Pending و ImagePullBackOff و CrashLoopBackOff و OOMKilled
- تشخيص مشاكل الـ Service والـ Endpoints
- الـ port-forward والـ kubectl debug
- خريطة التشخيص خطوة بخطوة

إضافة

الفصل ده **إضافة**: بيجمّع

21.1 صندوق العدة: أوامر التشخيص

`kubectl get pods -o wide`	نظرة عامة: الحالة، عدد الـ Restarts، والـ Node والـ IP.
`kubectl describe pod <name>`	التفاصيل كلها، والأهم جزء **Events** في الآخر.
`kubectl logs <name>`	اللي التطبيق طبعه (stdout/stderr).
`kubectl get events --sort-by=.lastTimestamp`	كل أحداث الـ Namespace بالترتيب.
`kubectl exec -it <name> -- sh`	ادخل جوه الـ Container وجرّب بنفسك.

الـ logs بالتفصيل

تشريح الأمر

```
kubectl logs web-7d4b -c app --previous --tail=50 -f
```
(1) (2) (3) (4) (5)

1. `logs web-7d4b` — اعرض الـ logs بتاعة الـ Pod ده.
2. `-c app` — لو الـ Pod فيه أكتر من Container، اختار أنهي واحد بالاسم. (من غيرها هيطلب منك تحدد).
3. `--previous` — الـ logs بتاعة الـ Container **اللي مات قبل كده**، مش الحالي. أهم flag في الـ CrashLoopBackOff لأن الجديد لسه مطبعش حاجة. (اختصارها `-p`).
4. `--tail=50` — آخر 50 سطر بس.
5. `-f` — اختصار `--follow`: فضّل متابع وكل سطر جديد يظهر لايف.. ومع `--previous` ملهاش لازمة لأن الـ Container القديم خلص أصلاً، فاستخدمهم كل واحد لوحده.

معلومة مهمة

تقدر كمان تشوف logs كل الـ Pods اللي عليها label: `kubectl logs -l app=web --all-containers`، أو logs الـ Deployment: `kubectl logs deploy/web`.

21.2 قاموس الحالات (STATUS)

الحالة	معناها	تعمل إيه؟
`Pending`	الـ Scheduler ملقاش Node مناسبة، أو مستني PVC	`describe pod` : شوف *Insufficient* *cpu/memory* أو taints أو nodeSelector أو PVC Pending
`ErrImagePull` / `ImagePullBackOff`	مش قادر يسحب الـ image	اتأكد من الاسم والـ tag، ومن الـ `imagePullSecrets` لو Registry خاص
`CrashLoopBackOff`	الـ Container بيقوم ويقع، والـ kubelet بيستنى وقت أطول كل مرة	`logs --previous`، وشوف الـ command والـ env والـ liveness probe
`OOMKilled`	عدّى الـ memory limit فاتقتل (exit code 137)	زوّد الـ limit أو صلّح الـ memory leak
`CreateContainerConfigError`	ناقص ConfigMap أو Secret أو key متشار عليه	`describe pod` هيقولك أنهي واحد مش موجود
`Running` بس `0/1 READY`	الـ readinessProbe فاشلة	راجع الـ probe (path/port) والـ Events
`Terminating` **لفترة طويلة**	فيه finalizer أو الـ Node مش بترد	استنى الـ grace period، واعرف السبب قبل أي force delete

جدول 21.1 أشهر الحالات الغلط ومعناها وأول خطوة

تشريح الأمر

```
kubectl get pod web-7d4b -o jsonpath='{.status.containerStatuses[0].lastState}'
```

1. `-o jsonpath=` اطبع جزء محدد من الـ Object.
2. `.status.containerStatuses` حالة الـ Containers جوه الـ status.
3. `[0]` أول Container في القايمة (الترقيم يبدأ من صفر).
4. `.lastState` حالة آخر مرة الـ Container مات: فيها السبب (زي `OOMKilled`) والـ exitCode.

21.3 لو المشكلة في الـ Service

الـ Pods شغالة بس مش قادر توصل للتطبيق؟ اتأكد إن الـ Service لاقية Pods أصلاً:

Terminal

```
moaz@Linux:~$ kubectl get endpoints web
NAME   ENDPOINTS                         AGE
web    10.244.1.5:8080,10.244.2.7:8080   5m
```

- لو الـ ENDPOINTS فاضية `<none>`: الـ `selector` في الـ Service مش مطابق لـ labels الـ Pods، أو الـ Pods مش Ready.
- لو موجودة: اتأكد إن الـ `targetPort` هو البورت اللي التطبيق بيسمع عليه فعلاً.

تشريح الأمر

```
kubectl port-forward svc/web 8080:80
```

1. `port-forward` افتح نفق من جهازك لحد جوه الـ Cluster، من غير Ingress ولا NodePort.
2. `svc/web` النوع/الاسم: الـ Service `web`. وينفع `pod/...` أو `deploy/...`.
3. `8080:80` البورت على جهازك (شمال) : البورت على الـ Service (يمين). افتح `localhost:8080`.

ولو عايز تختبر من جوه الـ Cluster (DNS مثلاً):

Terminal

```
kubectl run tmp --rm -it --image=busybox -- nslookup web.default.svc.cluster.local
```

21.4 الـ kubectl debug

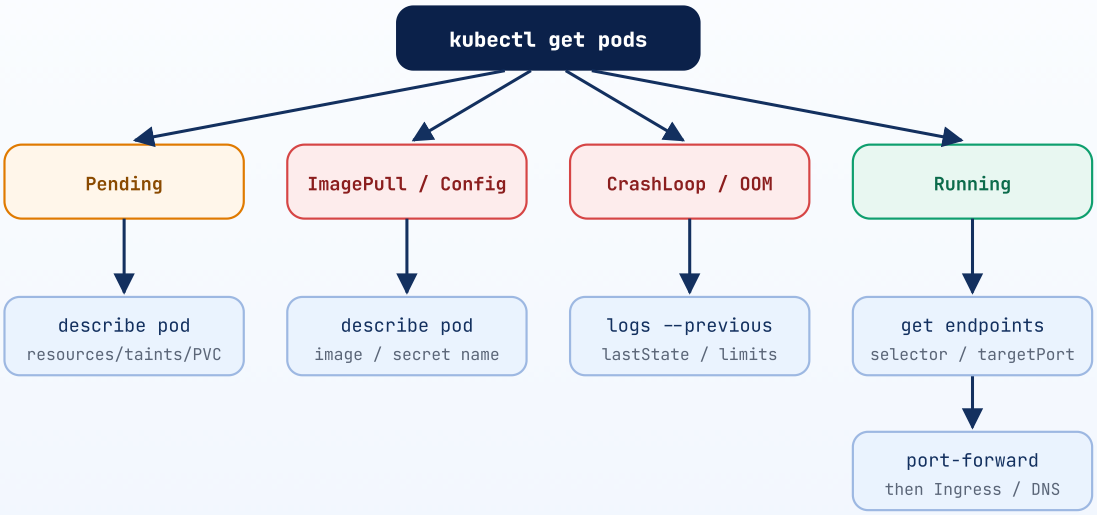
فيه images صغيرة جداً (distroless) مفيهاش حتى `sh`، فالـ exec مش هينفع. هنا بنستخدم **Ephemeral Container**: Container مؤقت بيتضاف للـ Pod الشغال وفيه أدوات:

تشريح الأمر

```
kubectl debug -it web-7d4b --image=busybox --target=app
```

1	`debug -it web-7d4b`	ضيف Container للتشخيص للـ Pod ده وادخل عليه تفاعلي.
2	`--image=busybox`	الـ image اللي فيها الأدوات.
3	`--target=app`	شارك الـ processes بتاعة Container اسمه `app`، فتقدر تشوفها بـ `ps`.

21.5 خريطة التشخيص



شكل 21.1 ابدأ دايماً من الـ STATUS، وهو هيقولك تكمّل في أنهي طريق.

عادة المحترفين

قبل ما تغير أي حاجة، اقرا الـ **Events**. في 80% من المشاكل السبب مكتوب بالحرف في آخر `kubectl describe`.

ملخص الفصل

- الترتيب: get ← describe (Events) ← logs (previous--) ← exec/debug.
- الـ STATUS بيحدد الطريق: Scheduling = Pending، image = BackOff أو تشغيل، memory = OOMKilled.
- مشاكل الـ Service غالباً selector أو targetPort، واكشفها بـ `get endpoints`.
- الـ port-forward بيعزل المشكلة: التطبيق ولا الشبكة؟

أسئلة مراجعة

1. Pod في CrashLoopBackOff و `kubectl logs` فاضي. تعمل إيه؟
2. إيه معنى exit code 137؟
3. الـ Service شغالة بس الـ ENDPOINTS فاضية، إيه السببين الأشهر؟
4. إمتى تستخدم `kubectl debug` بدل `kubectl exec`؟

صح ولا غلط؟

1. الحالة Pending معناها غالباً إن الـ image فيها مشكلة. صح / غلط
2. في `port-forward 8080:80` الرقم الأول هو البورت على جهازك. صح / غلط
3. الـ CreateContainerConfigError ممكن يكون سببه Secret مش موجود. صح / غلط

22

المرحلة 6 · التشغيل الاحترافي

الفصل التاني والعشرين

الـ Helm: مدير الحزم لـ Kubernetes

اتعلمنا إزاي تعمل Pods و Deployments و Services بملفات YAML، ولاحظت أكيد إن حتى تطبيق بسيط محتاج ملفات كتير، وإدارتهم عبر بيئات مختلفة (dev، staging، production) بتبقى صعبة بسرعة. هنا بييجي دور Helm.

بعد الفصل ده هتكون عارف

- المشكلة اللي Helm بيحلها
- الـ Chart والـ Release والـ Repository
- التسطيب والترقية والرجوع والمسح
- الـ values.yaml و --set و -f
- عمل Chart بنفسك وقراية الـ templates

22.1 المشكلة اللي Helm بيحلها

اتعلمنا إزاي تعمل Pods و Deployments و Services بملفات YAML. لاحظت أكيد إن حتى تطبيق بسيط محتاج ملفات كتير، وإدارتهم عبر بيئات مختلفة (dev، staging، production) بتبقى صعبة بسرعة.

الـ **Helm** هو "مدير الحزم" (Package Manager) لـ Kubernetes، زي `apt` في Ubuntu. بدل ما تكتب 10 ملفات YAML لكل بيئة، بتكتبهم مرة واحدة كـ **Templates** فيها متغيرات، وبتغيّر القيم بس لكل بيئة. وكمان تقدر تسطّب تطبيقات جاهزة (زي cert-manager اللي سطبناه في فصل الـ TLS) بأمر واحد.

22.2 المصطلحات الأساسية إضافة

Chart

الحزمة نفسها: فولدر فيه templates لملفات YAML + ملف قيم افتراضية `values.yaml`.

Release

نسخة متسطّبة من Chart في الـ Cluster باسم معين. ممكن تسطب نفس الـ Chart مرتين باسمين مختلفين.

Repository

مكان على النت فيه Charts كتير (زي `charts.jetstack.io`). وبتدور فيها على **Artifact Hub**.

22.3 دورة حياة الـ Release إضافة

Terminal

```
helm repo add podinfo https://stefanprodan.github.io/podinfo    # أضف مستودع
helm repo update                                                 # حدّث القايمة
helm search repo podinfo                                         # دوّر على Chart
helm install my-web podinfo/podinfo -n web --create-namespace
helm list -A                                                     # كل الـ Releases
helm upgrade my-web podinfo/podinfo -n web --set replicaCount=3
helm history my-web -n web                                       # النسخ اللي فاتت
helm rollback my-web 1 -n web                                    # ارجع للنسخة 1
helm uninstall my-web -n web                                     # امسح كل حاجة
```

تشريح الأمر

```
helm install my-web podinfo/podinfo -n web --create-namespace
     1       2          3              4          5
```

1. `install` سطّب Chart كـ Release جديد.
2. `my-web` اسم الـ Release (من اختيارك)، وبيدخل في أسماء الـ Objects اللي هتتعمل.
3. `podinfo/podinfo` اسم المستودع/اسم الـ Chart (podinfo تطبيق تجريبي صغير مشهور للتجارب).
4. `-n web` الـ Namespace اللي هيتسطب فيها، زي kubectl بالظبط.
5. `--create-namespace` لو الـ Namespace مش موجودة اعملها.

تشريح الأمر

```
helm upgrade --install my-web ./mychart -f values-prod.yaml --set image.tag=1.2.0
     1          2          3       4          3              4
```

1. `upgrade --install` رقّي الـ Release، ولو مش موجود سطّبه. مثالي في الـ CI/CD لأنه بيشتغل في المرتين.
2. `./mychart` Chart من فولدر على جهازك بدل مستودع.
3. `-f values-prod.yaml` ملف قيم بيغطي على القيم الافتراضية (لبيئة الـ production مثلاً).
4. `--set image.tag=1.2.0` غيّر قيمة واحدة من الأمر. النقطة معناها مستوى أعمق: `image:` جواها `tag:`. وليها الأولوية على `-f`.

معلومة مهمة

كل `upgrade` بيعمل Revision جديدة (1، 2، 3 ...). والـ `rollback` نفسه بيعمل Revision جديدة بمحتوى القديمة، فالـ history مش بيضيع.

22.4 جوه الـ Chart إضافة

```
helm create mychart
```

```
mychart/
├── Chart.yaml          # اسم الـ Chart ونسخته (version) ونسخة التطبيق (appVersion)
├── values.yaml         # القيم الافتراضية
├── charts/             # Charts تانية بيعتمد عليها (dependencies)
└── templates/
    ├── deployment.yaml
    ├── service.yaml
    ├── _helpers.tpl    # دوال مساعدة
    └── NOTES.txt       # رسالة بتظهر بعد التسطيب
```

values.yaml

1 قيم عادية، وهي اللي بتغيّرها بـ `--set` أو `-f`.

```yaml
replicaCount: 2
image:
  repository: nginx
  tag: "1.27"
service:
  port: 80
```

templates/deployment.yaml (جزء)

1 أي حاجة بين `{{ }}` بتتحسب. `.Release.Name` = اسم الـ Release، فلو `my-web` الاسم يبقى `my-web-web`.

```yaml
metadata:
  name: {{ .Release.Name }}-web
```

2 `.Values` = محتوى الـ values.yaml بعد دمج `-f` و `--set`.

```yaml
spec:
  replicas: {{ .Values.replicaCount }}
```

3 بتطلع `nginx:1.27`.

```yaml
      containers:
      - name: web
        image: "{{
.Values.image.repository }}:{{
.Values.image.tag }}"
```

4 `if` = حط الجزء ده لو القيمة موجودة. `-` جنب الأقواس تشيل المسافات والسطور الفاضية. `toYaml` تحوّل القيمة لـ YAML، و `| nindent 10` (Pipe زي الـ Shell) تبدأ سطر جديد وتزيح 10 مسافات.

```yaml
        {{- if .Values.resources }}
        resources: {{- toYaml
.Values.resources | nindent 10 }}
        {{- end }}
```

تشريح الأمر

```
helm template my-web ./mychart -f values-prod.yaml > out.yaml
```

1. `template` اعرض الـ YAML النهائي بعد ما القيم تتحط، **من غير** ما تسطب حاجة. أحسن طريقة تتأكد إن الـ Chart مظبوط.
2. `my-web ./mychart` اسم Release تجريبي والـ Chart.
3. `-f values-prod.yaml` بالقيم دي.
4. `> out.yaml` الـ Redirection: احفظ الناتج في ملف بدل الشاشة (ولو الملف موجود هيتكتب عليه).

قبل أي upgrade في Production

- `helm lint ./mychart` لمراجعة الأخطاء.
- `helm upgrade ... --dry-run` عشان تشوف إيه اللي هيتغير.
- `helm get values my-web` عشان تعرف القيم الحالية قبل ما تغير.

ملخص الفصل

- الـ Helm مدير حزم: Chart = حزمة، Release = نسخة متسطبة، Repository = مكان الـ Charts.
- install / upgrade / rollback / uninstall، و `upgrade --install` للـ CI/CD.
- القيم من `values.yaml` ← `-f` ← `--set` (الأخير هو الأقوى).
- `helm template` بيوريك الـ YAML النهائي من غير تسطيب.

أسئلة مراجعة

1. إيه الفرق بين الـ Chart والـ Release؟
2. لو حطيت نفس القيمة في `-f` وفي `--set`، مين يكسب؟
3. إزاي ترجع لنسخة قديمة بعد upgrade بايظ؟
4. إيه فايدة `helm template`؟

صح ولا غلط؟

1. ينفع تسطب نفس الـ Chart مرتين بأسماء Releases مختلفة. صح / غلط
2. `helm rollback` بيمسح الـ history. صح / غلط
3. في `--set image.tag=1.2.0` النقطة معناها مستوى أعمق في الـ values. صح / غلط

ملحق أ

Cheat Sheet: أوامر kubectl و Helm

ورقة مراجعة سريعة لكل الأوامر اللي في الكتاب، مقسّمة حسب الموضوع، وبعدها مرجع لكل Flag وكل علامة بتظهر في الأوامر ومعناها.

Cluster & Context

الـ Cluster والإعدادات

`minikube start`	تشغيل Cluster محلي
`minikube start --profile=dev`	Cluster تاني باسم dev
`minikube status`	حالة الـ Cluster
`kubectl cluster-info`	عنوان الـ API Server
`kubectl get nodes -o wide`	الـ Nodes بتفاصيلها
`kubectl config get-contexts`	كل الـ Contexts (Clusters)
`kubectl config use-context dev`	التبديل لـ Cluster تاني
`kubectl config set-context --current --namespace=dev`	تغيير الـ Namespace الافتراضية

Pods

الـ Pods

`kubectl run web --image=nginx:1.27`	إنشاء Pod بسرعة
`kubectl get pods -o wide`	عرض الـ Pods بالـ IP والـ Node
`kubectl get pods --show-labels`	عرض الـ Labels
`kubectl describe pod web`	التفاصيل والـ Events
`kubectl exec -it web -- sh`	الدخول جوه الـ Container
`kubectl delete pod web`	مسح Pod
`kubectl top pod`	الاستهلاك الحالي (محتاج metrics-server)

Apply & YAML

الملفات

`kubectl apply -f app.yaml`	إنشاء أو تحديث من ملف
`kubectl apply -f ./k8s/`	كل الملفات في فولدر
`kubectl delete -f app.yaml`	مسح اللي في الملف
`kubectl diff -f app.yaml`	الفرق بين الملف واللي في الـ Cluster
`kubectl create deploy web --image=nginx --dry-run=client -o yaml > d.yaml`	توليد YAML جاهز
`kubectl explain deployment.spec.strategy`	شرح أي حقل في الـ YAML
`kubectl edit deploy web`	تعديل مباشر

Deployments

الـ Deployments والـ ReplicaSets

`kubectl get deploy,rs`	عرض الـ Deployments والـ ReplicaSets
`kubectl scale deploy web --replicas=5`	تغيير العدد
`kubectl set image deploy/web web=nginx:1.28`	تحديث الـ image (Rolling Update)
`kubectl rollout status deploy/web`	متابعة التحديث
`kubectl rollout history deploy/web`	النسخ اللي فاتت
`kubectl rollout undo deploy/web`	الرجوع للنسخة اللي قبلها
`kubectl rollout restart deploy/web`	إعادة تشغيل كل الـ Pods

Labels & Namespaces — التنظيم

```
kubectl get pods -l app=web
```
فلترة بالـ Label

```
kubectl label pod web tier=frontend
```
إضافة Label

```
kubectl label pod web tier=backend --overwrite
```
تغيير Label موجود

```
kubectl label pod web tier-
```
مسح Label

```
kubectl get ns
```
عرض الـ Namespaces

```
kubectl create ns dev
```
إنشاء Namespace

```
kubectl get pods -A
```
من كل الـ Namespaces

Workloads — Jobs و DaemonSet

```
kubectl get ds -n kube-system
```
الـ DaemonSets

```
kubectl label nodes node-1 hardware=gpu
```
Label على Node

```
kubectl get jobs,cronjobs
```
الـ Jobs والـ CronJobs

```
kubectl create job test --from=cronjob/backup
```
تشغيل CronJob دلوقتي

Networking — الـ Services والـ Ingress

```
kubectl expose deploy web --port=80 --target-port=8080
```
عمل Service

```
kubectl get svc,endpoints
```
الـ Services والـ Endpoints

```
kubectl port-forward svc/web 8080:80
```
نفق من جهازك لـ Service

```
minikube service web --url
```
رابط Service على minikube

```
minikube addons enable ingress
```
تفعيل Ingress Controller

```
kubectl get ingress
```
عرض الـ Ingress

```
kubectl get certificate
```
حالة شهادات cert-manager

Config & Storage — الإعدادات والتخزين

```
kubectl create configmap app --from-literal=K=V
```
ConfigMap من قيم

```
kubectl create configmap ng --from-file=nginx.conf
```
ConfigMap من ملف

```
kubectl create secret generic db --from-literal=PASS=x
```
Secret عام

```
kubectl create secret tls t --cert=c.crt --key=k.key
```
Secret للشهادات

```
kubectl get secret db -o jsonpath='{.data.PASS}' | base64 -d
```
قراية قيمة Secret

```
kubectl get pv,pvc
```
التخزين الدائم

Security & Scaling — الصلاحيات والـ Scaling

```
kubectl auth can-i delete pods --as=ahmed -n dev
```
اختبار صلاحية

```
kubectl create sa app-sa
```
ServiceAccount

```
kubectl create rolebinding rb --clusterrole=view --user=ahmed -n dev
```
إعطاء صلاحية

```
kubectl autoscale deploy web --cpu-percent=50 --min=2 --max=10
```
عمل HPA

```
kubectl get hpa -w
```
متابعة الـ HPA

Troubleshooting — التشخيص

```
kubectl logs web -f
```
متابعة الـ logs لايف

```
kubectl logs web --previous
```
logs الـ Container اللي مات

```
kubectl logs web -c sidecar --tail=50
```
Container معين، آخر 50 سطر

```
kubectl get events --sort-by=.lastTimestamp
```
الأحداث بالترتيب

```
kubectl debug -it web --image=busybox
```
Container مؤقت للتشخيص

Helm	الحزم
`helm repo add NAME URL && helm repo update`	إضافة مستودع وتحديثه
`helm install rel repo/chart -n ns --create-namespace`	تسطيب
`helm upgrade --install rel ./chart -f values.yaml`	ترقية أو تسطيب
`helm list -A`	الـ Releases
`helm rollback rel 1`	الرجوع لنسخة
`helm template rel ./chart`	عرض الـ YAML النهائي
`helm uninstall rel`	مسح

1.أ مرجع الـ Flags

الـ Flag	الاسم الطويل	المعنى
`-f`	`--filename`	الملف أو الفولدر اللي فيه الـ YAML
`-n`	`--namespace`	الـ Namespace اللي هيتنفّذ فيها الأمر
`-A`	`--all-namespaces`	كل الـ Namespaces
`-o wide`	`--output`	أعمدة زيادة (IP و Node)
`-o yaml / json`	`--output`	الـ Object كامل بصيغة YAML أو JSON
`-o jsonpath='{...}'`	`--output`	حقل واحد بس من الـ Object
`-l key=value`	`--selector`	فلترة بالـ Labels
`--show-labels`	—	عمود فيه الـ Labels
`-w`	`--watch`	متابعة التغييرات لايف
`-it`	`-i -t`	تفاعلي: input مفتوح + Terminal
`--`	—	فاصل: اللي بعده أمر جوه الـ Container
`-c NAME`	`--container`	اختيار Container معين في الـ Pod
`-p`	`--previous`	logs الـ Container اللي مات
`--tail=N`	—	آخر N سطر

`--sort-by=PATH`	—	ترتيب النتايج بحقل
`--dry-run=client`	—	جرّب من غير ما تعمل حاجة في الـ Cluster
`--image=`	—	الـ image للـ Pod أو الـ Deployment
`--port / --target-port`	—	بورت الـ Service / بورت التطبيق
`--replicas=N`	—	عدد النسخ
`--overwrite`	—	استبدال Label موجود
`--rm`	—	امسح الـ Pod بعد ما تخرج
`--as=USER`	—	نفّذ كأنك المستخدم ده (للاختبار)
`--from-literal / --from-file`	—	مصدر قيم الـ ConfigMap أو الـ Secret
`--set a.b=v`	`(helm)`	تغيير قيمة في الـ values، النقطة = مستوى أعمق

جدول أ.1 أشهر الـ Flags في kubectl ومعناها

أ.2 مرجع العلامات (Symbols)

العلامة	المعنى	مثال
`|`	Pipe: ناتج الأمر الأول يدخل للتاني	`kubectl get pods | grep web`
`$( )`	نفّذ اللي جوه وحط ناتجه مكانه	`curl -LO .../$(curl -Ls .../stable.txt)/...`
`\`	الأمر مكمّل في السطر اللي تحت	`kubectl create cm a \`
`&&`	نفّذ التاني لو الأول نجح	`helm repo add ... && helm repo update`
`>`	احفظ الناتج في ملف (يكتب فوقه)	`... -o yaml > d.yaml`
`>>`	ضيف الناتج في آخر الملف	`date >> log.txt`
`#`	تعليق: اللي بعده مش بيتنفّذ	`# comment`
`;`	افصل بين أوامر في سطر واحد	`sleep 5; date`
`' '`	نص حرفي، الـ Shell مش بيفسّر جواه حاجة	`'{.data.PASS}'`
`<name>`	مكان تكتب فيه قيمتك (مش بتتكتب كده)	`kubectl describe pod <name>`
`type/name`	نوع واسم في كلمة واحدة	`svc/web , deploy/web`
`a,b`	أكتر من نوع في أمر واحد	`kubectl get deploy,rs`

جدول أ.2 العلامات اللي بتظهر في الأوامر ومعناها

ملحق ب

قاموس المصطلحات

كل مصطلح مهم في الكتاب في سطر واحد، مترتب أبجدياً. ارجعله كل ما تقابل كلمة ومش فاكر معناها.

A

accessModes
طريقة الوصول للتخزين: RWO و ROX و RWX و RWOP.

Annotation
معلومة إضافية على الـ Object بيقراها أدوات تانية (زي cert-manager)، ومش بتستخدم في الاختيار زي الـ Labels.

API Server
بوابة الـ Cluster: كل الأوامر (kubectl وغيره) بتكلمه، وهو اللي بيكلم الـ etcd.

B

base64
ترميز بيحوّل أي بيانات لنص. مش تشفير.

C

CA
Certificate Authority: جهة موثوقة بتختم الشهادات.

cert-manager
أداة بتجيب شهادات TLS (زي Let's Encrypt) وتجددها أوتوماتيك.

Chart
حزمة Helm: values.yaml + Templates.

ClusterIP
نوع Service افتراضي: عنوان ثابت جوه الـ Cluster بس.

ClusterRole
Role على مستوى الـ Cluster كله.

CNI
الـ Plugin المسؤول عن شبكة الـ Pods وإعطاءهم IPs (زي Calico و Cilium).

ConfigMap
Object بيخزن إعدادات غير سرية منفصلة عن الـ image.

Container Runtime
البرنامج اللي بيشغّل الـ Containers فعلاً (containerd، CRI-O).

Context
مجموعة (Cluster + User + Namespace) في الـ kubeconfig.

Controller Manager
بيشغّل الـ Controllers اللي بتقارن الحالة الحالية بالمطلوبة وتصلّح.

CRD
Custom Resource Definition: نوع Object جديد بتضيفه أداة على Kubernetes.

CronJob
بيعمل Job على ميعاد متكرر.

D

DaemonSet
بيشغّل Pod واحدة على كل Node (أو Nodes مختارة).

Declarative
توصف الحالة المطلوبة وKubernetes يوصلها.

Deployment
بيدير ReplicaSets ويعمل Rolling Update و Rollback.

E

emptyDir
Volume مؤقت بيعيش مع الـ Pod.

Endpoints
قايمة الـ IPs الـ Pods اللي الـ Service بتبعتلهم.

etcd
قاعدة البيانات اللي فيها حالة الـ Cluster كلها.

Events
سجل الأحداث اللي حصلت لـ Objects، بيظهر في describe.

G

Gateway API
الجيل الأحدث بعد الـ Ingress لإدارة الـ Traffic الداخل.

H

Helm
مدير الحزم لـ Kubernetes.

hostPath
Volume بيركّب فولدر من الـ Node نفسها.

HPA
Horizontal Pod Autoscaler: بيغير عدد الـ Pods حسب الاستهلاك.

I

Ingress
قواعد بتوجّه HTTP/HTTPS حسب الـ Host والـ Path للـ Services.

Ingress Controller
البرنامج اللي بينفّذ قواعد الـ Ingress فعلاً.

J

Job
بيشغّل Pod لحد ما تخلص بنجاح.

K

kube-proxy
بيظبط قواعد الشبكة على كل Node عشان الـ Services تشتغل.

kubectl
أداة الأوامر اللي بتكلم الـ API Server.

kubelet
الـ Agent على كل Node، بيشغّل الـ Pods ويتابعها.

L

Label
value=key على الـ Object بيستخدم في الاختيار والتنظيم.

LimitRange
قيم requests/limits افتراضية وحدود لكل Container في Namespace.

Limits
أقصى موارد ممكن الـ Container يستخدمها.

Liveness Probe
فحص: الـ Container عايش؟ لو لأ يتعمله restart.

LoadBalancer
نوع Service بياخد IP خارجي من الـ Cloud.

M

metrics-server
بيجمع استهلاك الـ CPU والـ Memory للـ HPA و kubectl top.

minikube
Cluster محلي للتجربة على جهازك.

N

Namespace
تقسيم منطقي جوه الـ Cluster.

Node
جهاز (حقيقي أو VM) في الـ Cluster.

Node Affinity
نسخة أقوى من الـ nodeSelector بشروط In و NotIn و Exists.

NodePort
نوع Service بيفتح بورت (30000-32767) على كل Node.

nodeSelector
تشغيل الـ Pod على Nodes عليها Label معينة.

O

OOMKilled
الـ Container اتقتل لأنه عدّى الـ memory limit.

P

PersistentVolume (PV)
مساحة تخزين حقيقية في الـ Cluster.

Pod
أصغر وحدة في Kubernetes: Container أو أكتر بيشاركوا الشبكة والتخزين.

PVC
طلب تخزين بيرتبط بـ PV.

Q

QoS Class
Guaranteed و Burstable و BestEffort: بتحدد مين يتطرد الأول.

R

RBAC
نظام الصلاحيات: Roles و Bindings.

Readiness Probe
فحص: الـ Container جاهز ياخد Traffic؟

Reconciliation Loop
المقارنة المستمرة بين الحالة المطلوبة والحالية وتصليح الفرق.

Release
نسخة متسطبة من Chart.

ReplicaSet
بيضمن عدد ثابت من الـ Pods.

Requests
الموارد المحجوزة للـ Container، والـ Scheduler بيعتمد عليها.

ResourceQuota
حد أقصى لموارد وعدد Objects في Namespace.

Rolling Update
تحديث تدريجي من غير توقف.

S

Scheduler
بيختار الـ Node المناسبة لكل Pod.

Secret
Object للبيانات الحساسة (base64، مش تشفير).

Selector
شرط بيختار Objects بالـ Labels.

Service
عنوان ثابت و Load Balancing لمجموعة Pods.

ServiceAccount
هوية للـ Pods لما تكلم الـ API.

Sidecar
Container مساعد جنب الأساسي في نفس الـ Pod.

Startup Probe
فحص للتطبيقات البطيئة في البداية، بيأجّل الـ liveness.

StatefulSet
زي الـ Deployment بس كل Pod ليه اسم وتخزين ثابت.

StorageClass
بيعمل PVs أوتوماتيك (Dynamic Provisioning).

T

Taint / Toleration
Taint على الـ Node بيمنع الـ Pods، والـ Toleration تصريح بالدخول.

TLS Termination
فك تشفير الـ HTTPS عند الـ Ingress.

V

Volume
تخزين بيتركّب جوه الـ Container من برّه.

Y

YAML
صيغة الملفات اللي بنوصف بيها الـ Objects.

ملحق ج

إجابات أسئلة المراجعة و صح ولا غلط

حاول تجاوب على الأسئلة بنفسك الأول، وبعدين قارن إجابتك هنا. لكل فصل: إجابات أسئلة المراجعة، وبعدها إجابات صح ولا غلط.

الفصل 1: المقدمة

1. إدارة يدوية مستحيلة، الـ Container لو وقع محدش يقومه، توزيع الحمل والـ Scaling، والتحديث من غير توقف.
2. مجموعة Nodes شغالين مع بعض كوحدة واحدة. الـ Node جهاز، والـ Pod أصغر وحدة شغل بتتشغل على Node.
3. إنك توصف الحالة اللي عايزها (3 نسخ من nginx) وKubernetes يوصلها ويحافظ عليها، بدل ما تقوله الخطوات.
4. الـ Cluster Autoscaler (أو خدمة الـ Cloud)، مش Kubernetes لوحده.

صح ولا غلط: (1) غلط: بيتعامل مع الـ Pods. · (2) صح. · (3) غلط: محتاج Cluster Autoscaler.

الفصل 2: الـ Structure

1. الـ API Server.
2. حالة الـ Cluster كلها بتضيع، فلازم Backup دوري لـ etcd.
3. الـ Scheduler بيختار، والـ kubelet على الـ Node بيشغّل عن طريق الـ Container Runtime.
4. Observe (يشوف الحالي) ← Compare (يقارن بالمطلوب) ← Act (يصلّح الفرق)، وتتكرر على طول.

صح ولا غلط: (1) صح. · (2) غلط: بيستخدم Runtime زي containerd. · (3) غلط: ده شغل الـ kubelet والـ Controllers.

الفصل 3: الـ Pods

1. Pods، لأن الـ Pod وحدة الـ Scaling، والـ Containers جوه الـ Pod بيتعاملوا كوحدة واحدة.
2. على `localhost` لأنهم بيشاركوا نفس الشبكة.
3. Container بيجمع الـ logs أو Proxy جنب التطبيق.
4. الـ Container لما يقع الـ kubelet بيقومه جوه نفس الـ Pod، لكن الـ Pod لما يتمسح بيروح خالص ومحتاج Controller يعمل واحد جديد.

صح ولا غلط: (1) غلط: بيشاركوا IP واحد. · (2) غلط: محتاج Controller زي ReplicaSet. · (3) صح.

الفصل 4: minikube

1. `minikube start --profile <name>`.
2. فيه 2 Containers في الـ Pod، واحد بس جاهز.
3. `kubectl run web --image=nginx:1.27`
4. لأنها Pods لوحدها من غير Controller، ومحدش هيقومها لو اتمسحت؛ في الشغل بنستخدم Deployment.

صح ولا غلط: (1) صح. · (2) غلط: مسافات بس. · (3) صح.

الفصل 5: الـ Probes

1. `exec` أو `tcpSocket`.
2. لأن لو الـ DB وقعت كل الـ Pods هتعمل restart على الفاضي والمشكلة مش فيهم؛ ده مكانه الـ Readiness.
3. الـ Startup Probe، لأنها بتستنى البداية بس، وبعدها الـ liveness تشتغل بسرعة بدل تأخير ثابت دايماً.
4. بيتشال من الـ Endpoints فمياخدش Traffic، من غير restart، ويرجع لما ينجح.

صح ولا غلط: (1) غلط: بيتشال من الـ Endpoints بس. · (2) غلط: لازم 1 في الـ liveness. · (3) صح.

الفصل 6: الـ Resources

1. الـ Memory: يتقتل OOMKilled. الـ CPU: يتعمله Throttling (يبطأ) من غير ما يتقتل.
2. لأنه بيختار Node فيها مكان فاضي على حسب الـ requests.
3. `Mi` = 1024×1024 بايت، و `M` = 1000×1000 بايت.
4. Guaranteed.

صح ولا غلط: (1) صح. · (2) غلط: يقدر لحد الـ limit. · (3) صح (BestEffort).

الفصل 7: الـ Labels

1. لأن الاسم فريد لكل Pod، والـ Service محتاجة تختار مجموعة متغيرة.
2. `app: myapp` ، `tier: backend` ، `env: staging` .
3. لأ، الـ Labels حساسة لحالة الحروف.
4. `kubectl get pods -l tier=backend`

صح ولا غلط: (1) غلط. · (2) صح. · (3) غلط: في الـ metadata.

الفصل 8: الـ ReplicaSets

1. عشان الـ High Availability وتوزيع الحمل.
2. هتعمل Pod جديدة عشان ترجع 3.
3. لازم يطابقوا، وإلا الـ ReplicaSet مش هتلاقي الـ Pods بتاعتها (والـ API بيرفضها).

صح ولا غلط: (1) غلط. · (2) صح. · (3) غلط: محتاج Deployment.

الفصل 9: الـ Namespaces

1. `api.prod` أو كامل `api.prod.svc.cluster.local` .
2. لأنها لمكونات Kubernetes نفسها، وأي غلطة فيها ممكن توقع الـ Cluster.
3. الـ Pod هيترفض؛ حط resources أو LimitRange بقيم افتراضية.
4. `kubectl config set-context --current --namespace=staging`

صح ولا غلط: (1) غلط. · (2) صح. · (3) غلط: محتاج NetworkPolicy.

الفصل 10: الـ Deployment

1. لأنها مش بتحدّث Pods موجودة، والـ Deployment بيعمل ReplicaSet جديدة.
2. بيعمل ReplicaSet جديدة ويزوّدها تدريجياً ويقلل القديمة لحد صفر، والقديمة بتفضل للـ rollback.
3. `kubectl rollout undo deployment/<name>`
4. لما مينفعش نسختين يشتغلوا مع بعض (زي schema DB).

صح ولا غلط: (1) غلط: تدريجي. · (2) صح. · (3) غلط.

الفصل 11: الـ DaemonSet

1. لأن العدد = عدد الـ Nodes المناسبة.
2. `volumeMounts` = المسار جوه الـ Container، و `volumes` = المصدر على الـ Node (hostPath).
3. Node Affinity: `key: disk` ، `operator: In` ، `values: [ssd, nvme]` .
4. `kubectl describe ds` و `get pods -o wide` وشوف الـ taints.

صح ولا غلط: (1) صح. · (2) غلط. · (3) صح.

الفصل 12: الـ Jobs

1. لأن الـ Deployment هيعيد تشغيله بعد ما يخلص على طول.
2. `0 23 * * 5`
3. بعد ما الفشل يعدّي الـ backoffLimit (مع الفشل الرابع) الـ Job بيتعلن Failed ويبطل يحاول.
4. `kubectl create job test --from=cronjob/<name>`

صح ولا غلط: (1) غلط. · (2) صح. · (3) صح.

الفصل 13: الـ Services

1. لأن الـ IP الـ Pod بيتغير كل ما يتعمل من جديد.
2. الـ selector مش مطابق للـ Labels (أو الـ Pods مش Ready).
3. `port: 80` ، `targetPort: 3000` .
4. مفيش Cloud يدي IP؛ استخدم `minikube tunnel` .

صح ولا غلط: (1) غلط: الـ ClusterIP. · (2) غلط: المدى `30000-32767` . · (3) صح.

الفصل 14: الـ Ingress

1. Load Balancer واحد بدل واحد لكل Service.
2. الـ Resource قواعد، والـ Controller البرنامج اللي بينفذها.
3. `Prefix` .
4. `host: shop.example.com` ← `path: /` ← `service: shop-service` port 80.

صح ولا غلط: (1) غلط: لازم تسطّب واحد. · (2) صح. · (3) غلط.

الفصل 15: الـ TLS

1. لأنه تقيل على الـ CPU جداً.
2. لأنها مختومة من CA المتصفح واثق فيها.
3. بتقول للـ cert-manager يعمل شهادة من الـ ClusterIssuer ده.
4. عشان متتحظرش من الـ Rate Limits وإنت بتجرب.

صح ولا غلط: (1) غلط: بيفضل مع السيرفر بس. · (2) صح. · (3) صح.

الفصل 16: الـ Volumes

1. `volumes` المصدر في الـ Pod، و `volumeMounts` مكان التركيب في الـ Container.
2. لأن البيانات على Node واحدة، ولو الـ Pod راح Node تانية مش هيلاقيها.
3. `describe pvc` : StorageClass غلط أو مفيش PV مناسب.
4. لما البيانات مهمة ومينفعش تتمسح مع الـ PVC.

صح ولا غلط: (1) غلط. · (2) غلط: Node واحدة. · (3) صح.

الفصل 17: الـ ConfigMaps

1. عشان تغير القيم في مكان واحد من غير ما تبني image جديدة.
2. `valueFrom` متغير واحد باسم تختاره، و `envFrom` كل الـ keys.
3. الـ env بتتقري وقت التشغيل بس؛ اعمل `rollout restart` .

صح ولا غلط: (1) غلط: Secret. · (2) صح. · (3) صح.

الفصل 18: الـ Secrets

1. لأنه ترميز أي حد يقدر يفكه.
2. `data` قيم base64، و `stringData` نص عادي.
3. عشان الـ newline ميتحسبش جزء من القيمة.
4. RBAC، و Encryption at Rest (أو أدوات زي External Secrets).

صح ولا غلط: (1) غلط. · (2) صح. · (3) صح.

الفصل 19: الـ RBAC

1. الـ Role لـ Namespace، والـ ClusterRole للـ Cluster كله.
2. لأن الـ pods في الـ core group واسمه فاضي.
3. `resources: ["pods", "pods/log"]` ، `verbs: ["get", "list"]` .
4. `kubectl auth can-i list secrets --as=system:serviceaccount:ns:name`

صح ولا غلط: (1) غلط. · (2) صح. · (3) غلط.

الفصل 20: الـ HPA

1. مفيش metrics-server، أو مفيش CPU requests.
2. `ceil(3 × 90 / 60) = ceil(4.5) = 5` Pods.
3. الـ HPA بيزوّد Pods، والـ Cluster Autoscaler بيزوّد Nodes.

صح ولا غلط: (1) غلط: الـ requests. · (2) صح. · (3) غلط.

الفصل 21: الـ Troubleshooting

1. `kubectl logs <pod> --previous` و `describe` .
2. الـ Container اتقتل بـ SIGKILL، وغالباً OOMKilled.
3. الـ selector مش مطابق، أو الـ Pods مش Ready.
4. لما الـ image مفيهاش shell (distroless).

صح ولا غلط: (1) غلط: Scheduling. · (2) صح. · (3) صح.

الفصل 22: الـ Helm

1. الـ Chart الحزمة، والـ Release نسخة متسطبة منها.
2. `--set` .
3. `helm history` وبعدين `helm rollback <release> <rev>` .
4. يوريك الـ YAML النهائي من غير تسطيب.

صح ولا غلط: (1) صح. · (2) غلط: بيعمل revision جديدة. · (3) صح.

بالتوفيق في المذاكرة

أحسن طريقة تثبّت بيها اللي اتعلمته: شغّل minikube وجرّب كل مثال بإيدك، وكل ما حاجة تبوظ ارجع لفصل الـ Troubleshooting.

MOAZ ABDEL-FATTAH